CHRocodileLib

Support Library for CHRocodile Device Connections

Application Programming Interface Reference

Version 5.1.0.3 (created on 2023-05-25)

© Precitec Optronik GmbH

This documentation is a copyrighted work of Precitec Optronik GmbH.

It may not be reproduced or used in a manner contrary to the company's legal interests without prior written approval of Precitec Optronik GmbH. It is strictly intended for use in the context of service operations. Any other use is impermissible. Any sharing of this documentation with third parties requires the prior express written approval of Precitec Optronik GmbH.

Changes in the technical details from the descriptions, data and figures in this documentation are reserved.

Printed in the Federal Republic of Germany.

Document applies to API version: 5.1.0.3

Responsible for contents:

Precitec Optronik GmbH
Schleussnerstraße 54
63263 Neu-Isenburg
Deutschland

Telephone: +49 6102 / 36 76 - 100
Telefax: +49 6102 / 36 76 - 126
e-mail: info@precitec-optronik.de
Website: www.precitec.com

Contents

I Introduction 2

1 Introduction 3
- 1.1 CHRocodileLib Feature Summary . . . 3
  - 1.1.1 What is the CHRocodile library? . . . 3
  - 1.1.2 CHRocodile core library feature summary . . . 3
  - 1.1.3 Basic Concepts . . . 4
- 1.2 Quick Start (C-API) . . . 6

2 Different modes of operation 7
- 2.1 Synchronous mode . . . 7
- 2.2 Asynchronous Mode . . . 8
  - 2.2.1 Designing applications in asynchronous mode . . . 11
- 2.3 Making use of `_pUser`, the user context pointer . . . 12

3 Using Multiple Threads 14

4 Sharing Device Connections 15

5 Plugins 17

6 Ini File 18

II API Reference 20

7 Exported functions and data types 21
- 7.1 Scalar type Cmd_h . . . 21
- 7.2 Scalar type Conn_h . . . 21
- 7.3 Scalar type FilePath_t . . . 21
- 7.4 Scalar type Handle_t . . . 21
- 7.5 Scalar type LibSize_t . . . 21
- 7.6 Scalar type Plugin_h . . . 21
- 7.7 Scalar type Rsp_h . . . 22
- 7.8 Structure TErrorInfo . . . 23
- 7.9 Structure TPluginInfo . . . 24
- 7.10 Structure TResponseInfo . . . 25
- 7.11 Structure TRspCallbackInfo . . . 26
- 7.12 Structure TSampleSignalGeneralInfo . . . 27
- 7.13 Structure TSampleSignalInfo . . . 28
- 7.14 Callback ResponseAndUpdateCallback . . . 29
- 7.15 Callback SampleDataCallback . . . 30
- 7.16 Function ActivateAsyncDataUserBuffer() . . . 31
- 7.17 Function ActivateAutoBufferMode() . . . 33
- 7.18 Function AddCommandBlobArg() . . . 35
- 7.19 Function AddCommandFloatArg() . . . 36
- 7.20 Function AddCommandFloatArrayArg() . . . 37
- 7.21 Function AddCommandIntArg() . . . 38
- 7.22 Function AddCommandIntArrayArg() . . . 39
- 7.23 Function AddCommandStringArg() . . . 40

 © Precitec Optronik GmbH

7.24 Function AddConnectionPlugIn() . . . 41
7.25 Function AddConnectionPlugInByID() . . . 42
7.26 Function CancelCHRDeviceAutoSearch() . . . 43
7.27 Function ClearErrors() . . . 44
7.28 Function CloseConnection() . . . 45
7.29 Function CmdNameToID() . . . 46
7.30 Function DeactivateAsyncDataUserBuffer() . . . 47
7.31 Function DeactivateAutoBufferMode() . . . 48
7.32 Function DestroyHandle() . . . 49
7.33 Function DetectedCHRDeviceInfo() . . . 50
7.34 Function ErrorCodeToString() . . . 51
7.35 Function ExecCommand() . . . 52
7.36 Function ExecCommandAsync() . . . 53
7.37 Function FlushConnectionBuffer() . . . 54
7.38 Function GetAsyncDataUserBufferSavedSampleCount() . . . 55
7.39 Function GetAutoBufferSavedSampleCount() . . . 56
7.40 Function GetAutoBufferStatus() . . . 57
7.41 Function GetConnectionMode() . . . 58
7.42 Function GetConnectionOutputSignalInfos() . . . 59
7.43 Function GetConnectionPlugInInfo() . . . 60
7.44 Function GetCurrentAvailableSampleCount() . . . 61
7.45 Function GetDeviceChannelCount() . . . 62
7.46 Function GetDeviceConnectionInfo() . . . 63
7.47 Function GetDeviceOutputSignals() . . . 64
7.48 Function GetDeviceType() . . . 65
7.49 Function GetHandleType() . . . 66
7.50 Function GetLastSample() . . . 67
7.51 Function GetNextSamples() . . . 69
7.52 Function GetOutputDataFormatMode() . . . 71
7.53 Function GetResponseArgType() . . . 72
7.54 Function GetResponseBlobArg() . . . 73
7.55 Function GetResponseFloatArg() . . . 74
7.56 Function GetResponseFloatArrayArg() . . . 75
7.57 Function GetResponseInfo() . . . 76
7.58 Function GetResponseIntArg() . . . 77
7.59 Function GetResponseIntArrayArg() . . . 78
7.60 Function GetResponseStringArg() . . . 79
7.61 Function GetSingleOutputSampleSize() . . . 80
7.62 Function IsCHRDeviceAutoSearchFinished() . . . 81
7.63 Function LastErrors() . . . 82
7.64 Function NewCommand() . . . 83
7.65 Function NewCommandFromString() . . . 84
7.66 Function OpenConnection() . . . 85
7.67 Function OpenSharedConnection() . . . 87
7.68 Function PerformLibOperation() . . . 88
7.69 Function PluginTypeIDToName() . . . 89
7.70 Function ProcessDeviceOutput() . . . 90
7.71 Function RegisterGeneralResponseAndUpdateCallback() . . . 92
7.72 Function RegisterSampleDataCallback() . . . 93
7.73 Function ResponseToString() . . . 95
7.74 Function SetIniFile() . . . 96
7.75 Function SetLibConfigFlags() . . . 97
7.76 Function SetLibLogFileDirectory() . . . 98
7.77 Function SetLibLogLevel() . . . 99
7.78 Function SetMultiChannelProfileInterpolation() . . . 100
7.79 Function SetOutputDataFormatMode() . . . 101
7.80 Function SetSampleDataBufferSize() . . . 102
7.81 Function StartAutomaticDeviceOutputProcessing() . . . 103
7.82 Function StartCHRDeviceAutoSearch() . . . 104

© Precitec Optronik GmbH

7.83 Function StartConnectionDataStream() . . . 105
7.84 Function StopAutomaticDeviceOutputProcessing() . . . 106
7.85 Function StopConnectionDataStream() . . . 107
7.86 Function UploadConfocalCalibrationTableFromFile() . . . 108
7.87 Function UploadFirmware() . . . 109
7.88 Function UploadMultiChannelMaskTable() . . . 110
7.89 Function UploadRefractiveIndexTableFromFile() . . . 111
7.90 Function UploadWaveLengthTableFromFile() . . . 112

A Predefined Constants 113
A.1 Result Codes . . . 113
A.2 Constants: Handle Types . . . 115
A.3 Constants: Sample Data Types . . . 115
A.4 Constants: Response flag . . . 115
A.5 Constants: Device Response Parameter Types . . . 115
A.6 Constants: Connection Constants . . . 115
A.7 Constants: Connection Mode . . . 115
A.8 Constants: Library Data Formats . . . 115
A.9 Constants: Automatic Buffer Saving Constants . . . 116
A.10 Constants: Return value of data read . . . 116
A.11 Constants: Device Command Specifier ID Constants . . . 117
A.12 Constants: Spectra Types . . . 117
A.13 Constants: Table Types . . . 118
A.14 Constants: Device Trigger Modes . . . 118
A.15 Constants: Encoder Counter Input Source . . . 118
A.16 Constants: Encoder Preload Event Property Constants . . . 118
A.17 Constants: Library Configuration Flags . . . 118
A.18 Constants: Device Search Constants . . . 118
A.19 Constants: Performance save setting . . . 119

B The CHRocodileLib API grouped by topic 120
B.1 Asynchronous Mode . . . 120
B.2 Automatic Buffer Saving . . . 120
B.3 Automatic Device Output Processing . . . 121
B.4 Automatic Device Search . . . 121
B.5 Callback Functions . . . 121
B.6 Command Response . . . 121
B.7 Connection . . . 122
B.8 Connection Information . . . 122
B.9 DataCallback Function . . . 122
B.10 Device Command . . . 123
B.11 Device Information . . . 123
B.12 Error Handling . . . 123
B.13 Handles . . . 124
B.14 Library Configuration . . . 124
B.15 Plugins . . . 124
B.16 Sample Data . . . 125
B.17 Synchronous Mode . . . 125
B.18 Update and Response Callback Function . . . 125

C CHRocodileLib Internal Commands 126
C.1 Encoder Counter Value Command . . . 126
C.2 Encoder Counter Source Command . . . 126
C.3 Encoder Preload function Command . . . 127
C.4 Encoder Trigger Enable Command . . . 127
C.5 Encoder Trigger Property Command . . . 128
C.6 Device Trigger Mode Command . . . 128

 © Precitec Optronik GmbH

III Programming Language Bindings 129

D C# 131
D.1 Preface . . . 131
D.2 Architecture . . . 131
D.2.1 Connection Objects . . . 132
D.2.2 Command Objects . . . 132
D.2.3 Response Objects . . . 132
D.2.4 Updates . . . 132
D.2.5 Data Objects . . . 133
D.3 Basic Usage . . . 133
D.3.1 Synchronous Mode . . . 133
D.3.2 Recording Data . . . 134
D.4 Asynchronous Mode . . . 135
D.4.1 Minimal Example . . . 135
D.4.2 A Super-Minimal Scanning Example . . . 136
D.4.3 SynchronousCommandGroup . . . 136
D.4.4 Executing Commands that generate Updates . . . 137
D.5 Data Objects . . . 141
D.5.1 Detaching Data Buffers . . . 141

E Python 144
E.1 Introduction . . . 144
E.2 Requirements . . . 144
E.3 Basic usage . . . 144
E.4 Command Response and Data objects . . . 146
E.5 Data Format . . . 146
E.6 Error handling . . . 147
E.7 Auto buffer mode . . . 147
E.8 Plugins . . . 148
E.9 Shared connections . . . 148
E.10 Performance Tips . . . 149
E.11 Good To Know . . . 149
E.12 Troubleshooting . . . 149
E.13 Support . . . 150

IV Plug-ins 151

F CLS2 Calibration Plugin 152
F.1 Calibration Correction Active Command . . . 152
F.1.1 Calibration Correction Mode Command . . . 152
F.1.2 Calibration File Name Command . . . 153
F.1.3 Calibration File List Command . . . 153
F.1.4 Resample Count Command . . . 153

G Flying Spot Scanner Plugin 155
G.1 Overview . . . 155
G.1.1 Flying Spot Scanner Operation . . . 155
G.2 The FSS Scripting Language . . . 156
G.2.1 Language design . . . 157
G.2.2 Case sensitivity . . . 157
G.2.3 The script's structure . . . 157
G.2.4 Mathematical expressions . . . 158
G.3 Built-in language constructs . . . 158
G.3.1 The optional `init` section . . . 158
G.3.2 The function `main()` . . . 159
G.3.3 User Functions . . . 159
G.3.4 Statements . . . 159

 © Precitec Optronik GmbH

G.3.5 Z-Correction for thickness signals . . . 161
G.3.6 The `shape` environment . . . 161
G.3.7 The `interp2D` environment . . . 162
G.3.8 DSP embedded control commands . . . 163
G.3.9 Rectangular scan interpolation algorithm details . . . 164
G.3.10 FSS Plugin usage . . . 167
G.3.10.1 Executing plugin script . . . 167
G.3.10.2 Configuration file . . . 167
G.3.10.3 How the plugin recognizes different figures and **setMarker** command . . . 168
G.4 Script Command Reference . . . 169
G.4.1 adaptiveExposure . . . 169
G.4.2 add . . . 170
G.4.3 avd . . . 170
G.4.4 avs . . . 170
G.4.5 clear . . . 170
G.4.6 correction . . . 170
G.4.7 doubleExposure . . . 171
G.4.8 ellipse . . . 171
G.4.9 enc . . . 171
G.4.10 exposure . . . 171
G.4.11 fn main . . . 172
G.4.12 help . . . 172
G.4.13 initSectionItem . . . 172
G.4.14 interp2D . . . 172
G.4.15 line . . . 173
G.4.16 lineTo . . . 173
G.4.17 moveTo . . . 173
G.4.18 qth . . . 173
G.4.19 rect . . . 173
G.4.20 setFrequency . . . 174
G.4.21 setMarker . . . 174
G.4.22 shape . . . 174
G.4.23 spiral . . . 174
G.4.24 startMeasure . . . 175
G.4.25 stopMeasure . . . 175
G.4.26 userFunc . . . 175
G.4.27 waitExtAxisPos . . . 175
G.4.28 waitTrigger . . . 175
G.4.29 waitUsec . . . 175
G.4.30 while . . . 176

 © Precitec Optronik GmbH

Part I

Introduction

© Precitec Optronik GmbH

Chapter 1

Introduction

1.1 CHRocodileLib Feature Summary

This manual is part of the Precitec documentation set on CHRocodile device programming (guiding people who want to access CHRocodile devices from within their own software.)

It introduces you to using the CHRocodile core library and provides a thorough reference part describing all functions of the library and their usage.

For using the library, a deeper understanding of the commands that a given device supports and their use is required. → See command reference of device

1.1.1 What is the CHRocodile library?

The CHRocodile core library provides a comprehensive set of functions that can be used from programs in any programming language that can call C-language functions. The core library can also be extended by plug-in modules to add specific functionality, such as dedicated scanning features for use in conjunction with a FSS device.

While the core library's functions are both versatile and flexible, using them is less convenient than working with language-specific interfaces/wrappers. The language-specific interfaces, which are based on the core library, allow software coding with dedicated features and mechanisms of a specific programming language (such as C# or Python). In most cases, using such an interface is more convenient or straight forward than directly calling C API functions of the core library; however, a language-specific interface might not provide access to all features covered by the core library. In this case, direct use of the library functions is required.

You should prefer using the core library over implementing the device communication protocol yourself, which is more error-prone and requires a thorough understanding of the device, its features, and commands.

1.1.2 CHRocodile core library feature summary

- **Multi-sensor Family Support**
  This library supports all families of Precitec CHRocodile optical measurement devices (i.e. sensors): CHRocodile 1 (first generation sensors), CHRocodile 2 (second generation sensors), CHRocodile Mini, CHRocodile C (CHRocodile Compact), and CLS/MPS for which it provides a consistent, uniform programming interface, facilitating porting applications to different device families.
- **Wide Range of Functions**
  Including service functions such as device-family-specific table upload
- **Multi-device Support**
  Multiple devices can be operated in parallel and independently

 © Precitec Optronik GmbH

- **Multi-connection Support (only one physical connection)**
  A single device (of any[1] family) can be operated by multiple library-hosted logical connections, where each connection can send commands, receive responses, updates, and data independently (or named as shared connection). This is internally realized by means of effective packet buffering and serialization, and can be used directly to support applications with multiple controls/data views.

- **Synchronous/Asynchronous Sensor Control**
  Most of device query and control functions can be executed in either synchronous or asynchronous mode.

  – **Synchronous mode:** Send sensor command and wait for response (blocking call)
  – **Asynchronous mode:** Send commands asynchronously with callback registration for responses (non-blocking call)

- **Multi-threading Support:**
  Is provided as follows:

  – For each connection: Both executing commands and receiving data can be done in different threads.
  – For multiple connections: Each connection can use its own thread(s). The connections are independent of each other.

- **Multi-platform Support**
  CHRocodileLib is available as:

  – Dynamic Link Library (`.dll`) for Windows 7, 8, 10
  – Shared Object (`.so`) for the Linux platform[2]

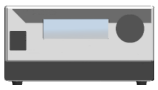
- **Extensibility**
  CHRocodileLib's functionality can be extended by means of plug-ins:

  – Plug-ins have access to (and can interfer with) a given connection's up and down stream (i. e. commands sent to the sensor as well as data received from the sensor). This way, they can act as custom sensor controllers and / or data filters.
  – Plug-ins are controlled by means of the same API used for sensor control
  – Compatible plug-ins can be chained to combine functionality
  – By means of ini files, plug-ins may be configured independently of the host application

1.1.3 Basic Concepts

To effectively use the core library, it is important to understand the concepts of connections, handles, commands, signals and some more.

Connections A connection represents one sensor and one client which basically exchange commands and data. Connections can be operated in a synchronous or asynchronous mode (see below for more details)



Commands, Command Responses Controlling CHRocodileLib and the sensor is done by creating a *command*, adding arguments to it and passing it to an execution function. Any results are wrapped into *response* objects that the execute function returns directly (synchronous mode) or posts later on by means of callback function invocations (asynchronous mode).

[1] including CHRocodile 1
[2] Please contact our sales team for details

Command Updates Updates are structurally identical to responses. The difference: Updates can be posted even *without prior commands* - triggered by external events, commands from other connections, or as side effects of other commands. Updates help the application keep up-to-date with the sensor state - automatically. They are the key feature that makes multiple connections in parallel even possible.

Samples and Signals CHRocodile sensors produce measurement data which is organized as samples which contain certain signal values. A *sample* is the set of results generated from one single exposure, taken at one point in time. A *signal* is a certain aspect of the measurement such as the distance of the target, the thickness of a given layer, but also other data such as the sample's timestamp or encoder counter values. The user can select signals of interest such that for each sample and each selected signal, one value is produced.

Synchronous mode In synchronous mode, you use ExecCommand() to have commands executed by the sensor, a plug-in or the library itself. This function *blocks* until the command has been completed. Data samples are being picked up by means of GetNextSamples() - or collected in a user-provided buffer that is filled by CHRocodileLib.

Asynchronous mode If that mode, you use ExecCommandAsync() which does not block. Instead, a callback function is used to receive the response later. Notifications of new data are transferred by means of callback functions, too.

Handles Given that CHRocodileLib communicates though a C API, objects, such as connections, commands, and responses to such commands, are managed within CHRocodileLib and accessed through handles, i. e. integer numbers that identify the objects behind them.

To free or not to free: Some objects must be disposed by the user (e. g. connections) others are freed automatically, e. g. commands which, when created, can only be used once or responses which have a strictly defined lifetime.

Tickets are integer numbers that are used in asynchronous mode to identify a command transaction. ExecCommandAsync() returns a ticket for each command. The same ticket is later passed into the callback function when the result has arrived.

Error handling Most functions of the CHRocodile Library will return a completion code indicating whether a function completed successfully, or has detected a warning or error condition. The completion code is a 32-Bit integer value

- A non-negative value indicates that the function has completed successfully.
- A negative value indicates that the function either detected a warning condition (meaning that the function was able to execute, however, the result may not as expected due to the condition detected), or was unable to execute due to an error condition.

The warning and error codes a library function can return are listed in Appendix **??**. You may also use the function ErrorCodeToString() to convert a warning or error code into human-readable text.

Plug-Ins CHRocodileLib functionality can be extended by means of plug-ins provided by Precitec. These are DLLs / shared object libraries that can be integrated into the CHRocodileLib's internal command / data streams to cover certain functionality such as FSS control.

© Precitec Optronik GmbH

1.2 Quick Start (C-API)

Connect As a starting point, to connect to a sensor, simply use the OpenConnection() function. It takes four arguments:

1. the connection specification, e.g. `COM15` or `/dev/ttyS0` for a serial connection on Windows and Linux respectively, or `192.168.170.2` for TCP/IP-based network connections[3]
2. the device type given as an integer enumeration
3. the device buffer size, i. e. the size of an internal circular buffer for temporary device output storage (defaulting to 32 MB when 0 is passed)
4. the pointer to store a connection handle

Upon successful connection, a connection handle is returned, which is used later for communication (samples, sensor control commands).

Select Signals After the connection is established, the next step is to select the measurement signals to acquire. This is done by creating an output signal command, adding required signals to the command and executing the command.

Receive Data Generally, the library first stores all device output in an internal circular buffer. From here, GetNextSamples() will read a number of samples into a user-visible data sample buffer. The number of samples is limited by a user-defined maximum and a time-out, whichever is reached first. GetLastSample(), on the other hand, will read the latest data and fast-forward the read pointer to past the last received telegram. The address of the data sample buffer will be indicated by the GetNextSamples()/ GetLastSample() function. If the circular buffer is not being read for a longer period, the internal buffer may overflow and hence the functions return an error. In this case, call the function FlushConnectionBuffer() to move the internal read position to the current device data write position.

Buffer Data It is also possible to pass a user buffer to the library which will be filled automatically with multiple samples. The relevant function for this scenario is ActivateAutoBufferMode().

Device Setup The device is configured and controlled by commands. Executing a command is done like this:

- create a new command by calling NewCommand() or NewCommandFromString(). Details are given in the respective sections.
- pass this command into ExecCommand()
- evaluate the returned response using the functions GetResponseInfo(), GetResponseArgType(), and GetResponseIntArg() etc.

Disconnect Finally, CloseConnection() will close the connection.

NOTE: The above approach of “Quick Start” uses the *synchronous* mode, i. e. “send commands and wait for response”. It means if a command is sent to the sensor, all the data samples already received will be ignored and the library waits for the response (*synchronous mode*).

[3] Currently CHRocodileLib is limited to IPv4.

© Precitec Optronik GmbH

Chapter 2

Different modes of operation

Operating a CHRocodile sensor has two main aspects:

- Controlling the device, i.e. sending commands which query or modify its status
- Acquiring measurement data

There are two different ways of accomplishing these tasks, the *synchronous* and the *asynchronous* mode.

2.1 Synchronous mode

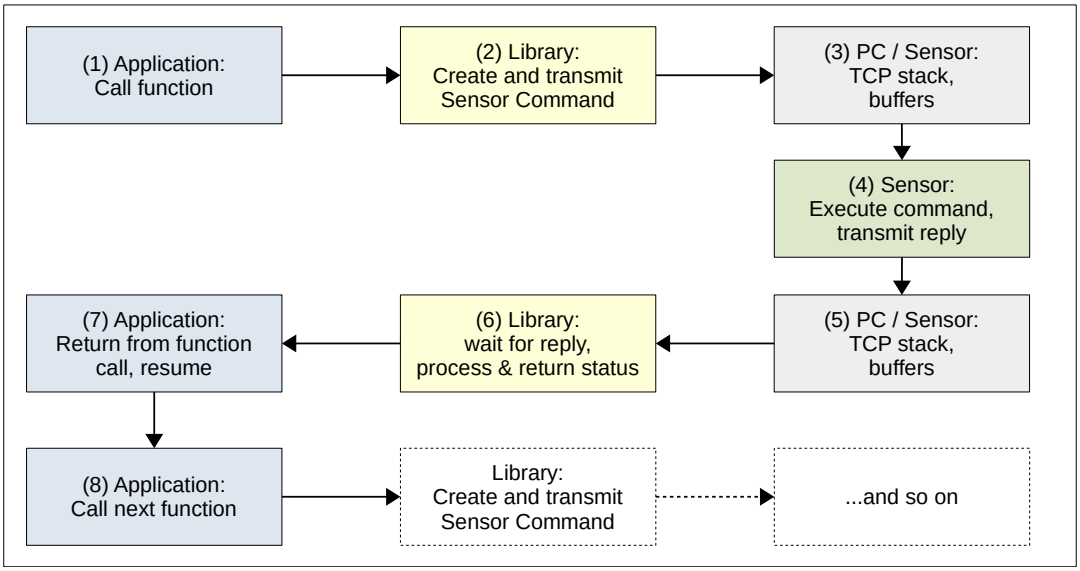


Figure 2.1: The most straight forward approach: Synchronous sensor control

Execute command When using the synchronous mode ExecCommand() function, the function call typically[1] causes a device command to be sent to the CHRocodile device. The CHRocodile device will then process the received command and respond. The response will in turn be further processed by the library to form result codes for the caller. This sequence is illustrated in figure 2.1.

After making an API function call, the caller checks the result code for potential errors and further processes the values passed back by the library. If no error occurred, the caller can safely assume that the command has successfully completed and the next one can be issued.

[1] There are cases in which a single command issued from the API results in several device commands.

 © Precitec Optronik GmbH

In *synchronous mode*, the function call for executing command will thus not return until the command has was created, sent, processed and responded to by the device. That is, the call will block the caller until it is done. Any sample data produced by the sensor in the meantime will be ignored and skipped.

Collect Data Measurement data is typically obtained by calling GetNextSamples(). This function does not block and returns either the required number of samples or the available number within the specified timeout. One can also use the *auto buffer save* functionality (see ActivateAutoBufferMode()) which runs in the background, moving data into the specified user buffer. Note, however, during the period in which data is automatically stored into the user buffer, you **should not** send any commands to the device as this has the potential to cause inconsistencies in the saved data[2].

Pros & Cons The *synchronous mode* can thus be summarized as follows:

+ sending a command is a strictly serial and atomic operation
+ the mode is simple and safe
– batch command processing (as during initialization) is slow because only one single command is sent and processed at a time

A general pattern for writing software that runs in synchronous mode:

- Open connection
- Set up device parameters with ExecCommand() or convenient functions/wrapper classes
- Collect data with GetNextSamples() or ActivateAutoBufferMode()

Demos under the folder “C++Demo\Basic_Multi_Channel”, “C++Demo\Basic_Single_Channel” and “CSharpDemo\Basic” illustrate the simple synchronous communication. All the demos demonstrate how to send commands (using basic API function or convenient functions/wrappers) and collect data (using GetNextSamples() or ActivateAutoBufferMode()).

2.2 Asynchronous Mode

What for? In many usage scenarios, the synchronous mode above will be a perfect fit. Initialization of the sensor is often a one-time task.

If your application falls into this category, then you can skip the rest of this chapter and read the sections covering GetNextSamples(), ActivateAutoBufferMode(), and so on.

If, on the other hand, device reconfiguration happens frequently in your application, the time to issue multiple discrete commands and wait for their respective responses may become significant. The *asynchronous mode* sidesteps the bottleneck of strict serialization by enabling you to issue commands without waiting for the completion of previous ones.

Or, if you need to design an interactive application that helps the user to set up some measurement, goals may be to:

- adjust the probe while observing a live spectrum
- see how peak height values are changing in response to measurement frequency or light intensity adjustments
- see how noise levels vary with different averaging or median settings

[2] In synchronous mode it is even forbidden to do so.

Synchronous mode would not only cause the loss of data samples but might also degrade the application's responsiveness (since synchronous command execution is blocking). In this case, you should consider the *asynchronous mode*.

Process Device Output In synchronous mode, synchronous functions, like ExecCommand(), GetNextSamples(), trigger the processing of CHRocodile device output (stored in the internal circular buffer). In asynchronous mode, ExecCommandAsync() is used for submitting commands - with the key difference that it returns immediately, leaving device output processing to some other function to be called later or in another thread: ProcessDeviceOutput().

`ProcessDeviceOutput` collects all device output and wraps it into data samples or command responses / updates.

An application may actively call this function e. g. in a GUI timer routine (i. e. in the same thread as the `ExecCommandAsync()` call). Then a button handler may initiate a device command while the timer routine updates / refreshes the dialog state.

Alternatively, the host application can request CHRocodileLib to take care of this task by calling StartAutomaticDeviceOutputProcessing(). `StartAutomaticDeviceOutputProcessing` actually initializes an extra internal thread to call ProcessDeviceOutput() constantly. In both approaches, processed data and command responses are passed to the host application via callback functions, see figure 2.2.

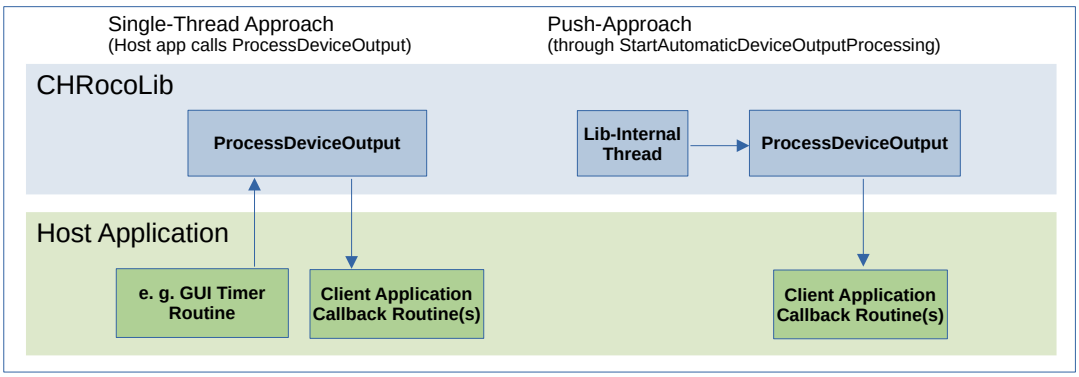


Figure 2.2: `ProcessDeviceOutput` can be called either from the host application or a thread inside CHRocodileLib.

As the callback functions are invoked from inside ProcessDeviceOutput(), they also run in the same thread as the ProcessDeviceOutput() call(see figure 2.3).

A single-threaded approach has some noticeable advantages:

- being single-threaded, it is easy and safe to implement, no specific locking / synchronization measures need to be taken.
- given reasonable load, it typically results in quite responsive applications.

However, the main downside is poor scalability in terms of data bandwidth / load since it only works well, if the GUI timer routine does not take significant time to execute.

In general, leveraging the power of modern multi-core PCs requires a multi-thread approach. For more details on the library's thread model, please refer to chapter 3.

 © Precitec Optronik GmbH

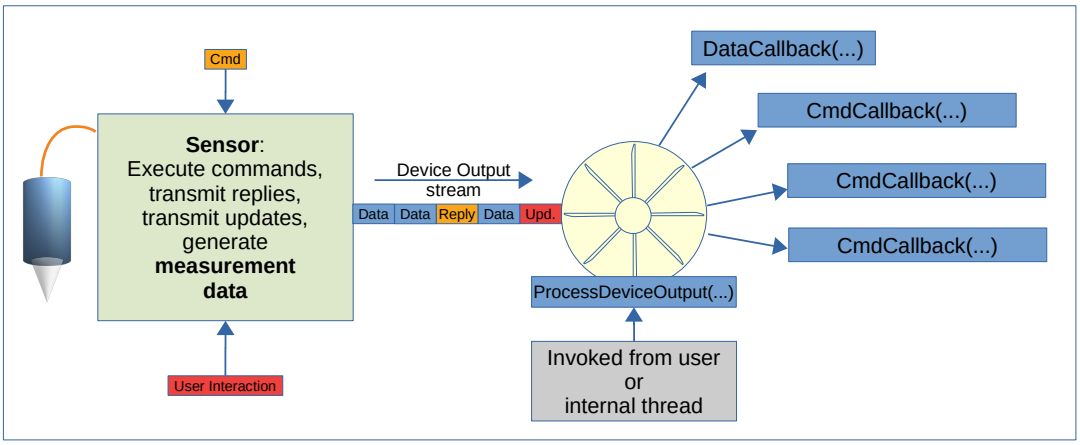


Figure 2.3: Function ProcessDeviceOutput() serves as a message pump. It reads all incoming data, command responses, and updates and delivers their content to the application by calling the registered callback functions.

Command Callback Function As mentioned above, the command responses are forwarded through callback functions (see figure 2.4). Callback functions can be registered for specific command responses (directly in ExecCommandAsync()) as well as for all the responses/updates in RegisterGeneralResponseAndUpdateCallback().

The callback function which is registered through RegisterGeneralResponseAndUpdateCallback() will be invoked upon receiving any responses/updates from the device, hence the name “general callback function”. Due to the nature of command updates as “spontaneous” messages from the device without prior command, a general callback function is the only way of receiving such updates. Normally the application code should set a "general callback function" right after opening a connection.

The callback function which is passed into ExecCommandAsync() will, on the other hand, only apply to the command in this specific ExecCommandAsync() call. If "general callback function" has been previously registered, the callback function in ExecCommandAsync() will be called first, followed by the "general callback function". If there is no special treatment for the command, it is thus not necessary to pass a callback function into every call to ExecCommandAsync().

Data Callback Function Similarly to responses, sample data is also delivered through calls to a user-defined callback function (see figure 2.4). This function is registered by calling RegisterSampleDataCallback(). There are two additional parameters given, the maximum number of samples and a timeout in milliseconds. For example, if arguments 1000 and 10 are passed, then this function will be invoked either when 1000 samples have been received or after 10 milliseconds have passed, whichever comes first. Since the device output stream is buffered in a library-internal buffer, even timeouts of zero milliseconds are allowed and are – in certain situations described later – the value of choice (for details on the data callback function see SampleDataCallback).

Executing Commands In asynchronous mode, ExecCommandAsync() only queues the device commands to be sent out to the CHRocodile device. The call returns almost instantly. In contrast to the synchronous mode, it does not wait for the command to be actually sent and the responses to be received. The transmission of the command will be automatically done by a thread of the library.

 © Precitec Optronik GmbH

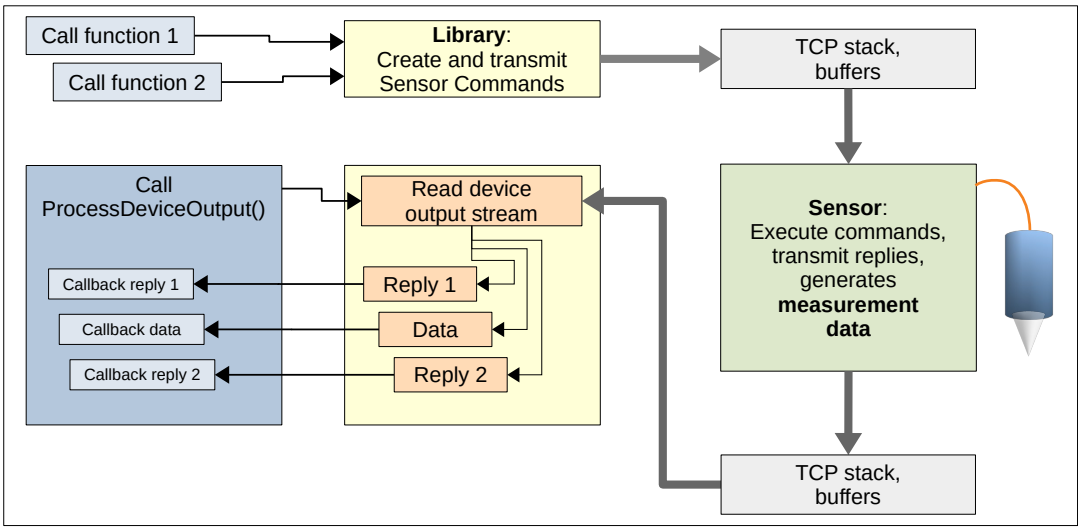


Figure 2.4: Consistent way of receiving data and command responses

When later the CHRocodile device sends back the responses, they are then processed either by the library itself or through calling ProcessDeviceOutput() (as outlined in figure 2.1). Any incoming responses (and updates) will eventually result in calls to the user-defined callback functions. Any data arriving in the meantime is not disposed as in the synchronous mode but delivered to the user through data callback function calls. Therefore in asynchronous mode, there is no data loss. ExecCommandAsync() can be invoked from a thread other than the thread calling ProcessDeviceOutput().

2.2.1 Designing applications in asynchronous mode

A general pattern for writing software that runs in asynchronous mode:

- Open connection
- Register general callbacks
- Register data callback
- *Either* constantly call ProcessDeviceOutput(), e.g. in a GUI timer routine
  or StartAutomaticDeviceOutputProcessing() to process CHRocodile device output

The steps in more detail:

Upon establishing the connection with OpenConnection(), your application should first register general command callbacks for all the device parameters of interest, regardless of whether the command response has been triggered by your application or somebody else. This implies that, commands, initially sent by your application itself or any other source, are handled uniformly. In this general command callback function you can, for example, put an implementation that updates your application's state (GUI control etc., start/stop data storage or drawing charts, whatever is needed).

After the general command callback function is registered, a data callback function SampleDataCallback needs to be registered with RegisterSampleDataCallback() to process data samples from the device.

If your application calls ProcessDeviceOutput() actively to process device output, e.g. from a timer routine. Your timer routine should look a bit like this:

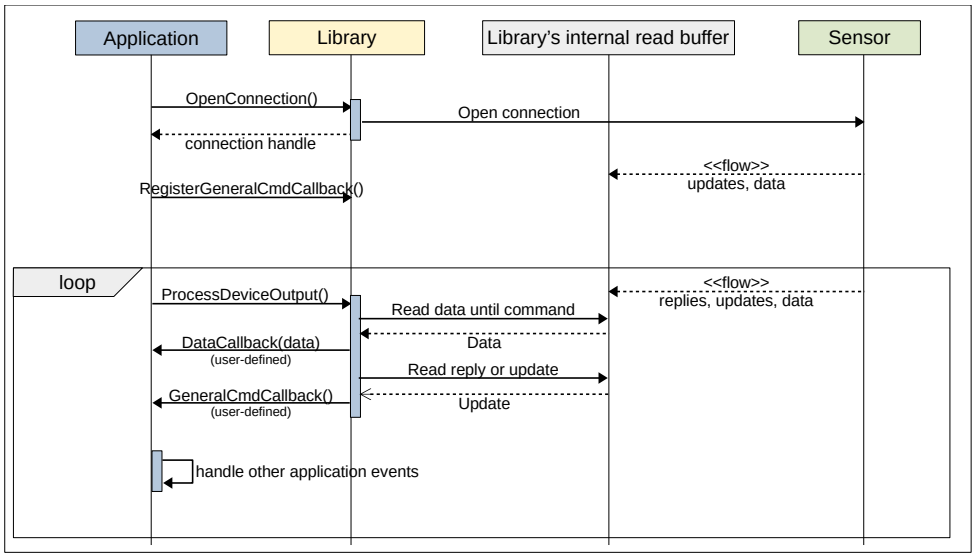


Figure 2.5: Simplified sequence diagram indicating the general steps of an interactive GUI application: Open connection, register callbacks for responses, updates, and data samples, start loop calling ProcessDeviceOutput() which in turn will read the device output stream and invoke the corresponding callback routines of the application

Listing 2.1: Calling ProcessDeviceOutput() from a timer routine

```
void MyForm::OnTimer()
{
  //return value from ProcessDeviceOutput: negative: error code,
  //0: timeout in waiting for preset amount of data to be read,
  //1: successfully read in preset amount of the data,
  //2: command response has been received,
  //3: library data sample buffer is not large enough to store preset amount of data.
  //4: User buffer is full
  //5: Data format has been changed when user buffer is used
  int32_t res = Read_Data_Not_Enough;
  do
    res = ProcessDeviceOutput(myConnectionHandle);
  while (res != Read_Data_Not_Enough);
}
```

As shown in listing 2.1, ProcessDeviceOutput() must be called until all pending data and commands are handled.

Note that care must be taken to ensure that all the callback routines does not block for more than maybe a couple of milliseconds. If a callback function is not finished in time, then ProcessDeviceOutput() will not return. Thus the following data samples/commands will not be processed. This causes slow process of device output. If CHRocodile device is running at high speed, for example 70KHz, it can result in data loss.

Please check out the demos under the folder “C++Demo\Async” and “CSharpDemo\Async” about asynchronous mode.

2.3 Making use of `_pUser`, the user context pointer

Callback functions are registered either by passing them into a respective RegisterGeneralResponseAndUpdateCallback() function or as an additional argument passed into ExecCommandAsync(). Example:

```
Res_t ExecCommandAsync(Conn_h _hConnection, Cmd_h _hCmd, void *_pUser,
    ResponseAndUpdateCallback _pCB, int32_t *_pTicket)
```

One of the arguments, `_pUser`, will be stored and later forwarded to the callback function when the command response has been processed by the device:

```
1 void ResponseCallback(const TRspCallbackInfo _sInfo, Rsp_h _hRsp)
2 {
3 // may use _sInfo.User ....
4 }
```

The library itself does not do anything with `_pUser` apart from storing and forwarding it – which gives the user the opportunity to employ it for application-specific purposes. The code below illustrates a typical C++ pattern, casting `_pUser` to a `this` pointer:

```
1 class MyClass
2 {
3   /*
4    * omitted declarations...:
5    */
6   int MeasuringMode;
7   void CheckState(...);
8
9   void SetMeasuringMethodAsync(int _nMode)
10  {
11    Cmd_h hCmd;
12    NewCommand (CmdID_Measuring_Method, false, &hCmd);
13    AddCommandIntArg (hCmd, _nMode);
14    Check(ExecCommandAsync(hCon, hCmd, this,&MyClass::MeasuringModeCallback,
          nullptr));
15  }
16
17  static void MeasuringModeCallback(TCmdCallbackInfo _sInfo, Rsp_h _hRsp)
18  {
19    MyClass * p = static_cast<MyClass *>(_sInfo.User);
20    p->CheckState(_sInfo.State);
21    GetResponseIntArg(_hRsp, 0, &p->MeasuringMode);
22  }
23 }
```


© Precitec Optronik GmbH

Chapter 3

Using Multiple Threads

CHRocodileLib provides multi-thread support. Under both synchronous and asynchronous connection, command can be executed (for example ExecCommand() and ExecCommandAsync()) and data can be fetched (for example GetNextSamples(), ProcessDeviceOutput()) in any thread.

Note that any callback function, either registered by means of a RegisterGeneralResponseAndUpdateCallback() function or as argument to ExecCommandAsync(), is executed in the context of the ProcessDeviceOutput() call. Consequently, if ProcessDeviceOutput() is called from a different thread, then the callback functions are executed within that thread, too.

If the connection is *shared*, i.e. a second handle is created based on an existing open “physical” device connection, these connections can also operate on different threads. Details on how to share connections can be found in chapter 4.

The only exception is, CloseConnection() should *not* be called within a callback function. This will cause a dead lock, since CloseConnection is waiting for callback function to finish, and callback function is waiting for CloseConnection to succeed.

© Precitec Optronik GmbH

Chapter 4

Sharing Device Connections

Motivation Consider the following situation:

- You are creating an interactive GUI application where users can manipulate certain parameters and view some life data such as the current thickness of a sample.
- For the sake of responsiveness you pick the asynchronous way of sensor communication (see section 2.2), i.e.:
  - You implement different button, combo box, etc. ... handlers that translate user actions to ExecCommandAsync() calls.
  - You implement a general callback function that receives all sensor replies / updates and dispatches all necessary GUI refresh procedures.
  - You call ProcessDeviceOutput() from a timer routine or a dedicated thread.
- But now you also need to offer some long-running functionality such as saving data into a file for a certain period. Or uploading large tables.

Standard Approaches You typically handle these requirements with either a state machine-based or a thread-based approach. In the former case you have to process the task in small steps and maintain the book-keeping. In the latter case, you may spawn some task running in parallel that needs to share any command input and reply / data output channels in a (thread-)safe way.

Shared Connections Shared connections offer an alternative here:

- Create a thread for the task where you...
- ...open a shared connection
- run the time-consuming task using that connection
- ...and close the connection, wait for the thread to be joined.

In the meantime, the rest of the application runs its business as usual. No special sensor communication synchronization required. If the task involves sensor control / state changes, the standard event loop of the application will be notified.

General Command Response Handler to enable loose coupling. The reason why this works is that any commands initiated by one shared connection will be normally replied there while – at the same time – a respective *update* will show up in the other shared connection. Your general command response handlers receive both command responses and updates. Figure 4.1 illustrates this.

Figure 4.2 gives a more detailed view of how this is managed inside CHRocodileLib: When e. g. ExecCommandAsync() is called, a respective command is stored in a common command queue and then automatically sent to the sensor. Once the sensor has finished the operation and the reply

 © Precitec Optronik GmbH

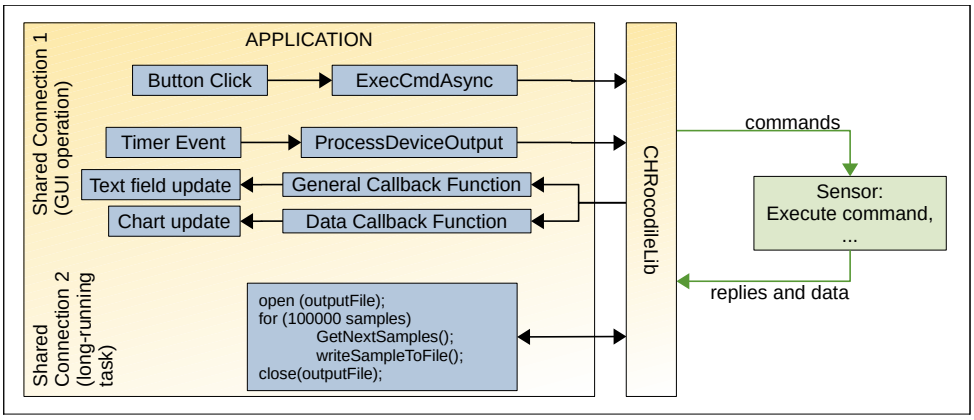


Figure 4.1: Sharing Connections

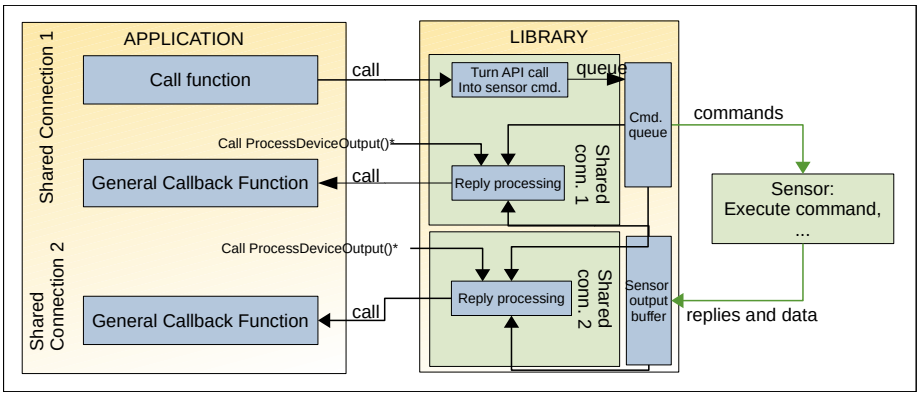


Figure 4.2: Sharing Connections - A closer look.

arrives in the library's sensor output buffer, the content of the reply is associated to it's counterpart in the command queue. These together enable the library to generate the command callback function arguments with the correct "source connection" ID. "Source connection" ID states from which connection the orginal command has been sent. It has the same value of the connection handle. If the "Source connection" ID is 0xFFFFFFFF, value of Invalid_Handle, that means it is an update from CHRocodile device, not a reponse of a command from any shared connection.

Shared connections are logical connections to the same device. They communicate with the CHRocodile device independently. As stated above, the commands they send will be queued together to be transferred to the device. At the same time they read out and process device output (data and command reponses) independently.

Each shared connection can choose either synchronous or asynchronous mode provided by the library to communicate with the device. Therefore by means of a shared connection, you can even combine synchronous / asynchronous communication. You can keep specific, potentially long-running tasks local in your code, virtually independent of the normal interactive operation.

You can also dedicate shared connections to specific application views such as data charts or life spectra. This may also play nicely with the fact that shared connections – just like connections in general – are thread-protected against each other.

The usage examples of the shared connections are under the folder "C++Demo\SharedConnection" and "CSharpDemo\SharedConn".

Chapter 5

Plugins

CHRocodileLib also offers the mechanism of plugins to perform extra data processing / filtering. Each connection / shared connection can independently add different plugins. It is also possible for each connection to have more than one plugin. AddConnectionPlugIn() or AddConnectionPlugInByID() provides the interface for adding the plugin. Through GetConnectionPlugInInfo() you can query added plugin information, i. e. plugin type, plugin handle.

As shown in figure 5.1 the measurement data from the CHRocodile device will be passed to the first plugin, the processed data from the first plugin will be passed to the second and so on... To obtain the data, the user utilizes the same functions as used without any plugins active (still based on the connection handle returned by OpenConnection()), for example GetNextSamples() under synchronous mode or through sample data callback function under asynchronous mode. In this case, the received data is the output data from the last plugin. Currently the original data from the device is available through an additional shared connection only. Output data of plugins other than the last plugin is not accessible for the host application.

To send a command to a given plugin or set plugin parameters, ExecCommand() and ExecCommandAsync() are used in the same way as for sensor commands. However, here the plugin handle which is obtained from AddConnectionPlugIn(), instead of the connection handle, is passed into ExecCommand() and ExecCommandAsync(). Based on the given handle, CHRocodileLib dispatches the command to either the device or the target plugin. Under asynchronous mode, CHRocodileLib will try to keep the sequence of the responses consistent to the commands sequence, i. e. the response to the previous command triggers its response callback function earlier than the response of the later command, regardless of whether the command is a plugin or device command. Consider for example a device command which is sent through CHRocodileLib, *immediately followed* by a plugin command. The device command will typically take much longer to execute than the plugin command, due to the much longer communication path. Yet, the device command's response will be returned first. This FIFO sequence applies in most cases, only under special circumstances (like e. g. a plugin command triggering spectrum downloads) the response sequence may change.

Currently available plugins are described below.

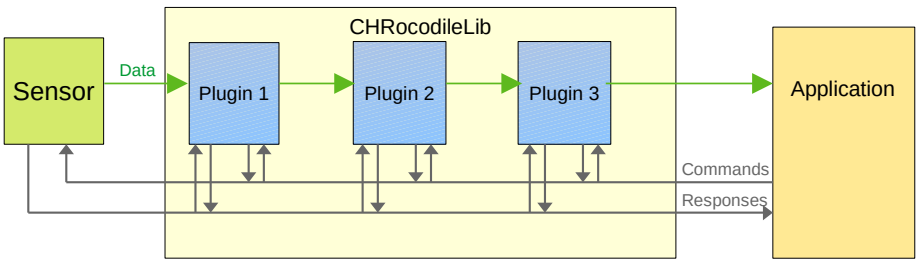


Figure 5.1: How plugin works

© Precitec Optronik GmbH

Chapter 6

Ini File

It is possible to preset a few parameters for CHRocodileLib through the file “Chrocodile.ini” (the file name cannot be changed and it should be located in the current working directory of the user application). If “Chrocodile.ini” exists, it will be automatically read upon loading of CHRocodileLib. The entries in “Chrocodile.ini” basically have the format of key-value pairs:

```
PluginDir = .\Plugin
DefaultPlugin = CLS2CalibrationPlugin
DefaultPluginParam = CmdID_CalibFile_Name: ".\pixel-calibration-sn-15.pc-csv"
LogDir = .\
EnableLog = 1
```

Not all entries need to appear in “Chrocodile.ini”, only the necessary ones.

PluginDir The directory where plugins are located. CHRocodileLib automatically tries to scan and load available plugins in this direcory. The default directory is the subdirectory “Plugin” of current working directory. It is highly recommended, to put CHRocodileLib plugins into a separate folder from the reset of user application libraries. The dynamic loading of certain third party library may throw error.

DefaultPlugin Default plugin for each (shared) connection. If “DefaultPlugin” is set, upon OpenConnection() the specified plugin will be created and added to the connection (if successful). More than one default plugin can be specified with the name. GetConnectionPlugInInfo() provides the possibility to query default added plugin information.

DefaultPluginParam Default plugin parameter value for the previous specified plugin. It should directly appear behind the targeted plugin. Different parameters/-commands are separated by “;”. The name of the parameter/command and its value are separated by “:”. As we can see in the above example, the name of the parameter is simply the defined command name in each plugin. If the parameter value is a string, it should be inside “ ”.

LogDir Directory where CHRocodileLib log files should be saved to. Default log directory is the current working directory. This directory can also be set through SetLibLogFileDirectory().

MaxLogFileSize Maximum size of one log file in KB. When the size of the log file exceeds the maximum size, CHRocodileLib starts with a new log file. Default Maximum size is 512KM. It can also be set through SetLibLogFileDirectory().

MaxlogFileNumber Maximum number of log files in the directory. If there are too many files in the log directory, the oldest file will be deleted. Default maximum log file number is 10. It can also be set through SetLibLogFileDirectory().

EnableLog Enables CHRocodileLib to write log files. By default, CHRocodileLib does not write any log files.

InternthreadPriority Thread priority of CHRocodileLib’s internal thread, which is used for buffering the CHRocodile device output stream. This parameter should

 © Precitec Optronik GmbH

normally not need to be set. It currently takes effect with the Windows version of the library only.

SocketBufferSize TCPIP socket buffer size of CHRocodileLib. Default size is 16MB.

LibFlag CHRocodileLib configuration flag, which can also be set through SetLibConfigFlags(). For details, please refer to SetLibConfigFlags().

Traceflag Flag for automatically saving CHRocodileLib internal status. 1: under asynchronous modus, if internal buffer is full, CHRocodileLib will start to save status; 2: CHRocodileLib automatically saves the status at each defined internal in “AutoStatusSaveInterval”; 4: CHRocodileLib saves the status when too many bytes are received once at TCPIP socket. Status saving is for the purpose of problem tracing/debugging. Default value is 0, i.e. no trace mechanism is set.

SocketMaxBytes Maximum bytes of data, which is available at TCPIP socket. If available data exceeds this limit, based on preset “TraceFlag”, CHRocodileLib may automatically save internal status. Default limit is 4MB.

AutoStatusSaveInterval Interval in second, at which internal status should automatically saved. Default value is 60.

PerformanceStatusCount Status history enytry count for current connection. Default value is 4096.

InternalStatusCount Status history enytry count for internal thread. Default value is 2048.

AutoSampleProcessCount The maximum count of the device original sample before passed to the plugins for further processing. This sample count will be only used when plugin(s) is added to the connection and when CHRocodileLib is under autobuffer mode. Default value is 1000.

RspResourceManage Sets whether CHRocodileLib should automatically invalidate old command responses. When “RspResourceManage” is set to true, the previous response will be automatically deleted/invalidated upon receiving new responses. User cannot fetch any information of the previous response anymore. If "RspResourceManage" is false, you can access the old responses at any time. However to avoid memory leak, DestroyHandle() needs to be called to release the response when it is not used anymore. Default value is true.

ConnCTNMode Decides whether upon connecting to the CHRocodile device, CHRocodileLib should automatically send “CTN” command to the device. Default value is true

EnableKeepalive Enables keepalive for TCPIP connection with the CHRocodile device. Default value is false.

© Precitec Optronik GmbH

Part II

API Reference

© Precitec Optronik GmbH

Chapter 7

Exported functions and data types

7.1 Scalar type Cmd_h

Description

Type definition `typedef Handle_t Cmd_h;`

7.2 Scalar type Conn_h

Description

Type definition `typedef Handle_t Conn_h;`

7.3 Scalar type FilePath_t

Description

Type definition `typedef const char * FilePath_t;`

7.4 Scalar type Handle_t

Description

Type definition `typedef uint32_t Handle_t;`

7.5 Scalar type LibSize_t

Description

Type definition `typedef int64_t LibSize_t;`

7.6 Scalar type Plugin_h

Description

Type definition `typedef Handle_t Plugin_h;`

© Precitec Optronik GmbH

7.7 Scalar type Rsp_h

Description

Type definition `typedef Handle_t Rsp_h;`

© Precitec Optronik GmbH

7.8 Structure TErrorInfo

Description Information of each single error stored in the buffer returned by LastErrors()

Type definition

```
typedef struct {
    Handle_t SourceHandle;
    int32_t ErrorCode;
} TErrorInfo;
```

Members

SourceHandle	The handle of the connection where the error occurs.
ErrorCode	Error code.

© Precitec Optronik GmbH

7.9 Structure TPluginInfo

Description Plugin information returned by GetConnectionPlugInInfo()

Type definition

```
typedef struct {
    Plugin_h PluginHandle;
    uint32_t TypeID;
} TPluginInfo;
```

Members

PluginHandle	Plugin handle.
TypeID	Type ID of the plugin.

© Precitec Optronik GmbH

7.10 Structure TResponseInfo

Description Command response information.

Type definition

```
typedef struct {
    uint32_t CmdID;
    int32_t Ticket;
    uint32_t Flag;
    uint32_t ParamCount;
} TResponseInfo;
```

Members

CmdID	Command/response ID.
Ticket	Internal ticket for the original command.
Flag	Response flag see A.4.
ParamCount	Number of parameters in the response.

© Precitec Optronik GmbH

7.11 Structure TRspCallbackInfo

Description General command response information in callback function.

Type definition

```
typedef struct {
    void * User;
    int32_t * State;
    int32_t Ticket;
    Handle_t SourceConnection;
} TRspCallbackInfo;
```

Members

User	Value as passed to RegisterGeneralResponseAndUpdateCallback or ExecCommandAsync() function. Forwarded but not interpreted by the library. See also section 2.3 of the documentation.
State	Status of command response: negative: error or warning code, `0`: no error. If `*_State` is set to `1` by the callback function, general command callback function for this response will not be called.
Ticket	Internal ticket for command.
SourceConnection	Handle of the connection/plugin which has originally issued the command for current response. If response has been sent out by device as an update (for example in CHRocodile 2 device), `0xFFFFFFFF` will be given out.

More details A structure of this type will be passed to a user- supplied callback function that has been specified by a call to either the RegisterGeneralResponseAndUpdateCallback() or the ExecCommandAsync() function. If a general callback function has been registered with RegisterGeneralResponseAndUpdateCallback() and another callback function has been specified in a call to ExecCommandAsync(), the following will happen:

- The callback function specified in the call to ExecCommandAsync() will be executed first; the general callback function passed to RegisterGeneralResponseAndUpdateCallback() will be executed afterwards.
- If the callback function specified in the call to ExecCommandAsync() sets the integer variable of the TRspCallbackInfo structure to “1” (for example, `*_State = 1;`), the library will *not* execute the general callback function.

© Precitec Optronik GmbH

7.12 Structure TSampleSignalGeneralInfo

Description Structure to represent the general information of output signals.

Type definition

```
typedef struct {
    uint32_t InfoIndex;
    int32_t PeakSignalCount;
    int32_t GlobalSignalCount;
    int32_t ChannelCount;
} TSampleSignalGeneralInfo;
```

Members

InfoIndex	Index of sample signal data format; increases with changing of the data format.
PeakSignalCount	Number of peak signals in data sample. For single channel device, all the signals are regarded as global signal.
GlobalSignalCount	Global signal count in data sample.
ChannelCount	Device channel count.

© Precitec Optronik GmbH

7.13 Structure TSampleSignalInfo

Description Structure to represent the detailed information of each output signal.

Type definition

```
typedef struct {
    uint16_t SignalID;
    int16_t DataType;
} TSampleSignalInfo;
```

Members

SignalID	Signal ID.
DataType	Data type. For a detailed definition see section A.3.

© Precitec Optronik GmbH

7.14 Callback ResponseAndUpdateCallback

Description The format of callback function upon receiving command response.

Prototype

```
typedef void(* ResponseAndUpdateCallback)(
    TRspCallbackInfo _sInfo,
    Rsp_h _hRsp
);
```

Arguments

in	_sInfo	Struct contains the general information about the response (see **TRspCallbackInfo**)
in	_hRsp	Response handle.

More details Note: the response handle "_hRsp" will be automatically invalidated in the next respond callback. Therefore user cannot save the handle for later use.

© Precitec Optronik GmbH

7.15 Callback SampleDataCallback

Description Data sample callback function, which is called to pass device data samples.

Prototype

```
typedef void(* SampleDataCallback)(
    void * _pUser,
    int32_t _nState,
    int64_t _nSampleCount,
    const double * _pSampleBuffer,
    LibSize_t _nSizePerSample,
    TSampleSignalGeneralInfo _sSignalGeneralInfo,
    TSampleSignalInfo * _pSignalInfo
);
```

Arguments

in	_pUser	Value as passed to RegisterSampleDataCallback. Forwarded but not interpreted by the library. See also section 2.3 of the reference manual.
in	_nState	Data reading status: negative: error or warning code, 0: time out in waiting for preset amount of data to be read, 1: successfully read in preset amount of the data, 2: command response has been received, 3: data Sample buffer is not large enough to store preset amount of data.
in	_nSampleCount	Number of data samples received.
in	_pSampleBuffer	Buffer which stores received data sample. Buffer is maintained by library.
in	_nSizePerSample	Size of each data sample.
in	_sSignalGeneralInfo	General information of signals.
in	_pSignalInfo	Array which contains the detailed description of each signal.

© Precitec Optronik GmbH

7.16 Function ActivateAsyncDataUserBuffer()

Description in asychronous mode save data from device to the user target buffer.

Prototype

```
Res_t ActivateAsyncDataUserBuffer(
    Conn_h _hConnection,
    double * _pBuffer,
    int64_t _nSampleCount,
    LibSize_t * _pBufferSizeInBytes
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_pBuffer	A double pointer to the buffer where the data is written to.
in	_nSampleCount	Number of the sample to be written to the buffer.
inout	_pBufferSizeInBytes	Size of the target buffer in byte. If the target buffer is not large enough or _pBuffer is not assigned, required minimum size is written to *_pBufferSizeInBytes.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

More details

Call the ActivateAsyncDataUserBuffer() function to select an application-supplied buffer that will received sample data when using an asynchronous connection. (If you prefer using a synchronous connection, you can use the function ActivateAutoBufferMode() for collecting sample data in an application-defined buffer.) The device output signals, which are inside one sample, must be ordered from the device before calling ActivateAsyncDataUserBuffer().

The CHRocodile Library will store sample data from the device only temporarily in an internal circular buffer. When using an asynchronous connection, the library will invoke the specified callback function, whenever new sample data becomes available in that buffer. However, the callback function must then either process new sample data immediately, or copy the data from the library-internal data sample buffer to an application-supplied buffer for later processing. (Because the CHRocodile Library will store sample data only temporarily inside the sample buffer, your application might otherwise lose sample data.) By calling ActivateAsyncDataUserBuffer(), you can direct the library to copy new sample data into an application buffer automatically, hence the callback function does not need to take care about "saving" new sample data for later processing.

To utilize the automatic collection of sample data in the background, do the following:

1. Register the callback function to be invoked when new sample data is available by calling the function RegisterSampleDataCallback().
2. Use the device SODX command to specify the signals inside one sample
3. Call the function StartAutomaticDeviceOutputProcessing() to enable the automatic processing of device output in the background.
4. Upon receiving SODX response, call ActivateAsyncDataUserBuffer() and pass a pointer to the application buffer as well as the buffer's size.

The collection procedure initiated by calling StartAutomaticDeviceOutputProcessing() will continue to store samples in the application-supplied buffer until either of the following conditions become true:

- There is no space left in the application buffer, i.e. the buffer is full.
- Signals or signal format have been changed, for example in between SODX command has been sent to the device again,
- The function DeactivateAsyncDataUserBuffer() is called. (Afterwards, the library will use its own internal data buffer for storing new sample data. A pointer to this internal data buffer will be passed to the sample data callback function the next time it is invoked.)
- The collection procedure has failed for some reason.
- The automatic data processing of the library gets stopped by a call to the function StopAutomaticDeviceOutputProcessing().

When invoked, the callback function will receive a status code informing about the state of the sample data retrieval process.

If there is no space left in the application buffer, the following will happen:

- The status code passed to the callback function will be `Read_Data_User_Buf_Full`.
- No new samples will be retrieved. When the callback function gets invoked subsequently, the sample count will be always zero. In addition, the status code will be always `Read_Data_User_Buf_Full`.
- In addition, no command responses or command updates will be passed to the general command response/update callback routine. (If you have specified such a callback routine for the connection.)

© Precitec Optronik GmbH

To resume retrieving sample data (and to continue processing command responses and command updates), your sample callback function must call the function ActivateAsyncDataUserBuffer() once more; this will cause the library to re-use the application buffer for filling in new sample data, like

7.17 Function ActivateAutoBufferMode()

Description Automatically save data from device to the target buffer.

Prototype

```
Res_t ActivateAutoBufferMode(
    Conn_h _hConnection,
    double * _pBuffer,
    int64_t _nSampleCount,
    LibSize_t * _pBufferSizeInBytes
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_pBuffer	A double pointer to the buffer where the data is written to.
in	_nSampleCount	Number of the sample to be written to the buffer.
inout	_pBufferSizeInBytes	Size of the target buffer in byte. If the target buffer is not large enough or _pBuffer is not assigned, required minimum size is written to *_pBufferSizeInBytes.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

More details

The ActivcateAutoBufferMode() function will initiate the automatic collection of sample data in the background. (Note that your software must use a synchronous connection for calling ActivcateAutoBufferMode(); if you prefer using an asynchronous connection, apply the function ActivateAsyncDataUserBuffer() instead.) The device output signals, a sample will consist of, must be ordered before calling ActivcateAutoBufferMode().

Normally, the CHRocodile Library will store sample data retrieved from a device only temporarily using an internal circular buffer. Your software can read data from this circular buffer by calling functions such as GetNextSamples() or GetLastSample(); however, you may alternatively direct the CHRocodile Library to copy retrieved sample data automatically into a buffer your software provides by calling the ActivateAutoBufferMode() function.

Note that the collection process initiated by calling ActivcateAutoBufferMode() will start to read data from the library- internal circular buffer that has not yet been read by a call to the GetNextSamples() function. If you want to discard all sample data that has already been retrieved from a device and start to read "new" data just acquired, call the function FlushConnectionBuffer() first.

The collection procedure initiated by calling ActivcateAutoBufferMode() will continue until either of the following condition becomes true:

- The specified number of data samples to be collected has been received.
- The function DeactivateAutoBufferMode() is called.
- The collection procedure has failed for some reason.
- The library has detected that the set of device output signals a sample consists of has been changed
- In case the flag `CHRLib_Flag_Rsp_Deactivate_Auto_Buffer` has been set using the function SetLibConfigFlags(), the collection procedure will also stop if the library detects that a command response or command update has been found in the data stream.

The status of a progressing sample data collection procedure can be queried using the function GetAutoBufferStatus(); this function returns either of the following state codes:

- Error (–1) — The collection procedure has failed.
- Saving (0) — The sample collection is progressing in the background.
- Finished (1) — All requested samples have been collected.
- Received Response (2) — The collection procedure has terminated because a command response or command update has been received.
- Deactivated (3) — The collection procedure has been terminated by a call to function DeactivateAutoBufferMode().
- Uninit (4) — A collection procedure has not yet been started (by a call to the ActivcateAutoBufferMode() function.)

Use the function GetAutoBufferSavedSampleCount() to retrieve the number of samples that have already been collected.

Similar to other data retrieval functions—such as GetNextSamples(), GetLastSample(), etc.—, the data copied to the specified sample data buffer is organized sample-wise and *not* ordered by signal. For example, if the peak-related output signals "distance/peak 1" and "intensity/peak 1" have been ordered, the data found in the buffer will be saved according to the following scheme:

sample 1: distance/peak 1, intensity/peak 1
sample 2: distance/peak 1, intensity/peak 1
sample 3: distance/peak 1, intensity/peak 1
. . . ;
sample n: distance/peak 1, intensity/peak 1

For multi-channel devices—for example, CLS, MPS or CLS2—the "dis-

© Precitec Optronik GmbH

7.18 Function AddCommandBlobArg()

Description Add binray data argument to the command.

Prototype

```
Res_t AddCommandBlobArg(
    Cmd_h _hCmd,
    const int8_t * _pArg,
    int32_t _nLength
);
```

Arguments

in	_hCmd	Command handle returned by NewCommand().
in	_pArg	A pointer, pointing to where binary data resides
in	_nLength	Size of the binary data
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.19 Function AddCommandFloatArg()

Description Add float argument to the command.

Prototype

```
Res_t AddCommandFloatArg(
    Cmd_h _hCmd,
    float _nArg
);
```

Arguments

in	_hCmd	Command handle returned by NewCommand().
in	_nArg	Float argument.
return		Zero on success. Negative error or warning code..

© Precitec Optronik GmbH

7.20 Function AddCommandFloatArrayArg()

Description Add float array argument to the command.

Prototype

```
Res_t AddCommandFloatArrayArg(
    Cmd_h _hCmd,
    const float * _pArg,
    int32_t _nLength
);
```

Arguments

in	_hCmd	Command handle returned by NewCommand().
in	_pArg	Pointer to the float array argument.
in	_nLength	Length of the float array.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.21 Function AddCommandIntArg()

Description Add integer argument to the command.

Prototype

```
Res_t AddCommandIntArg(
    Cmd_h _hCmd,
    int32_t _nArg
);
```

Arguments

in	_hCmd	Command handle returned by NewCommand().
in	_nArg	Integer argument.
return		Zero on success. Negative error or warning code..

© Precitec Optronik GmbH

7.22 Function AddCommandIntArrayArg()

Description Add integer array argument to the command.

Prototype

```
Res_t AddCommandIntArrayArg(
    Cmd_h _hCmd,
    const int32_t * _pArg,
    int32_t _nLength
);
```

Arguments

in	_hCmd	Command handle returned by NewCommand().
in	_pArg	Pointer to the integer array argument.
in	_nLength	Length of the integer array.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.23 Function AddCommandStringArg()

Description Add string argument to the command.

Prototype

```
Res_t AddCommandStringArg(
    Cmd_h _hCmd,
    const char * _pArg,
    int32_t _nLength
);
```

Arguments

in	_hCmd	Command handle returned by NewCommand().
in	_pArg	A zero-terminated argument string.
in	_nLength	Length of the string.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.24 Function AddConnectionPlugIn()

Description Add plugin by name to one specific connection.

Prototype

```
Res_t AddConnectionPlugIn(
    Conn_h _hConnection,
    const char * _strPluginName,
    Plugin_h * _hPlugin
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_strPluginName	Plugin name. The name of each available plugin is defined in CHRocodileLibPluginDef.h
out	_hPlugin	Returned plugin handle on success
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.25 Function AddConnectionPlugInByID()

Description Add plugin by ID to one specific connection.

Prototype

```
Res_t AddConnectionPlugInByID(
    Conn_h _hConnection,
    uint32_t _nTypeID,
    Plugin_h * _hPlugin
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_nTypeID	Plugin type ID. The type ID of each available plugin is defined in CHRocodileLibPluginDef.h
out	_hPlugin	Returned plugin handle on success
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.26 Function CancelCHRDeviceAutoSearch()

Description Cancel automatic CHRocodile device search in unblocking search mode.

Prototype

```
void CancelCHRDeviceAutoSearch();
```


© Precitec Optronik GmbH

7.27 Function ClearErrors()

Description Clear errors.

Prototype

```
Res_t ClearErrors(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	When the valid connection handle is given, the error of that specific connection is cleared. When Invalid_Handle is passed, the errors of all the connections are cleared.
return		Zero on success, negative in case of an error or warning.

© Precitec Optronik GmbH

7.28 Function CloseConnection()

Description Close connection to CHRocodile device.

Prototype

```
Res_t CloseConnection(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative error or warning code..

More details When you close a connection, the remaining connections to the same device you may have opened by calling the OpenConnection() or OpenSharedConnection() functions are not affected and can still be used to communicate with the device. As soon as you close the last connection, the library will also terminate the communication link with the device.

© Precitec Optronik GmbH

7.29 Function CmdNameToID()

Description Translate device command name to Lib command ID.

Prototype

```
int32_t CmdNameToID(
    const char * _strCmd,
    int32_t _nLength
);
```

Arguments

in	_strCmd	Device command name in zero-terminated string.
in	_nLength	Length of the command name.
return		Corresponding command ID

© Precitec Optronik GmbH

7.30 Function DeactivateAsyncDataUserBuffer()

Description in asychronous mode reset to save data to library internal buffer.

Prototype

```
Res_t DeactivateAsyncDataUserBuffer(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative value: error or warning code.

© Precitec Optronik GmbH

7.31 Function DeactivateAutoBufferMode()

Description Manually quit automatic buffer data save mode.

Prototype

```
Res_t DeactivateAutoBufferMode(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative value: error or warning code.

© Precitec Optronik GmbH

7.32 Function DestroyHandle()

Description Release object related to the handle.

Prototype

```
void DestroyHandle(
    Handle_t _Handle
);
```

Arguments

in	_Handle	Handle which to be released

More details In most cases, calling DestroyHandle() explicitly is not required. However, if your software calls the NewCommand() or NewCommandFromString() functions for preparing a new device command, but never passes the resulting command handle to the ExecCommand() (or ExecCommandAsync()) function, you should use DestroyHandle() at some point to free the resources occupied by the command object created by NewCommand() or NewCommandFromString(). Or if CHRocodileLib is set in ini file not to automatically invalidate response (chapter 6), you should also use this function to release the old and unused responses.

Note also that passing a connection handle (returned by the functions OpenConnection() or OpenSharedConnection()) will close the respective connection. (That is, passing a connection handle to DestroyHandle() is synonymous to calling CloseConnection().)

© Precitec Optronik GmbH

7.33 Function DetectedCHRDeviceInfo()

Description Get connection information of automatically detected CHRocodile devices.

Prototype

```
Res_t DetectedCHRDeviceInfo(
    char * _aDeviceInfos,
    LibSize_t * _pnSize
);
```

Arguments

in	_aDeviceInfos	Pointer to the buffer where the connection information string of detected CHR devices should be saved to (zero-termination is included). Information for different devices is separated by ";".
inout	_pnSize	Size of the given string buffer. If it is not large enough or _aDeviceInfos is not assigned, the minimum required size is written back (including zero termination).
return		Zero on success. Negative error or warning code.

More details The function will copy information on the detected device(s) encoded as a null-terminated character string into the buffer you provide. If more than one device has been detected, the information sections will be separated by a semicolon character (';').

The device information returned by DetectedCHRDeviceInfo() is formatted similarly to the connection description string you have to pass as argument to the OpenConnection() function; that is, it comprises the device type and details on the link that can be used to connect to the respective device (such as a TCP/IP address.) As a result, you can pass the information string for a device directly to the OpenConnection() function (with the `_nDeviceType` argument of this function set to the value `CHR_Unspecified`.)

If the buffer size pointed to by the `_aDeviceInfos` argument is not large enough to store the device information string, the DetectedCHRDeviceInfo() function will return with an error status. In addition, the function will store the required minimum number of characters in the variable pointed to by the `_pnSize` argument. Thus, you may call the DetectedCHRDeviceInfo() function twice:

1. Call DetectedCHRDeviceInfo() with the `_aDeviceInfos` argument set to the null pointer. The function will determine the minimum buffer size and store the resulting value in the variable pointed to by the `_pnSize` argument.
2. Allocate a character string buffer with the size returned in the variable pointed to by `_pnSize`.
3. Call DetectedCHRDeviceInfo() again, this time with the `_aDeviceInfos` argument pointing to the character string buffer allocated in the preceding step.

© Precitec Optronik GmbH

7.34 Function ErrorCodeToString()

Description Get error string from error code.

Prototype

```
Res_t ErrorCodeToString(
    int32_t _nErrorCode,
    char * _aErrorStr,
    LibSize_t * _pnSize
);
```

Arguments

in	_nErrorCode	Error code.
in	_aErrorStr	Pointer to the string buffer where the zero-terminated error string should be saved to (zero termination included).
inout	_pnSize	Size of the given string buffer. The minimum required size is written back (including zero termination).
return		Zero on success, negative in case of an error or warning.

More details If the buffer size pointed to by the `_aErrorStr` argument is not large enough to store the description string, the ErrorCodeToString() function will return with an error status. In addition, the function will store the required minimum number of characters in the variable pointed to by the `_pnSize` argument. Thus, you may call the ErrorCodeToString() function twice:

1. Call ErrorCodeToString() with the `_aErrorStr` argument set to the null pointer. The function will determine the minimum buffer size and store the resulting value in the variable pointed to by the `_pnSize` argument.
2. Allocate a character string buffer with the size returned in the variable pointed to by `_pnSize`.
3. Call ErrorCodeToString() again, this time with the `_aErrorStr` argument pointing to the character string buffer allocated in the preceding step.

© Precitec Optronik GmbH

7.35 Function ExecCommand()

Description

Send command to device and wait for the response, i.e. synchronous command sending.

Prototype

```
Res_t ExecCommand(
    Handle_t _hHandle,
    Cmd_h _hCmd,
    Rsp_h * _phRsp
);
```

Arguments

in	_hHandle	Connection handle (returned by OpenConnection() or OpenSharedConnection()) or plugin handle (returned by AddConnectionPlugin()).
in	_hCmd	Command handle returned by NewCommand().
out	_phRsp	Pointer to a variable receiving the response handle.
return		Zero on success. Negative error or warning code.

More details

This function sends a command (prepared by a call to the functions NewCommand() or NewCommandFromString()) to a device and waits for the device's response. All sample data the device may transmit between sending the command and receiving the associated response will be ignored and not stored in the CHRocodile Library's internal sample data buffer.

The command response returned by the device is associated with a *response handle*; use this handle to query information on the response using commands such as GetResponseInfo(), GetResponseArgType(), GetResponseIntArg() and similar.

Note that the connection handle passed to this function must represent a synchronous connection. For executing commands using an asynchronous connection, call the function ExecCommandAsync().

Also, note that the response handle returned by this function will be, by default, automatically invalidated when subsequently calling ExecCommand() or ExecCommandAsync(). (Thus, you must not attempt to use the response handle after calling one of the command execution functions again.). If the response invalidation is not prefered, you can change this default behaviour in the ini file.

© Precitec Optronik GmbH

7.36 Function ExecCommandAsync()

Description

Send command to device without waiting for the response, i.e. asynchronous command sending.

Prototype

```
Res_t ExecCommandAsync(
    Handle_t _hHandle,
    Cmd_h _hCmd,
    void * _pUser,
    ResponseAndUpdateCallback _pCB,
    int32_t * _pTicket
);
```

Arguments

in	_hHandle	Connection handle (returned by OpenConnection() or OpenSharedConnection()) or plugin handle (returned by AddConnectionPlugin()).
in	_hCmd	Command handle returned by NewCommand().
in	_pUser	Can be freely used to forward user information to the function _pCB points to. Passed to _pCB but not interpreted by the library. See also section 2.3 of the reference manual.
in	_pCB	Callback function for current command when response from device comes back.
out	_pTicket	Pointer to a variable receiving the internal ticket for the issued device command.
return		Zero on success. Negative error or warning code.

More details

This function queues a command (prepared by a call to the functions NewCommand() or NewCommandFromString()) for transmission to a device and returns immediately.

When the device's response is received, the CHRocodile Library can automatically execute a callback function:

- You may use the function's `_pCB` argument to specify a callback function of type ResponseAndUpdateCallback(); this callback function will be executed when the device response is received.
- If you set `_pCB` to a null pointer but have specified a general command callback function by using the RegisterGeneralResponseAndUpdateCallback() function, the respective general callback function will be executed when the device response arrives.

Note that the connection handle passed to this function must represent an asynchronous connection. For executing commands using a synchronous connection, call the function ExecCommand().

© Precitec Optronik GmbH

7.37 Function FlushConnectionBuffer()

Description

Flush all the received data and command response, move the read pointer to the last valid position in the data stream from the device.

Prototype

```
Res_t FlushConnectionBuffer(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative error or warning code.

More details

Use this function to discard any data delivered by a device and buffered by the library in conjunction with a synchronous connection. When this function returns, command responses and command updates received previously, and all samples that the library has stored in its internal circular buffer (but not yet retrieved by your software) have been deleted.

Calling this function can be required for re-synchronizing your software with a device in certain scenarios, including:

- Your software has retrieved some samples—for example, by calling the GetNextSamples() function—but then deferred processing more samples for a longer period of time. In this case, older samples in the library's internal circular buffer may have been overwritten by new samples acquired and delivered by the device, causing your software to be out-of-synch with the actual data acquisition process.
- Your software has not yet retrieved all available samples from the library's internal circular buffer and then modifies the set of signals a sample actually consists of.
- Similarly, your software has not yet retrieved all available samples from the library's internal circular buffer and the library encounters a command response or command update. In this case, the sample data retrieved before the response has arrived may reflect a different device state than the sample data that has arrived afterwards.

In all of the above scenarios, calling FlushConnectionBuffer() will re- synchronize your software with the device.

© Precitec Optronik GmbH

7.38 Function GetAsyncDataUserBufferSavedSampleCount()

Description Get the number of the sample already saved into the preset user buffer in the asynchronous mode.

Prototype

```
Res_t GetAsyncDataUserBufferSavedSampleCount(
    Conn_h _hConnection,
    int64_t * _pnSampleCount
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
out	_pnSampleCount	Pointer to a variable receiving the number of saved samples.
return		Zero on success. Negative value: error or warning code.

© Precitec Optronik GmbH

7.39 Function GetAutoBufferSavedSampleCount()

Description

Get the number of the sample already saved into the buffer in the automatic buffer data save mode.

Prototype

```
Res_t GetAutoBufferSavedSampleCount(
    Conn_h _hConnection,
    int64_t * _pnSampleCount
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
out	_pnSampleCount	Pointer to a variable receiving the number of saved samples.
return		Zero on success. Negative value: error or warning code.

© Precitec Optronik GmbH

7.40 Function GetAutoBufferStatus()

Description Get the status of automatic buffer data save.

Prototype

```
int32_t GetAutoBufferStatus(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		negative: error or warning code; otherwise the status of automatic buffer data save, see Appendix A.9

© Precitec Optronik GmbH

7.41 Function GetConnectionMode()

Description Get device connection mode.

Prototype

```
int32_t GetConnectionMode(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		negative: error or warning code; otherwise see Appendix A.7

© Precitec Optronik GmbH

7.42 Function GetConnectionOutputSignalInfos()

Description Get the signal information of the data sample output by the library.

Prototype

```
Res_t GetConnectionOutputSignalInfos(
    Conn_h _hConnection,
    TSampleSignalGeneralInfo * _pSignalGeneralInfo,
    TSampleSignalInfo * _pSignalInfoBuf,
    LibSize_t * _pnSignalInfoBufSizeInBytes
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
out	_pSignalGeneralInfo	A pointer to the struct, where the general information of signals in data sample (see **TSampleSignalGeneralInfo**) is returned.
in	_pSignalInfoBuf	Pointer to the buffer where detailed description of each signal should be saved to.
inout	_pnSignalInfoBufSizeInBytes	Size of the given buffer. The minimum required size is written back.
return		Zero on success. Negative error or warning code..

More details This function returns information on the signal contents of each sample; use the library functions GetNextSamples(), GetLastSample() or ActivateAutoBufferMode() (or a sample data callback function; see the description of the callback function type SampleDataCallback() for details) to gain access to acquired samples.

© Precitec Optronik GmbH

7.43 Function GetConnectionPlugInInfo()

Description Get connection plugin information.

Prototype

```
Res_t GetConnectionPlugInInfo(
    Conn_h _hConnection,
    TPluginInfo * _aPluginInfos,
    LibSize_t * _pnBufSizeInBytes
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_aPluginInfos	Pointer to the buffer where plugin information should be save to (in the form of an array of **TPluginInfo** (alignment 4)).
inout	_pnBufSizeInBytes	Size of the given plugin information buffer in bytes. The minimum required size is written back.
return		number of plugins, or negative in case of an error or warning.

© Precitec Optronik GmbH

7.44 Function GetCurrentAvailableSampleCount()

Description Number of current available samples to be read.

Prototype

```
Res_t GetCurrentAvailableSampleCount(
    Conn_h _hConnection,
    int64_t * _pnSampleCount
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
out	_pnSampleCount	Current available sample count.
return		Zero on success. Negative error or warning code..

© Precitec Optronik GmbH

7.45 Function GetDeviceChannelCount()

Description Get device measuring channel count.

Prototype

```
Res_t GetDeviceChannelCount(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Negative: error or warning code.; 1: CHRocodile 1, CHRocodile C, CHRocodile 2; n: multi- channel devices, number of measurement channels (MPS) / points (CLS).

© Precitec Optronik GmbH

7.46 Function GetDeviceConnectionInfo()

Description Get device connection information.

Prototype

```
Res_t GetDeviceConnectionInfo(
    Conn_h _hConnection,
    char * _aConnInfoStr,
    LibSize_t * _pnSize
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_aConnInfoStr	Pointer to the string buffer where the connection information string should be saved to (zero termination included).
inout	_pnSize	Size of the given string buffer. The minimum required size is written back (including zero termination).
return		Zero on success. Negative error or warning code..

© Precitec Optronik GmbH

7.47 Function GetDeviceOutputSignals()

Description Get IDs of the signals output by the device.

Prototype

```
Res_t GetDeviceOutputSignals(
    Conn_h _hConnection,
    int32_t * _aSignalIDs,
    int32_t * _pSignalCount
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_aSignalIDs	Pointer to the integer buffer where ID of device output signals should be save to.
inout	_pSignalCount	In: the Size of the given integer buffer, out: the number of signals output by the device
return		Zero on success. Negative: error or warning code.

More details This function returns the identification codes of all peak and global signals that have been configured to be acquired and delivered by a device. The set of signals will comprise any signal that has previously been ordered, regardless of the connection used for ordering.

Note that your software must provide a buffer for storing the signal identification codes and pass the number of signals that this buffer can hold in a variable pointed to by the argument `_pSignalCount`. If a sample actually consists of more signals than the buffer can store, the GetDeviceOutputSignals() function will return an error code (`ERR_BUFSIZE_TOO_SMALL`); the required buffer size (number of signals) will be passed back in the variable pointed to by `_pSignalCount`.

© Precitec Optronik GmbH

7.48 Function GetDeviceType()

Description Get device type.

Prototype

```
int32_t GetDeviceType(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		negative: error or warning code; otherwise see Appendix A.6

© Precitec Optronik GmbH

7.49 Function GetHandleType()

Description Get handle type.

Prototype

```
int32_t GetHandleType(
    Handle_t _Handle
);
```

Arguments

in	_Handle	Handle whose type to be checked
return		handle type A.2..

© Precitec Optronik GmbH

7.50 Function GetLastSample()

Description Get the last valid sample from device.

Prototype

```
Res_t GetLastSample(
    Conn_h _hConnection,
    const double ** _ppData,
    LibSize_t * _pnSizePerSample,
    TSampleSignalGeneralInfo * _pSignalGeneralInfo,
    TSampleSignalInfo ** _pSignalInfo
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
out	_ppData	A pointer to the double sample buffer where data is saved.
out	_pnSizePerSample	A pointer to an integer variable which stores the size of each data sample.
out	_pSignalGeneralInfo	A pointer to the struct, which contains the general information of signals in data sample (see **TSampleSignalGeneralInfo**).
out	_pSignalInfo	A pointer to the array, where detailed description of each signal is stored one by one.
return		1: sample is available; 0: no sample has been received from device; negative: error or warning code.

© Precitec Optronik GmbH

More details

If a device has been configured to acquire and deliver samples (consisting of peak and/or global signals) and a **synchronous** connection with the device has been opened, calling the GetLastSample() function will retrieve the most recent sample data that the device has acquired.

When connected to a device, the CHRocodile library will store any sample data delivered by that device in an internal circular buffer. New sample data will be filled into the buffer continuously. By calling the GetLastSample() function, the read pointer of the circular buffer will be pushed to the last writing position, and at the same time the last saved sample will be copied into CHRocodileLib internal sampel buffer, whose address is passed to you through `_ppData`. Therefore all the previous unread data samples, if any, will be ignored/discarded.

The structure of sample data stored in the buffer pointed to by `_ppData` is as follows:

- For single channel devices, a sample will consist of a sequence of peak and/or global signal values in the exact order originally requested by means of an *SODX* (Set output data extended) command. If, for example, distance and intensity signals have been requested, the buffer contents would be:
  - Sample: Distance, Intensity
  - ...
- In the case of multi-channel devices, a sample will consist of a sequence of global signal values (if any global signals have been requested). The values of the requested peak signals are found after the global signal values; the peak signals will be ordered by channel. Consider the following example: The global signals requested are "Sample Counter" and "Exposure Count"; in addition, peak signals for intensity and distance have been ordered. The data structure in the buffer will be:
  - Sample: Sample Counter, Exposure Count, Intensity (channel #1), Distance (channel #1), Intensity (channel #2), Distance (channel #2), ...
  - ...

For more information on signals, see section
refsec:Signals

By default, all signal values are stored as double-precision floating point numbers—regardless of the "native" data type of a signal. The CHRocodile library will convert data for this purpose; for example, 16-Bit integer data representing a scaled fraction of the sensor's measurement range will be converted as double. The values of global signals representing 32-Bit counters will also be stored as double-precision floating point numbers; note that you may safely convert such values into integer numbers without a loss of resolution.

If you prefer using the "native", unprocessed data types of the signals, use the library function SetOutputDataFormatMode() to switch to "raw" mode. However, note that in "raw" mode, the values of different signal types may also differ in their bit lengths, hence the stored data will no longer be uniform double-precision floating point numbers and your software must ensure to access the buffer contents correctly, that is, according to the actual format and the bit length of a specific value. Details on the signals contained by each sample as well as information on the "native" data type of each signal can be found in the structures referred to by the `_pSignalGeneralInfo` and `_pSignalInfo` function arguments.

© Precitec Optronik GmbH

7.51 Function GetNextSamples()

Description Get the next available samples from the device.

Prototype

```
Res_t GetNextSamples(
    Conn_h _hConnection,
    int64_t * _pSampleCount,
    const double ** _ppData,
    LibSize_t * _pnSizePerSample,
    TSampleSignalGeneralInfo * _pSignalGeneralInfo,
    TSampleSignalInfo ** _pSignalInfo
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
inout	_pSampleCount	User passes over the number of samples to be read, lib gives back the actual number of read samples.
out	_ppData	A pointer to the double sample buffer where data is saved.
out	_pnSizePerSample	A pointer to an integer variable which stores the size of each data sample in byte.
out	_pSignalGeneralInfo	A pointer to the struct, which contains the general information of signals in data sample (see **TSampleSignalGeneralInfo**).
out	_pSignalInfo	A pointer to the array, where detailed description of each signal is stored one by one.
return		negative: error or warning code, 0 : present available data is fewer than the required number, 1 : successfully read in required number of the data, 2 : command response has been received, 3 : library data sample buffer is not large enough to store preset amount of data (A.10).

© Precitec Optronik GmbH

More details

If a device has been configured to acquire and deliver samples (consisting of peak and/or global signals) and a **synchronous** connection with the device has been opened, calling the GetNextSamples() function is one option to retrieve sample data. (The alternate option is using the "auto buffering" mode; see the description of the function ActivateAutoBufferMode() for details.) If your software uses an asynchronous connection, you must provide a callback function for processing sample data; see the description of the function RegisterSampleDataCallback() for more information on this topic.

When connected to a device, the CHRocodile library will store any sample data delivered by that device in an internal circular buffer. New sample data will be filled into the buffer continuously; by calling the GetNextSamples() function you can retrieve sample data from the buffer, starting with the oldest samples stored. You can specify the number of samples to be obtained in a call to GetNextSamples(); if the function completes successfully, it will return the following:

- A pointer to a library-internal memory location storing the sample data. (See description of the function parameter `_ppData`.)
- The number of samples actually obtained; this number can be less than the number of samples requested, in case there are too few "unread" samples available or CHRocodileLib sample buffer is not large enough or command response/update has been seen in the circular buffer. (See the description of the function parameter `_pSampleCount`.)

The structure of sample data stored in the buffer pointed to by `_ppData` is as follows:

- For single channel devices, a sample will consist of a sequence of peak and/or global signal values in the exact order originally requested by means of an *SODX* (Set output data extended) command. If, for example, distance and intensity signals have been requested, the buffer contents would be:
  - Sample #1: Distance, Intensity
  - Sample #2: Distance, Intensity
  - ...
- In the case of multi-channel devices, each sample will consist of a sequence of global signal values (if any global signals have been requested). The values of the requested peak signals are found after the global signal values; the peak signals will be ordered by channel. Consider the following example: The global signals requested are "Sample Counter" and "Exposure Count"; in addition, peak signals for intensity and distance have been ordered. The data structure in the buffer will be:
  - Sample #1: Sample Counter, Exposure Count, Intensity (channel #1), Distance (channel #1), Intensity (channel #2), Distance (channel #2), ...
  - Sample #2: Sample Counter, Exposure Count, Intensity (channel #1), Distance (channel #1), Intensity (channel #2), Distance (channel #2), ...
  - ...

For more information on signals, see section *Signals* of the CHRocodile Command Reference.

By default, all signal values are stored as double-precision floating point numbers—regardless of the "native" data type of a signal. The CHRocodile library will convert data for this purpose; for example, 16-Bit integer data representing a scaled fraction of the sensor's measurement range will be converted as double. The values of global signals representing 32-Bit counters will also be stored as double-precision floating point numbers; note that you may safely convert such values into integer numbers without a loss of resolution.

If you prefer using the "native", unprocessed data types of the signals, use

© Precitec Optronik GmbH

7.52 Function GetOutputDataFormatMode()

Description Get the mode of library output data format.

Prototype

```
Res_t GetOutputDataFormatMode(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		negative: error or warning code; 0 - double; 1 - raw.

© Precitec Optronik GmbH

7.53 Function GetResponseArgType()

Description Get response argument type.

Prototype

```
Res_t GetResponseArgType(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    int32_t * _pArgType
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArgType	A pointer to the integer variable receiving the argument type.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.54 Function GetResponseBlobArg()

Description Get binary response argument.

Prototype

```
Res_t GetResponseBlobArg(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    const int8_t ** _pArg,
    int32_t * _pLength
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArg	A pointer to the buffer where the binary data is restored. The buffer is maintained by the library.
out	_pLength	A pointer to the integer variable, where the size of the binary data.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.55 Function GetResponseFloatArg()

Description Get float response argument.

Prototype

```
Res_t GetResponseFloatArg(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    float * _pArg
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArg	A pointer to the float variable receiving the argument value.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.56 Function GetResponseFloatArrayArg()

Description Get float array response argument.

Prototype

```
Res_t GetResponseFloatArrayArg(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    const float ** _pArg,
    int32_t * _pLength
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArg	A pointer to the array, where the float array argument is returned. Array is maintained by the library.
out	_pLength	A pointer to the integer variable receiving the length of the float array argument.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.57 Function GetResponseInfo()

Description Get response information.

Prototype

```
Res_t GetResponseInfo(
    Rsp_h _hRsp,
    TResponseInfo * _pRspInfo
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
out	_pRspInfo	A pointer to the struct, where the response information (see **TResponseInfo**) is returned.
return		Zero on success. Negative error or warning code.

More details A command that has been sent to a device by calling the function ExecCommand() or ExecCommandAsync() will always cause the device to return a *command response*. A command response states whether the associated command was executed successfully or has raised some warning or error condition. In addition, a command response will also indicate the command parameters actually used by the device. (In some scenarios, a device may modify a parameter you have specified; for example, if the parameter value is out of range in the device's state.)

When the library receives a command response from a device, it will pass a response handle either as return value of the ExecCommand() function, or as argument to a user-supplied callback function of type ResponseAndUpdateCallback(). You may use such a response handle in a call to the GetResponseInfo() function to obtain general information on the command response. Detailed information about a command response can be retrieved by using the following functions:

- GetResponseArgType()
- GetResponseIntArg()
- GetResponseFloatArg()
- GetResponseStringArg()
- GetResponseBlobArg()
- GetResponseIntArrayArg()
- GetResponseFloatArrayArg()

© Precitec Optronik GmbH

7.58 Function GetResponseIntArg()

Description Get integer response argument.

Prototype

```
Res_t GetResponseIntArg(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    int32_t * _pArg
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArg	A pointer to the integer variable receiving the argument value.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.59 Function GetResponseIntArrayArg()

Description Get integer array response argument.

Prototype

```
Res_t GetResponseIntArrayArg(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    const int32_t ** _pArg,
    int32_t * _pLength
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArg	A pointer to the array, where the integer array argument is returned. Array is maintained by the library.
out	_pLength	A pointer to the integer variable receiving the length of the integer array argument.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.60 Function GetResponseStringArg()

Description Get string response argument.

Prototype

```
Res_t GetResponseStringArg(
    Rsp_h _hRsp,
    uint32_t _nIndex,
    const char ** _pArg,
    int32_t * _pLength
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_nIndex	Argument index, start with 0
out	_pArg	A pointer to the zero-terminated string containing the argument value
out	_pLength	A pointer to the integer variable, where the length of the string - including the terminating zero - is returned.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.61 Function GetSingleOutputSampleSize()

Description Get the size of each data sample output by the library.

Prototype

```
Res_t GetSingleOutputSampleSize(
    Conn_h _hConnection,
    LibSize_t * _pnSampleSize
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
out	_pnSampleSize	A pointer to an integer variable which stores the size of each data sample.
return		Zero on success. Negative error or warning code..

More details The sample size is the size of each single sample that you can access by means of the library functions GetNextSamples(), GetLastSample() or ActivateAutoBufferMode(), or that the library will pass to a sample data callback function. (See the description of the callback function type SampleDataCallback() also.)

© Precitec Optronik GmbH

7.62 Function IsCHRDeviceAutoSearchFinished()

Description Query whether CHRocodile device search in unblocking mode has been finished.

Prototype

```
Res_t IsCHRDeviceAutoSearchFinished();
```


© Precitec Optronik GmbH

7.63 Function LastErrors()

Description Get the information of the last errors (maximum 1024 for one connection)

Prototype

```
Res_t LastErrors(
    Conn_h _hConnection,
    TErrorInfo * _aErrorInfos,
    LibSize_t * _pnBufSizeInBytes
);
```

Arguments

in	_hConnection	When the valid connection handle is given, function returns the errors of that specific connection. When Invalid_Handle is passed, the errors of all the connections will be returned.
in	_aErrorInfos	Pointer to the buffer where error information should be save to (in the form of an array of **TErrorInfo** (alignment 4)).
inout	_pnBufSizeInBytes	Size of the given error buffer in bytes. The minimum required size is written back.
return		number of errors, or negative in case of an error or warning.

© Precitec Optronik GmbH

7.64 Function NewCommand()

Description Create new command.

Prototype

```
Res_t NewCommand(
    uint32_t _nCmdID,
    int32_t _bQuery,
    Cmd_h * _hCmd
);
```

Arguments

in	_nCmdID	Command ID.
in	_bQuery	Whether command is query or not.
out	_hCmd	A pointer to a variable receiving the command handle.
return		Zero on success. Negative error or warning code..

More details Calling this function will prepare a new command object inside the library and return a command handle representing this object. To add parameters to the command, you may use one of the following commands:

- AddCommandIntArg()
- AddCommandFloatArg()
- AddCommandStringArg()
- AddCommandBlobArg()
- AddCommandIntArrayArg()
- AddCommandFloatArrayArg()

Once all parameters (if any) have been added, the command can be transferred to the device by passing the command handle to either the ExecCommand() or the ExecCommandAsync() function. Note that ExecCommand() and ExecCommandAsync() will implicitly invalidate the command handle; you must not pass a command handle to a library function once that handle has been used in a call to ExecCommand() or the ExecCommandAsync() function.

Note that you can call NewCommand() multiple times for preparing several device commands (represented by different command handles) that can later be executed by calling ExecCommand() or ExecCommandAsync().

© Precitec Optronik GmbH

7.65 Function NewCommandFromString()

Description Translate string command to internal command and give back related command handle.

Prototype

```
int32_t NewCommandFromString(
    const char * _strCommand,
    Cmd_h * _hCmd
);
```

Arguments

in	_strCommand	Zero terminated string command.
out	_hCmd	A pointer to an unsigned integer variable receiving the command handle.
return		Zero on success. Negative error or warning code.

More details Similar to the NewCommand() function, calling this function will prepare a new command object inside the library and return a command handle representing this object. However, in contrast to NewCommand(), NewCommandFromString() identifies the command to be prepared by the mnemonic (such as “SHZ” or ”SODX”) passed in the `_strCommand` argument. In addition, all command parameters (if any) will also be extracted from the contents of the `_strCommand` argument. (The library expects the command parameters separated by space characters; for example “SODX 16640 83” or “ABE 0 155 32.5”.) Note also that floating point parameters are expected as a simple set of digits containing a radix point character (“.”); the scientific notation is *not* supported. (For example, “3.14” would be correct while “314e-2” is actually unsupported.)

The command prepared by a call to the NewCommandFromString() function can be transferred to the device by passing the command handle to either the ExecCommand() or the ExecCommandAsync() function. Note that ExecCommand() and ExecCommandAsync() will implicitly invalidate the command handle; you must not pass a command handle to a library function once that handle has been used in a call to ExecCommand() or the ExecCommandAsync().

Note that you can call NewCommandFromString() multiple times for preparing several device commands (represented by different command handles) that can later be executed by calling ExecCommand() or ExecCommandAsync().

© Precitec Optronik GmbH

7.66 Function OpenConnection()

Description Connect to CHRocodile device.

Prototype

```
Res_t OpenConnection(
    const char * _strConnectionInfo,
    int32_t _nDeviceType,
    int32_t _eConnectionMode,
    LibSize_t _nDevBufSizeInBytes,
    Conn_h * _pHandle
);
```

Arguments

in	_strConnectionInfo	A zero-terminated string that contains the connection information.
in	_nDeviceType	For a list of device types see A.6.
in	_eConnectionMode	Connection mode: synchornous or asynchronous A.7.
in	_nDevBufSizeInBytes	The size of library internal buffer for storing device output (only powers of 2 allowed (e.g. 16x1024x1024)). By default (when _nDevBufSizeInBytes<=0), 32MB.
out	_pHandle	A pointer to a variable receiving the connection handle. This handle is passed when calling other functions related to this connection.
return		Zero on success. Negative error or warning code..

© Precitec Optronik GmbH

More details

To be able to connect to a device, you must specify the communication details to be used by providing a character string:

- For connecting to a **CHRocodile 2**, **CHRocodile C**, **CHRocodile Mini** or **CHRocodile CLS/MPS/CLS2/MPS2** device, the library will use the "packet protocol" over a TCP/IP (Ethernet) connection. Hence, the communication string must specify the IP address of the device and optionally the packet protocol's IP port number (which is 7891). For example, a complete specification for a CHRocodile 2 device could be "`192.168.170.2, Port: 7891`". If you omit the IP port number and specify just the IP address, the library will default the port number to 7891.
- For connecting to a **CHRocodile 1** device, the library will use the older "dollar protocol" in binary mode. Hence, the communication string must name the computer's serial port to be used and optionally the baud rate and the handshake mode. (For example, a complete specification could be "`COM15, Baud: 921600, Handshake: Off`".) If the baud rate is not specified, the library will set the rate to 921600, if the handshake mode is not provided, the library will default it to `Handshake: On`. Note that the library will always use the binary-mode variant of the dollar protocol, hence the device command "ASC" for enabling the ASCII-variant will be blocked.

The CHRocodile library is also capable of looking up the communication details of devices connected to a computer; use the library functions StartCHRDeviceAutoSearch() and DetectedCHRDeviceInfo() for this purpose. Note that you may pass a communication details string received by a call to DetectedCHRDeviceInfo() directly as parameter `_strConnectionInfo` when calling OpenConnection(). In this case – and only in this case! – you can set the parameter `_nDeviceType` to "CHR_Unspecified".

A connection can be either *synchronous* or *asynchronous*, and its crucial to understand the differences between these two types. (Refer to chapter 2 for more information.)

© Precitec Optronik GmbH

7.67 Function OpenSharedConnection()

Description Open an extra connection shared with the exising one.

Prototype

```
Res_t OpenSharedConnection(
    Conn_h _nExistingConnection,
    int32_t _eConnectionMode,
    Conn_h * _pHandle
);
```

Arguments

in	_nExistingConnection	Handle of existing connection, which to be shared with.
in	_eConnectionMode	Connection mode: synchornous or asynchronous A.7.
out	_pHandle	A pointer to a variable receiving the connection handle. This handle is passed when calling other functions related to this connection.
return		Zero on success. Negative error or warning code..

More details A shared connection is always based on an existing connection and all shared connections will use the same communication channel (either a TCP/IP connection or a serial link) established by a call to the OpenConnection() function. In general, you may want to open a shared connection, for example the “original” connection is of synchronous type, but for specific purposes you also need an asynchronous connection. (Refer to chapter 2 for more information on connection types.)

When you close a connection (by calling the CloseConnection() function), the remaining connections are not affected and can still be used to communicate with a device. As soon as you close the last connection, the library will also terminate the communication link with the device.

© Precitec Optronik GmbH

7.68 Function PerformLibOperation()

Description Perform CHRocodileLib internal operations.

Prototype

```
Res_t PerformLibOperation(
    Conn_h _hConnection,
    Cmd_h _hCmd,
    Rsp_h * _phRsp
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection(), if Invalid_Handle is passed, the operation is performed in the CHRocodileLib, not related to any connection.
in	_hCmd	Command handle returned by NewCommand().
out	_phRsp	Pointer to a variable receiving the response handle.
return		Zero on success. Negative error or warning code.

© Precitec Optronik GmbH

7.69 Function PluginTypeIDToName()

Description Get plugin name string from plugin type.

Prototype

```
Res_t PluginTypeIDToName(
    uint32_t _nTypeID,
    char * _aPluginNameStr,
    LibSize_t * _pnSize
);
```

Arguments

in	_nTypeID	Plugin type ID.
in	_aPluginNameStr	Pointer to the string buffer where the zero-terminated plugin name string should be saved to (zero termination included).
inout	_pnSize	Size of the given string buffer. The minimum required size is written back (including zero termination).
return		Zero on success, negative in case of an error or warning.

© Precitec Optronik GmbH

7.70 Function ProcessDeviceOutput()

Description Process device output. All the data samples and command responses are passed through callback functions.

Prototype

```
Res_t ProcessDeviceOutput(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		negative: error or warning code, 0: timeout in waiting for preset amount of data to be read, 1: successfully read in preset amount of the data, 2: command response has been received, 3: data sample buffer is not large enough to store preset amount of data, 4: set user bufer is full, 5: data format has been changed (A.10).

© Precitec Optronik GmbH

More details

In case your software is using an asynchronous connection, use this function to retrieve acquired samples and command responses (and/or command updates). Your code should call ProcessDeviceOutput() in regular, short intervals to make sure that data from the device gets handled promptly.

There are two options for processing device output when using asynchronous connections:

1. Let the library process any device output automatically in the background; acquired samples and command responses/updates will be passed to user-defined callback routines. (See the description of the functions StartAutomaticDeviceOutputProcessing() and StopAutomaticDeviceOutputProcessing() for more information.)
2. Explicitly trigger the processing of device output by calling ProcessDeviceOutput(). Again, acquired samples and command responses/updates will be passed to user-defined callback routines.

In any case, the library will invoke the callback function registered by a previous call to the RegisterSampleDataCallback() function when new sample data has been retrieved from a device; command responses and command updates received will be passed to the callback function registered previously by a call to the RegisterGeneralResponseAndUpdateCallback() function.

Each time you call ProcessDeviceOutput(), the library will attempt to retrieve the number of samples you have specified when setting up the sample data callback by calling RegisterSampleDataCallback(). Consequently, ProcessDeviceOutput() will return if either of the following conditions are true:

- The configured number of samples has been acquired. (The function will return `Read_Data_Success` in this case.)
- The function timed-out while waiting for the configured number of samples to become acquired. (The function's return code will be `Read_Data_Not_Enough`.) You may specify the maximum period for which ProcessDeviceOutput() will wait for samples to arrive when setting up the sample data callback by calling the function RegisterSampleDataCallback().
- The library encounters a command response or a command update while processing retrieved sample data. In this case, it may assume that the sample data received before the response/update reflects a different device state than the sample data that will arrive next. To prevent inconsistent sample data in such a scenario, ProcessDeviceOutput() will terminate and return `Read_Data_Response` as result code.
- The library-internal sample data buffer is completely filled. (In this case, the ProcessDeviceOutput() function will return `Read_Data_Buffer_Small`. If the library is set not to automatically adjust the size of the sample data buffer, you can explicitly size the buffer by calling the function SetSampleDataBufferSize().
- The passed user buffer is full (User function ActivateAsyncDataUserBuffer() to set the user buffer). (The function's return code will be `Read_Data_User_Buf_Full`.)
- Data format has been changed since the user buffer is activated (User function ActivateAsyncDataUserBuffer() to activate the user buffer). (The function's return code will be `Read_Data_Format_Change`.)
- An error condition occurs. (In this case, the function's return code will be a negative number.)

Please refer to chapter 2.2 for more information on the device output processing options.

© Precitec Optronik GmbH

7.71 Function RegisterGeneralResponseAndUpdateCallback()

Description Register callback function for all the commands.

Prototype

```
Res_t RegisterGeneralResponseAndUpdateCallback(
    Conn_h _hConnection,
    void * _pUser,
    ResponseAndUpdateCallback _pGenCBFct
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_pUser	Can be freely used to forward user information to the function _pCB points to. Passed to _pCB() but not interpreted by the library. See also section 2.3 of the reference manual.
in	_pGenCBFct	Callback function for all the responses/updates (see ResponseAndUpdateCallback).
return		Zero on success. Negative error or warning code.

More details In case your software is using an asynchronous connection, you may specify a dedicated callback routine that the library will execute each time it has received a command response or a command update from the device. (Note that you may also specify a callback routine when calling the library function ExecCommandAsync(); however, such a callback routine will only be executed once, that is, when the response to the command referenced by ExecCommandAsync() has arrived.)

If a general response/update callback routine has been registered, and an additional callback routine is specified in a call to the ExecCommandAsync() function, the CHRocodile library will first execute the additional callback routine as soon as the device responds to the command. Next, when this additional callback routine has returned, the library will execute the general response/update callback routine.

Note also that you must register a callback routine using the RegisterGeneralResponseAndUpdateCallback() function if your software shall receive and process command updates that a device transmits unsolicitedly.

WHERE TO FIND INFO ON RESPONSES AND UPDATES?

© Precitec Optronik GmbH

7.72 Function RegisterSampleDataCallback()

Description

Register data sample callback function and set data processing related settings.

Prototype

```
Res_t RegisterSampleDataCallback(
    Conn_h _hConnection,
    int64_t _nMaxSampleCount,
    int32_t _nReadSampleTimeOut,
    void * _pUser,
    SampleDataCallback _pOnReadSample
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_nMaxSampleCount	Maximum number of samples to be read during single device output processing.
in	_nReadSampleTimeOut	Timeout in millisecond for waiting for _nReadSampleCount of samples. If _nReadSampleTimeOut is set to 0, only available samples are read without waiting.
in	_pUser	Can be used to carry user information (see 2.3). It is not interpreted by the library. When _pOnReadSample is called, this value will be passed into it.
in	_pOnReadSample	Sample callback function to pass over samples (format see SampleDataCallback), which is called either by function ProcessDeviceOutput() or by library automatic output processing.
return		Zero on success. Negative error or warning code..

© Precitec Optronik GmbH

More details

Using the RegisterSampleDataCallback() function you can specify a callback routine that the library will invoke each time new sample data has been retrieved from a device. (When using an asynchronous connection, this is the only method to inform an application about the availability of sample data the device has acquired.) The callback routine you specify will either be called if a maximum number of samples you specify has been acquired, or a designated time-out period has expired.

The callback function can access the sample data in the following manner:

- If you have previously called the ActivateAsyncDataUserBuffer() function for specifying an application-supplied sample buffer, you will find the acquired sample data in this buffer. (The library will pass a pointer to this buffer to the callback function also.)
- If you have just registered a callback function (but don't use an application-supplied data buffer), the library will pass a pointer to a library-internal memory location where stores the most recent samples from the device. Note that in this case, accessing sample data is only "safe" while your callback function is still executing. Hence, the function must either process new sample data immediately, or copy the data from the library-internal memory location to an application buffer for later processing. (Because the CHRocodile Library will store sample data only temporarily, your application might otherwise lose sample data.)

© Precitec Optronik GmbH

7.73 Function ResponseToString()

Description Output response in string format.

Prototype

```
Res_t ResponseToString(
    Rsp_h _hRsp,
    char * _aResponseStr,
    LibSize_t * _pnSize
);
```

Arguments

in	_hRsp	Response handle, either returned by ExecCommand() or passed through response and update callback function.
in	_aResponseStr	Pointer to the buffer where the response string should be saved to (zero- termination is included).
inout	_pnSize	Size of the given string buffer. If it is not large enough or _aResponseStr is not assigned, the minimum required size is written back (including zero termination).
return		Zero on success. Negative error or warning code.

More details This function will convert a command response (represented by the passed response handle) to a text string consisting of the command mnemonic preceded by a dollar symbol, followed by the response's arguments separated by semicolons (`$Command Argument #1; Argument #2; ...`). The downloaded binary data will not be included in the returned string, only the string "binary data" will be shown. Note that if the device returned an error condition, the respective message may be part of the text string. (For example, `$NOP 1; 2; 3; Invalid parameter count: Expected 1 or 2 but actual count is 3`)

© Precitec Optronik GmbH

7.74 Function SetIniFile()

Description

Set the ini file for the Lib. This function should be called right after loading the library. If another ini file was previously loaded or a connection has been openend, results in an error.

Prototype

```
Res_t SetIniFile(
    FilePath_t _pstrIniFile
);
```

Arguments

in	_pstrIniFile	The ini file to be used.
return		Zero on success. Non-zero otherwise.

More details

Various parameters affecting the CHRocodile library's performance and behavior can be configured by means of an *initialization file* (also referred to as "ini file".) In general, your software does not need to call SetIniFile(); it is strongly recommended to use this function only if advised by Precitec service personnel.

To ensure that the library gets actually configured correctly, application software must call the SetIniFile() function as early as possible, before executing other library functions. Note also that if another initialization file has been loaded previously, or any device connection has been opened, SetIniFile() will fail and result in an error condition.

© Precitec Optronik GmbH

7.75 Function SetLibConfigFlags()

Description Set library configuration flags.

Prototype

```
void SetLibConfigFlags(
    uint32_t _nFlag
);
```

Arguments

in	_nFlag	CHRLib_Flag_Rsp_Deactivate_Auto_Buffer: set library to automatically quit auto buffer save mode when response/update has been received from the device, by default this flag is not set. CHRLib_Flag_Auto_Change_Data_Buffer_Size: set library to automatically resize library internal data sample buffer to store preset number of data samples in RegisterSampleDataCallback or GetNextSamples(), by default this flag is set. CHRLib_Flag_DataFormat_Deactivate_Auto_Buffer. set library to automatically quit auto buffer save mode when data format packet has been received from the device, by default this flag is not set.

More details Call the SetLibConfigFlags() function to fine-tune some aspects of the library's data acquisition and function execution behavior. You can set more than one flag by combining the flag constants using the bitwise-OR operator. To clear a flag, negate its flag constant by means of the bitwise-NOT operator.

Note that calling SetLibConfigFlags() will take effect immediately; that is, subsequent library operations will use the new flag settings.

In general, your software does not need to call SetLibConfigFlags(); it is recommended to use this function only if requested by Precitec service personnel.

© Precitec Optronik GmbH

7.76 Function SetLibLogFileDirectory()

Description Set log file directory property.

Prototype

```
Res_t SetLibLogFileDirectory(
    FilePath_t _pstrDirectory,
    LibSize_t _nMaxFileSizeInKiB,
    int32_t _nMaxFileNumber
);
```

Arguments

in	_pstrDirectory	Log file directory.
in	_nMaxFileSizeInKiB	Maximum size of one log file in KiB.
in	_nMaxFileNumber	Maximum number of log files in the directory. If in the log directory, the number of log files are larger than _nMaxFileNumber, the oldest log file(s) will be automatically deleted.
return		Zero on success. Negative: error or warning code.

More details The CHRocodile library keeps specific log files storing information about internal states and events. By calling the function SetLibLogFileDirectory() one may specify the storage device and directory where these log files will be stored. Without calling this function, the internal information will not be saved. Therefore this funtion should be called as early as possible.

Calling SetLibLogFileDirectory() will change the log file directory immediately; messages generated by the library subsequently will be stored in a new log file created in the specified directory.

© Precitec Optronik GmbH

7.77 Function SetLibLogLevel()

Description

Set the level for logging library messages. The message which has the higher or equal level will be logged.

Prototype

```
Res_t SetLibLogLevel(
    int32_t _nLevel
);
```

Arguments

in	_nLevel	Minimum logging level. By default, the logging level is 2.
return		Zero on success. Non-zero otherwise.

More details

The CHRocodile library keeps specific log files storing information about internal states and events. By calling the function SetLibLogLevel() one may specify a threshold level determining which messages generated by the library will get logged.

Calling SetLibLogLevel() will take effect immediately; subsequent messages generated by the library will only get logged if their severity level is equal to (or exceeds) the specified threshold.

In general, your software does not need to call SetLibLogLevel(); it is recommended to use this function only if requested by Precitec service personnel.

© Precitec Optronik GmbH

7.78 Function SetMultiChannelProfileInterpolation()

Description Set profile interpolation parameter for CLS device.

Prototype

```
Res_t SetMultiChannelProfileInterpolation(
    Conn_h _hConnection,
    int32_t _nMaxHoleSize
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_nMaxHoleSize	Maximum hole size/missing sample point number supported by the interpolation.
return		Zero on success. Negative: error or warning code.

More details The library provides a special function to assist interpolating "missing" data in a line profile acquired by a CLS or CLS2 device. Each sample acquired by such a device consists of multiple data points (192 in case of a CLS device, 600 or 1200 in case of a CLS2 device) that form a line profile. However, for a couple of reasons, one or more of such a profile's data points may actually be invalid, thus causing a gap (or "hole") in the data sequence. The interpolation function provided by the library can help to close such gaps by using adjacent valid data points for interpolating correct values that substitute the invalid values:

- In case the first and/or last data point of a profile are invalid, the interpolation algorithm will substitute them by the first/last valid data points.
- For gaps caused by invalid data points at arbitrary other positions in the profile, the algorithm will connect the adjacent valid data points by a straight line, and compute the substitute values based on this line.

The interpolation algorithm applies to distance or thickness peak signals but not for other signals. Use the function parameter `_nMaxHoleSize` to specify the maximum number of consecutive invalid data points (the maximum width of a gap); if the algorithm detects a gap that exceeds the specified maximum width, it will not attempt to close it by interpolation, hence the gap will remain unprocessed. To disable the interpolation completely, set `_nMaxHoleSize` to 0.

© Precitec Optronik GmbH

7.79 Function SetOutputDataFormatMode()

Description Set the mode of library output data format **(Use carefully!)**.

Prototype

```
Res_t SetOutputDataFormatMode(
    Conn_h _hConnection,
    int32_t _eMode
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_eMode	Output data format mode: 0 - double; 1 - raw.
return		Zero on success. Negative error or warning code..

More details By default, all signal values a sample consists of are provided as double-precision floating point numbers—regardless of the “native” data type of a signal. The CHRocodile library will convert data for this purpose; for example, 16-Bit integer data representing a scaled fraction of the sensor’s measurement range will be converted als double. The values of global signals representing 32-Bit counters will also be stored as double-precision floating point numbers; note that you may safely convert such values into integer numbers without a loss of resolution. If you prefer using the “native”, unprocessed data types of the signals, use the library function SetOutput-DataFormatMode() to switch to “raw” mode. In “raw” mode, a signal value occupies less memory—because the “native” signal data types often require two or four bytes of memory, while a double-precision floating point number require eight bytes—; however, you must make sure that your software accesses signal values in sample data correctly (that is, with respect to the correct data type), and that signal values are interpreted correctly.

The following library functions and mechanisms can be used to retrieve sample data:

- GetNextSamples() (for synchronous connections)
- GetLastSample() (for synchronous connections)
- SampleDataCallback() (callback function, used for asynchronous connections)
- “Auto buffering” mode (see the descriptions of the library function ActivateAutoBufferMode() and ActivateAsyncDataUserBuffer() for more information.)

By default, all of the above functions and mechanisms will provide sample data as a uniform sequence (actually an array) of double-precision floating point numbers, hence addressing signal values is straightforward. In contrast, when switching to “raw” mode, the values of different signal types can also differ in their bit lengths, hence the stored data will no longer be uniform and your software must ensure to access the buffer contents correctly, that is, according to the actual format and the bit length of a specific value. Details on the signals contained by each sample as well as information on the “native” data type of each signal will usually be provided by the library function (for example, GetNextSamples()).

In general, unless memory consumption is an issue for your application, Precitec recommends to use double-precision floating point numbers as signal value data format.

 © Precitec Optronik GmbH

7.80 Function SetSampleDataBufferSize()

Description Set the size of buffer to store data sample.

Prototype

```
Res_t SetSampleDataBufferSize(
    Conn_h _hConnection,
    LibSize_t _nBufferSizeInBytes
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection(). If Invalid_Handle is passed, ths buffer size is applied to all the connections.
in	_nBufferSizeInBytes	Size of data sample buffer.
return		Zero on success. Negative error or warning code..

More details For each connection, the library copies the data in the internal circular buffer to a sample data buffer when data is retrievedd. (The contents of this buffer will either passed to a callback function of type SampleDataCallback() (if your application has not supplied an own sample buffer), or can be retrieved by using the functions GetNextSamples() and GetLastSample().) Initially, the size of this buffer is 512 kilobytes (kB).

By default, the library will automatically adjust the size of the buffer with respect to the maximum number of samples requested by your software (see the description of the functions GetNextSamples() and RegisterSampleDataCallback() respectively for more detailed information.) However, if your application must excercise the control over memory consumption explicitly for some reason, you can determine a fixed size the library shall use for each sample data buffer. To explicitly size the sample data buffer, do the following:

1. Call the function SetLibConfigFlags() to clear the flag `CHRLib_Flag_Auto_Change_Data_Buffer_Size`.
2. Call SetSampleDataBufferSize(), passing the connection for which the buffer size shall be set or Invalid_handle for all the connections, and the new buffer size.

© Precitec Optronik GmbH

7.81 Function StartAutomaticDeviceOutputProcessing()

Description Start library automatic processing of device output.

Prototype

```
Res_t StartAutomaticDeviceOutputProcessing(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative error or warning code..

More details In case your software is using an asynchronous connection, you may use either of two options for processing device output such as acquired samples, command responses and command updates:

1. Explicitly trigger the processing of device output by calling the function ProcessDeviceOutput(). Acquired samples and command responses/updates will be passed to user-defined callback routines.
2. Let the library process any device output automatically in the background; acquired samples and command responses/updates will also be passed to user-defined callback routines.

In any case, the library will pass acquired sample data to the callback function registered by a previous call to the RegisterSampleDataCallback() function; command responses and command updates retrieved will be passed to the callback function registered previously by a call to the RegisterGeneralResponseAndUpdateCallback() function.

By calling the function StartAutomaticDeviceOutputProcessing(), the CHRocodile library will create a new thread for the automatic device output processing in the background. Once the function returns, your application does not need to take care about periodically triggering the library-internal output processing any longer. (Consequently, your code must *not* call the library function ProcessDeviceOutput().)

With automatic device output processing enabled, sample data will be passed to a callback function of type SampleDataCallback(), while command responses and command updates will be passed to a callback function of type ResponseAndUpdateCallback().

To terminate the automatic processing of device output, call the function StopAutomaticDeviceOutputProcessing().

© Precitec Optronik GmbH

7.82 Function StartCHRDeviceAutoSearch()

Description Start automatic CHRocodile device search.

Prototype

```
Res_t StartCHRDeviceAutoSearch(
    int32_t _nConnectionType,
    int32_t _bSBlockingSearch
);
```

Arguments

in	_nConnectionType	0: search both serial and TCPIP connection; 1: search only serial connection; 2: search only TCPIP connection for Chrocodile connection (see A.18).
in	_bSBlockingSearch	Whether start search function return immediately (unblocking search) or wait until search is finished (blocking search).
return		Zero on success. Negative: error or warning code.

More details When device detection is already running in the background (that is, StartCHRDeviceAutoSearch() has already been called with the `_bSBlockingSearch` argument set to 'false'), you must not initiate another device detection operation. Use the IsCHRDeviceAutoSearchFinished() function to ensure that a previously device launched detection operation has terminated before calling StartCHRDeviceAutoSearch() again.

You can terminate a device detection operation running in the background by calling the function CancelCHRDeviceAutoSearch().

Once device detection has completed, call the function DetectedCHRDeviceInfo() to retrieve information on the detected device(s).

© Precitec Optronik GmbH

7.83 Function StartConnectionDataStream()

Description Start data stream for one specific connection.

Prototype

```
Res_t StartConnectionDataStream(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative error or warning code..

More details When the sample data stream of a specific connection has not been activated (or previously been stopped by calling the StopConnectionDataStream() library function), you may resume the stream by calling StartConnectionDataStream(). (All other connections and the data acquisition status of the device are not affected.) This has the following implications:

- If the specified connection is a synchronous connection, the functions GetNextSamples(), GetLastSample() and ActivateAutoBufferMode() can be used in conjunction with the connection.
- If the specified connection is an asynchronous connection, a registered SampleDataCallback() will be invoked when new sample data has been received from a device. In addition, if you have activated a dedicated sample data buffer, the library will start filling this buffer with the sample data received. (See the description of the function ActivateAsyncDataUserBuffer() for more information.)

© Precitec Optronik GmbH

7.84 Function StopAutomaticDeviceOutputProcessing()

Description Stop library automatic processing of device output.

Prototype

```
Res_t StopAutomaticDeviceOutputProcessing(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative error or warning code..

More details Calling this function will terminate the automatic device output processing in the background (started by a previous call to the StartAutomaticDeviceOutputProcessing() function.) Once the function returns, your application must take care about triggering the library-internal device output processing explicitly; see the description of the function ProcessDeviceOutput() for more information.

© Precitec Optronik GmbH

7.85 Function StopConnectionDataStream()

Description Stop data stream for one specific connection.

Prototype

```
Res_t StopConnectionDataStream(
    Conn_h _hConnection
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
return		Zero on success. Negative error or warning code..

More details Calling this function will stop the sample data stream for the specified connection. (All other connections and the data acquisition status of the device are not affected.) This has the following implications:

- If the specified connection is a synchronous connection, calling GetNextSamples(), GetLastSample() or ActivateAutoBufferMode() will result in an error condition.
- If the specified connection is an asynchronous connection, any registered SampleDataCallback() will no longer be called. In addition, if you have activated a dedicated sample data buffer by calling the function ActivateAsyncDataUserBuffer(), the library will stop filling this buffer.

To resume a sample data stream, use the StartConnectionDataStream() function.

© Precitec Optronik GmbH

7.86 Function UploadConfocalCalibrationTableFromFile()

Description Synchronously upload confocal calibration table from file to device.

Prototype

```
Res_t UploadConfocalCalibrationTableFromFile(
    Conn_h _hConnection,
    FilePath_t _strFullFileName,
    uint32_t _nProbeSerialNumber,
    uint32_t _nCHRTableIndex
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_strFullFileName	Calibration table file name (full name including file path).
in	_nProbeSerialNumber	Sensor probe serial number.
in	_nCHRTableIndex	Index of calibration table stored in device (from 0 to 15).
return		Zero on success. Negative: error or warning code.

More details Note that accidental erroneous usage of this function may cause device malfunction because of incorrect probe calibration! If in doubt, contact Precitec Optronik for assistance.

© Precitec Optronik GmbH

7.87 Function UploadFirmware()

Description Synchronously upload firmware file to device.

Prototype

```
Res_t UploadFirmware(
    Conn_h _hConnection,
    FilePath_t _strFullFileName
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_strFullFileName	Firmware file name (full name including file path).
return		Zero on success. Negative: error or warning code.

More details Use this function to upload and install new firmware to the device represented by the synchronous connection `_hConnection`. Depending on the device type and the communication link, the function will run for a longer period of time (up to several minutes!)

After the upload procedure has completed, the firmware will be installed and the device will restart. Note that the library will re-establish the connection with the device automatically; your software is not required to handle this explicitly.

© Precitec Optronik GmbH

7.88 Function UploadMultiChannelMaskTable()

Description Synchronously upload mask table from file to multi-channel device.

Prototype

```
Res_t UploadMultiChannelMaskTable(
    Conn_h _hConnection,
    FilePath_t _strFullFileName
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_strFullFileName	Mask table file name (full name including file path).
return		Zero on success. Negative: error or warning code.

More details Note that accidental erroneous usage of this function may cause device malfunction because of incorrect sensor adjustment! If in doubt, contact Precitec Optronik for assistance.

© Precitec Optronik GmbH

7.89 Function UploadRefractiveIndexTableFromFile()

Description Synchronously upload refractive index table from file to device.

Prototype

```
Res_t UploadRefractiveIndexTableFromFile(
    Conn_h _hConnection,
    FilePath_t _strFullFileName,
    const char * _strTableName,
    uint32_t _nCHRTableIndex,
    float _nRefSRI
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_strFullFileName	Refractive index table file name (full name including file path).
in	_strTableName	Table name.
in	_nCHRTableIndex	Table index stored in device (from 1 to 16).
in	_nRefSRI	Reference refractive index (see below).
return		Zero on success. Negative: error or warning code.

More details Note that the value of the argument `_nRefSRI` will only be used if the library is connected to a CHRocodile 1 device. If you are using any CHRocodile 2, a CHRocodile Mini, a CHRocodile C or a CHRocodile CLS2/MPS2 device, you can safely set this argument to an arbitrary value.

Please note also that accidental erroneous usage of this function may cause device malfunction because of incorrect sensor adjustment! If in doubt, contact Precitec Optronik for assistance.

© Precitec Optronik GmbH

7.90 Function UploadWaveLengthTableFromFile()

Description Synchronously upload wavelength table from file to device.

Prototype

```
Res_t UploadWaveLengthTableFromFile(
    Conn_h _hConnection,
    FilePath_t _strFullFileName
);
```

Arguments

in	_hConnection	Connection handle returned by OpenConnection() or OpenSharedConnection().
in	_strFullFileName	Wavelength table file name (full name including file path).
return		Zero on success. Negative: error or warning code.

More details Note that accidental erroneous usage of this function may cause device malfunction because of incorrect sensor adjustment! If in doubt, contact Precitec Optronik for assistance.

© Precitec Optronik GmbH

Appendix A

Predefined Constants

A.1 Result Codes

Please look at the `CHRocodileLibErrorDef.h` header for more details.

```
#define RES_LEVEL_SUCCESS 0
#define RES_LEVEL_INFORMATION 1
#define RES_LEVEL_WARNING 2
#define RES_LEVEL_ERROR 3
#define CHR_SUCCESS ((res) >= 0)
#define CHR_INFORMATION \
    ((((uint32_t)res) >> 30) == RES_LEVEL_INFORMATION)
#define CHR_WARNING ((((uint32_t)res) >> 30) == RES_LEVEL_WARNING)
#define CHR_ERROR ((((uint32_t)res) >> 30) == RES_LEVEL_ERROR)
#define SCS_SUCCESS ((int32_t)0x00000000L)
#define WRN_CONNECTION_ATTEMPT_UNDERWAY ((int32_t)0xA0041400L)
#define WRN_AUTOFLUSH_OCCURRED ((int32_t)0xA0047600L)
#define WRN_SIGNAL_CHANGED ((int32_t)0xA0049000L)
#define ERR_ALLOCATE ((int32_t)0xE0000C00L)
#define ERR_FREE ((int32_t)0xE0000D00L)
#define ERR_REALLOCATE ((int32_t)0xE0000E00L)
#define ERR_WRONG_HANDLE_TYPE ((int32_t)0xE0001E00L)
#define ERR_INVALID_HANDLE ((int32_t)0xE0001F00L)
#define ERR_NULLPTR ((int32_t)0xE0002000L)
#define ERR_BUFSIZE_TOO_SMALL ((int32_t)0xE0002100L)
#define ERR_UNCAUGHT ((int32_t)0xE0006300L)
#define ERR_BUFSIZE_POWER_OF_TWO ((int32_t)0xE0008600L)
#define ERR_UNKNOWN ((int32_t)0xE000FF00L)
#define ERR_OPEN_FILE ((int32_t)0xE0010A00L)
#define ERR_FILE_ALREADY_EXISTS ((int32_t)0xE0011400L)
#define ERR_NO_SUCH_FILE ((int32_t)0xE0011500L)
#define ERR_OPEN_COMPORT ((int32_t)0xE0020A00L)
#define ERR_COMPORT_WRITE_TIMEOUT ((int32_t)0xE0022800L)
#define ERR_READ_COMPORT ((int32_t)0xE0023200L)
#define ERR_WRITE_COMPORT ((int32_t)0xE0023300L)
#define ERR_GET_STATE_COMPORT ((int32_t)0xE0023400L)
#define ERR_SET_STATE_COMPORT ((int32_t)0xE0023500L)
#define ERR_FLUSH_SERIAL_CONNECTION_STREAM ((int32_t)0xE0026500L)
#define ERR_CONNECT_SOCKET ((int32_t)0xE0030A00L)
#define ERR_ALLOCATE_SOCKET ((int32_t)0xE0030C00L)
#define ERR_READ_CLOSED_SOCKET ((int32_t)0xE0031600L)
#define ERR_SOCKET_TIMEOUT ((int32_t)0xE0032800L)
#define ERR_READ_SOCKET ((int32_t)0xE0033200L)
#define ERR_WRITE_SOCKET ((int32_t)0xE0033300L)
#define ERR_GET_STATE_SOCKET ((int32_t)0xE0033400L)
#define ERR_SET_STATE_SOCKET ((int32_t)0xE0033500L)
#define ERR_SELECT_SOCKET ((int32_t)0xE0036600L)
#define ERR_CONNECTION_NOT_OPEN ((int32_t)0xE0041500L)
```


© Precitec Optronik GmbH

```
#define ERR_WRONG_CONNECTION_MODE ((int32_t)0xE0041E00L)
#define ERR_UNSUPPORTED_DEVICE_FUNCTIONALITY ((int32_t)0xE0042200L)
#define ERR_RESPONSE_TIMEOUT ((int32_t)0xE0042900L)
#define ERR_COMMAND_SEND_TIMEOUT ((int32_t)0xE0042A00L)
#define ERR_RESYNC_FAILED ((int32_t)0xE0046900L)
#define ERR_SIGNAL_MISSING ((int32_t)0xE0046A00L)
#define ERR_CONNECTION_OUT_OF_SYNC ((int32_t)0xE0046D00L)
#define ERR_AUTOFLUSH_FAILED ((int32_t)0xE0047700L)
#define ERR_AUTO_PROCESS_ACTIVE ((int32_t)0xE0048400L)
#define ERR_AUTO_SAVE_ACTIVE ((int32_t)0xE0048500L)
#define ERR_CONNECTION_INVALID_PARAMETER ((int32_t)0xE0048700L)
#define ERR_CONNECTION_UNKNOWN_DEVTYPE ((int32_t)0xE0048800L)
#define ERR_DATASTREAM_STOPPED ((int32_t)0xE0048900L)
#define ERR_NO_SIGNALS_SELECTED ((int32_t)0xE0048A00L)
#define ERR_DEVSEARCH_ALLOCATE ((int32_t)0xE0050C00L)
#define ERR_DEVSEARCH_REALLOCATE ((int32_t)0xE0050E00L)
#define ERR_DEVSEARCH_UNDERWAY ((int32_t)0xE0051400L)
#define ERR_DEVSEARCH_CANCELED ((int32_t)0xE0052B00L)
#define ERR_DEVSEARCH_GETIFADDRS ((int32_t)0xE0056E00L)
#define ERR_DEVSEARCH_UPNP_ALLOCATE_SEND_SOCKET ((int32_t)0xE0056F00L)
#define ERR_DEVSEARCH_UPNP_ALLOCATE_RECV_SOCKET ((int32_t)0xE0057000L)
#define ERR_DEVSEARCH_UPNP_SET_IFADDR ((int32_t)0xE0057100L)
#define ERR_DEVSEARCH_UPNP_BIND_MULTICAST ((int32_t)0xE0057200L)
#define ERR_DEVSEARCH_UPNP_REUSE_PORT ((int32_t)0xE0057300L)
#define ERR_DEVSEARCH_UPNP_ADD_MEMBERSHIP ((int32_t)0xE0057400L)
#define ERR_DEVSEARCH_UPNP_SEARCH ((int32_t)0xE0057500L)
#define ERR_DEVSEARCH_UNKNOWN ((int32_t)0xE005FF00L)
#define ERR_UNSUPPORTED_FUNCTIONALITY ((int32_t)0xE0062200L)
#define ERR_CLIENT_READER_ERROR ((int32_t)0xE0066700L)
#define ERR_INTERNAL_READER_ERROR ((int32_t)0xE0066800L)
#define ERR_DATAFMT_MISSING ((int32_t)0xE0066B00L)
#define ERR_DATAFMT_INVALID ((int32_t)0xE0066C00L)
#define ERR_LIB_STREAM_OUT_OF_SYNC ((int32_t)0xE0066D00L)
#define ERR_UNSUPPORTED_FUNCTIONALITY_DEBUG ((int32_t)0xE0067D00L)
#define ERR_WRONG_THREAD ((int32_t)0xE0067E00L)
#define ERR_SPLINE_CALC_FAILED ((int32_t)0xE0068000L)
#define ERR_AUTO_BUFFER_MODE_TIMEOUT ((int32_t)0xE0068D00L)
#define ERR_UNSUPPORTED_PLUGIN ((int32_t)0xE0068E00L)
#define ERR_INI_FILE_LOAD_DISALLOWED ((int32_t)0xE0068F00L)
#define ERR_WSASTARTUP ((int32_t)0xE0076400L)
#define ERR_TONUMBER ((int32_t)0xE0083C00L)
#define ERR_TOSTRING ((int32_t)0xE0083D00L)
#define ERR_CMD_RESPONSE ((int32_t)0xE0097800L)
#define ERR_CMD_RESPONSE_PARAM_INDEX ((int32_t)0xE0097A00L)
#define ERR_CMD_RESPONSE_PARAM_TYPE ((int32_t)0xE0097B00L)
#define ERR_CMD_RESPONSE_ID_DIFFERENT ((int32_t)0xE0097C00L)
#define ERR_FIRMWARE_UPLOAD ((int32_t)0xE0097F00L)
#define ERR_CMDBUF_EMPTY ((int32_t)0xE0098100L)
#define ERR_CMDBUF_INVALID_READPOS ((int32_t)0xE0098200L)
#define ERR_CMDBUF_INVALID_TICKET ((int32_t)0xE0098300L)
#define ERR_CMD_INVALID_PARAM ((int32_t)0xE0098B00L)
#define ERR_CMD_INVALID_FILE ((int32_t)0xE0098C00L)
#define ERR_CMD_UNKNOWN ((int32_t)0xE009FF00L)
```


© Precitec Optronik GmbH

A.2 Constants: Handle Types

```
const Handle_t Invalid_Handle = ((Handle_t)-1);
const int32_t Handle_Unknown_Type = -1;
const int32_t Handle_Connection = 0;
const int32_t Handle_Command = 1;
const int32_t Handle_Response = 2;
const int32_t Handle_Plugin = 3;
```

A.3 Constants: Sample Data Types

```
const int16_t Data_Type_Unsigned_Char = 0;
const int16_t Data_Type_Signed_Char = 1;
const int16_t Data_Type_Unsigned_Short = 2;
const int16_t Data_Type_Signed_Short = 3;
const int16_t Data_Type_Unsigned_Int32 = 4;
const int16_t Data_Type_Signed_Int32 = 5;
const int16_t Data_Type_Float = 6;
const int16_t Data_Type_Double = 255;
```

A.4 Constants: Response flag

```
const uint32_t Rsp_Flag_Query = 0x0001;
const uint32_t Rsp_Flag_Error = 0x8000;
const uint32_t Rsp_Flag_Warning = 0x4000;
const uint32_t Rsp_Flag_Update = 0x2000;
```

A.5 Constants: Device Response Parameter Types

```
const int32_t Rsp_Param_Type_Integer = 0;
const int32_t Rsp_Param_Type_Float = 1;
const int32_t Rsp_Param_Type_String = 2;
const int32_t Rsp_Param_Type_Byte_Array = 4;
const int32_t Rsp_Param_Type_Integer_Array = 254;
const int32_t Rsp_Param_Type_Float_Array = 255;
```

A.6 Constants: Connection Constants

```
const int32_t CHR_Unspecified = -1;
const int32_t CHR_1_Device = 0;
const int32_t CHR_2_Device = 1;
const int32_t CHR_Multi_Channel_Device = 2;
const int32_t CHR_Compact_Device = 3;
```

A.7 Constants: Connection Mode

```
const int32_t Connection_Synchronous = 0;
const int32_t Connection_Asynchronous = 1;
```

A.8 Constants: Library Data Formats

```
const int32_t Output_Data_Format_Double = 0;
const int32_t Output_Data_Format_Raw = 1;
```


© Precitec Optronik GmbH

A.9 Constants: Automatic Buffer Saving Constants

```
const int32_t Auto_Buffer_Error = -1;
const int32_t Auto_Buffer_Saving = 0;
const int32_t Auto_Buffer_Finished = 1;
const int32_t Auto_Buffer_Received_Response = 2;
const int32_t Auto_Buffer_Deactivated = 3;
const int32_t Auto_Buffer_UnInit = 4;
```

A.10 Constants: Return value of data read

```
const int32_t Read_Data_Not_Enough = 0;
const int32_t Read_Data_Success = 1;
const int32_t Read_Data_Response = 2;
const int32_t Read_Data_Buffer_Small = 3;
const int32_t Read_Data_User_Buf_Full = 4;
const int32_t Read_Data_Format_Change = 5;
```


© Precitec Optronik GmbH

A.11 Constants: Device Command Specifier ID Constants

```
const uint32_t CmdID_Output_Signals = 0x58444f53;
const uint32_t CmdID_Firmware_Version = 0x00524556;
const uint32_t CmdID_Measuring_Method = 0x00444d4d;
const uint32_t CmdID_Full_Scale = 0x00414353;
const uint32_t CmdID_Scan_Rate = 0x005a4853;
const uint32_t CmdID_Data_Average = 0x00445641;
const uint32_t CmdID_Spectrum_Average = 0x00535641;
const uint32_t CmdID_Serial_Data_Average = 0x53445641;
const uint32_t CmdID_Refractive_Indices = 0x00495253;
const uint32_t CmdID_Abbe_Numbers = 0x00454241;
const uint32_t CmdID_Refractive_Index_Tables = 0x00545253;
const uint32_t CmdID_Lamp_Intensity = 0x0049414c;
const uint32_t CmdID_Optical_Probe = 0x004e4553;
const uint32_t CmdID_Confocal_Detection_Threshold = 0x00524854;
const uint32_t CmdID_Peak_Separation_Min = 0x004D5350;
const uint32_t CmdID_Interferometric_Quality_Threshold = 0x00485451;
const uint32_t CmdID_Duty_Cycle = 0x00594344;
const uint32_t CmdID_Detection_Window_Active = 0x00414d4c;
const uint32_t CmdID_Detection_Window = 0x00445744;
const uint32_t CmdID_Number_Of_Peaks = 0x00504f4e;
const uint32_t CmdID_Peak_Ordering = 0x00444f50;
const uint32_t CmdID_Dark_Reference = 0x004b5244;
const uint32_t CmdID_Continuous_Dark_Reference = 0x4b445243;
const uint32_t CmdID_Start_Data_Stream = 0x00415453;
const uint32_t CmdID_Stop_Data_Stream = 0x004f5453;
const uint32_t CmdID_Light_Source_Auto_Adapt = 0x004c4141;
const uint32_t CmdID_CCD_Range = 0x00415243;
const uint32_t CmdID_Median = 0x5844454d;
const uint32_t CmdID_Analog_Output = 0x58414e41;
const uint32_t CmdID_Encoder_Counter = 0x53504525;
const uint32_t CmdID_Encoder_Counter_Source = 0x53434525;
const uint32_t CmdID_Encoder_Preload_Function = 0x46504525;
const uint32_t CmdID_Encoder_Trigger_Enabled = 0x45544525;
const uint32_t CmdID_Encoder_Trigger_Property = 0x50544525;
const uint32_t CmdID_Device_Trigger_Mode = 0x4d525425;
const uint32_t CmdID_Download_Spectrum = 0x444c4e44;
const uint32_t CmdID_Save_Settings = 0x00555353;
const uint32_t CmdID_Download_Upload_Table = 0x4C424154;
```

A.12 Constants: Spectra Types

```
const int32_t Spectrum_Raw = 0;
const int32_t Spectrum_Confocal = 1;
const int32_t Spectrum_FT = 2;
const int32_t Spectrum_2D_Image = 3;
const int32_t Spectrum_Capture = 4;
```


© Precitec Optronik GmbH

A.13 Constants: Table Types

```
const int32_t Table_Confocal_Calibration = 1;
const int32_t Table_WaveLength = 2;
const int32_t Table_Refractive_Index = 3;
const int32_t Table_Dark_Correction = 4;
const int32_t Table_Confocal_Calibration_Multi_Channel = 5;
const int32_t Table_CLS_Mask = 6;
const int32_t Table_MPS_Mask = 8;
const int32_t Table_CLS2_Confocal_Calibration = 18;
```

A.14 Constants: Device Trigger Modes

```
const int32_t Device_Trigger_Mode_Free_Run = 0;
const int32_t Device_Trigger_Mode_Wait_Trigger = 1;
const int32_t Device_Trigger_Mode_Trigger_Each = 2;
const int32_t Device_Trigger_Mode_Trigger_Window = 3;
```

A.15 Constants: Encoder Counter Input Source

```
const int32_t Encoder_Counter_Trigger_Source_A0 = 0;
const int32_t Encoder_Counter_Trigger_Source_B0 = 1;
const int32_t Encoder_Counter_Trigger_Source_A1 = 2;
const int32_t Encoder_Counter_Trigger_Source_B1 = 3;
const int32_t Encoder_Counter_Trigger_Source_A2 = 4;
const int32_t Encoder_Counter_Trigger_Source_B2 = 5;
const int32_t Encoder_Counter_Trigger_Source_A3 = 6;
const int32_t Encoder_Counter_Trigger_Source_B3 = 7;
const int32_t Encoder_Counter_Trigger_Source_A4 = 8;
const int32_t Encoder_Counter_Trigger_Source_B4 = 9;
const int32_t Encoder_Counter_Trigger_Source_SyncIn = 10;
const int32_t Encoder_Counter_Source_Quadrature = 15;
const int32_t Encode_Trigger_Source_Immediate = 15;
```

A.16 Constants: Encoder Preload Event Property Constants

```
const int32_t Encoder_Preload_Config_Once = 0;
const int32_t Encoder_Preload_Config_Eachtime = 1;
const int32_t Encoder_Preload_Config_Trigger_RisingEdge = 0;
const int32_t Encoder_Preload_Config_Trigger_FallingEdge = 2;
const int32_t Encoder_Preload_Config_Trigger_OnEdge = 0;
const int32_t Encoder_Preload_Config_Trigger_OnLevel = 4;
const int32_t Encoder_Preload_Config_Active = 8;
```

A.17 Constants: Library Configuration Flags

```
const uint32_t CHRLib_Flag_Rsp_Deactivate_Auto_Buffer = 1;
const uint32_t CHRLib_Flag_Auto_Change_Data_Buffer_Size = 2;
const uint32_t CHRLib_Flag_DataFormat_Deactivate_Auto_Buffer = 8;
```

A.18 Constants: Device Search Constants

```
const int32_t Search_Both_Serial_TCPIP_Connection = 0;
const int32_t Search_Only_Serial_Connection = 1;
const int32_t Search_Only_TCPIP_Connection = 2;
```


© Precitec Optronik GmbH

A.19 Constants: Performance save setting

```
const uint32_t OPID_Save_Performance = 0x5356504D;
const uint32_t OPID_Last_Error = 0x4C534552;
const uint32_t OPID_Device_Search = 0x44565345;
const uint32_t OPID_Device_Search_Result = 0x44565352;
const uint32_t OPID_Log_Level = 0x4C474C56;
const uint32_t OPID_Log_Dir = 0x4C474452;
const uint32_t OPID_Conf_Flag = 0x43464647;
const int32_t Performance_Snapshot_Mode = 0;
const int32_t Performance_Complete_Mode = 1;
```


© Precitec Optronik GmbH

Appendix B

The CHRocodileLib API grouped by topic

B.1 Asynchronous Mode

Functions, structures, constants and other entities related to using the asynchronous mode.

Also see

- Automatic Device Output Processing
- Callback Functions

Functions, structures and constants related to this

- ActivateAsyncDataUserBuffer()
- DeactivateAsyncDataUserBuffer()
- ExecCommandAsync()
- GetAsyncDataUserBufferSavedSampleCount()
- ProcessDeviceOutput()

B.2 Automatic Buffer Saving

Functions, structures, constants and other entities related to using the automatic buffer saving feature.

Also see

- Automatic Buffer Saving Constants

Functions, structures and constants related to this

- ActivateAutoBufferMode()
- DeactivateAutoBufferMode()
- GetAutoBufferSavedSampleCount()
- GetAutoBufferStatus()

© Precitec Optronik GmbH

B.3 Automatic Device Output Processing

Functions, structures and constants related to this

- StartAutomaticDeviceOutputProcessing()
- StopAutomaticDeviceOutputProcessing()

B.4 Automatic Device Search

Functions for automatic device search.
Also see

- Device Search Constants

Functions, structures and constants related to this

- CancelCHRDeviceAutoSearch()
- DetectedCHRDeviceInfo()
- IsCHRDeviceAutoSearchFinished()
- StartCHRDeviceAutoSearch()

B.5 Callback Functions

Everything related to callback functions.
Also see

- DataCallback Function
- Update and Response Callback Function

B.6 Command Response

Functions that relate to get the information from responses
Also see

- Device Response Parameter Types
- Response flag
- Update and Response Callback Function

Functions, structures and constants related to this

- GetResponseArgType()
- GetResponseBlobArg()
- GetResponseFloatArg()

© Precitec Optronik GmbH

- GetResponseFloatArrayArg()
- GetResponseInfo()
- GetResponseIntArg()
- GetResponseIntArrayArg()
- GetResponseStringArg()
- ResponseToString()
- **`TResponseInfo`**

B.7 Connection

Functionality related to establishing a connection, retrieving information about it and closing it.

Also see

- Connection Constants
- Connection Information
- Connection Mode

Functions, structures and constants related to this

- CloseConnection()
- OpenConnection()
- OpenSharedConnection()

B.8 Connection Information

Functions, structures and constants related to this

- GetConnectionMode()
- GetDeviceConnectionInfo()
- GetDeviceType()

B.9 DataCallback Function

Functions, structures and constants related to this

- RegisterSampleDataCallback
- SampleDataCallback

© Precitec Optronik GmbH

B.10 Device Command

Functions that map directly to a particular CHRocodile device command or other entities related to those.

Also see

- Device Command Specifier ID Constants
- Device Trigger Modes
- Spectra Types
- Table or Firmware Upload
- Table Types
- Update and Response Callback Function

Functions, structures and constants related to this

- AddCommandBlobArg()
- AddCommandFloatArg()
- AddCommandFloatArrayArg()
- AddCommandIntArg()
- AddCommandIntArrayArg()
- AddCommandStringArg()
- CmdNameToID()
- NewCommand()
- NewCommandFromString()

B.11 Device Information

Functions that to get the device information

Functions, structures and constants related to this

- GetDeviceChannelCount()
- GetDeviceOutputSignals()

B.12 Error Handling

Functions, structures, constants and other entities related to error information

Also see

- Result Code Constants

Functions, structures and constants related to this

- ClearErrors()
- ErrorCodeToString()
- LastErrors()
- Res_t
- **TErrorInfo**

© Precitec Optronik GmbH

B.13 Handles

Functions, structures, constants and other entities related to all kinds of handle

Also see

- Handle Types

Functions, structures and constants related to this

- DestroyHandle()
- GetHandleType()

B.14 Library Configuration

Functions to set CHRocodileLib configuration

Also see

- Library Configuration Flags
- Logging Configuration

Functions, structures and constants related to this

- SetLibConfigFlags()
- SetSampleDataBufferSize()

B.15 Plugins

Functions to configure plugins

Functions, structures and constants related to this

- AddConnectionPlugIn()
- AddConnectionPlugInByID()
- GetConnectionPlugInInfo()
- PluginTypeIDToName()
- **`TPluginInfo`**

© Precitec Optronik GmbH

B.16 Sample Data

Functions, structures, constants and other entities related to retrieve data.
Also see

- Automatic Buffer Saving
- DataCallback Function
- Library Data Formats
- Sample Data Types

Functions, structures and constants related to this

- GetConnectionOutputSignalInfos()
- GetCurrentAvailableSampleCount()
- GetOutputDataFormatMode()
- GetSingleOutputSampleSize()
- SetMultiChannelProfileInterpolation()
- SetOutputDataFormatMode()
- StartConnectionDataStream()
- StopConnectionDataStream()
- **`TSampleSignalGeneralInfo`**
- **`TSampleSignalInfo`**

B.17 Synchronous Mode

Functions, structures, constants and other entities related to using the synchronous mode.
Also see

- Automatic Buffer Saving
- Table or Firmware Upload

Functions, structures and constants related to this

- ExecCommand()
- FlushConnectionBuffer()
- GetLastSample()
- GetNextSamples()

B.18 Update and Response Callback Function

Functions, structures and constants related to this

- RegisterGeneralResponseAndUpdateCallback
- ResponseAndUpdateCallback
- **`TRspCallbackInfo`**

© Precitec Optronik GmbH

Appendix C

CHRocodileLib Internal Commands

Following commands are the internal commands of CHRocodileLib, user can pass these commands to the library through ExecCommand() or ExecCommandAsync(). The library will translate these commands to the corresponding device commands and forward to CHRocodile device. The response of the commands, which are delivered to the user, also have the form of the internal commands. The purpose of these internal commands is to simplify or unify the usage of some device commands. The lowest byte of the command ID of these interal commands is always “0x25”, the corresponding char is “%”. If the command string is passed over to NewCommandFromString(), please do not add “$” in front of the command name as for the device command.

C.1 Encoder Counter Value Command

Description Set/get the encoder counter value

Command ID CmdID_Encoder_Counter: 0x53504525

Command Name “%EPS”.

Command Format

Set command: %EPS <arg0> <arg1>

Query command: %EPS? <arg0>

Response: %EPS(?) <arg0> <arg1>

Parameter

int32_t arg0: Axis index;

int32_t arg1: Encoder counter position;

More Details This command is to set the encoder count of the CHRocodile device immediately. The supported axis index for CHRocodile 1 and CHRocodile C device is 0-2, other devices is 0-4.

C.2 Encoder Counter Source Command

Description Set/get the input source of the encoder counter

Command ID CmdID_Encoder_Counter_Source: 0x53434525

Command Name “%ECS”.

Command Format

Set command: %ECS <arg0> <arg1>

Query command: %ECS? <arg0>

Response: %ECS(?) <arg0> <arg1>

© Precitec Optronik GmbH

Parameter

int32_t arg0: Axis index;

int32_t arg1: Encoder counter source;

More Details This command sets the input source for the encoder counter of the given axis. The possible value of encoder counter source please check A.15.

C.3 Encoder Preload function Command

Description Set the property related to encoder counter preload function

Command ID CmdID_Encoder_Preload_Function: 0x46504525

Command Name “%EPF”.

Command Format

Command: %EPF <arg0> <arg1> <arg2> <arg3>

Response: %EPF <arg0> <arg1> <arg2> <arg3>

Parameter

int32_t arg0: Axis index;

int32_t arg1: Encoder counter position to be preloaded;

int32_t arg2: Property of preload function;

int32_t arg3: Trigger source of the preload function;

More Details Apart from immediately setting the value of encoder counter, user can also set the encoder counter to be changed/loaded upon a certain "preload" event. The supported axis index for CHRocodile 1 and CHRocodile C device is 0-2, other devices is 0-4. The property of the preload event can be combined with the value defined in A.16. The possible trigger source of the event please refer to A.15.

C.4 Encoder Trigger Enable Command

Description Set/get encoder triggle enable status

Command ID CmdID_Encoder_Trigger_Enabled = 0x45544525;

Command Name “%ETE”.

Command Format

Set command: %ETE <arg0>

Query command: %ETE?

Response: %ETE(?) <arg0>

Parameter

int32_t arg0: 1: Enable encoder trigger, 0: device is triggered by the synchronization signal;

More Details When CHRocodile device is in trigger mode (either “trigger each” or “wait for trigger” mode), it can be triggered either by the synchronization signal or by the encoder. This command is to (de)select encoder as the trigger source.

© Precitec Optronik GmbH

C.5 Encoder Trigger Property Command

Description Set/get the property related to encoder trigger

Command ID CmdID_Encoder_Trigger_Property: 0x50544525;

Command Name "%ETP".

Command Format

Set command: %ETP <arg0> <arg1> <arg2> <arg3> <arg4>

Query command: %ETP?

Response: %ETP(?) <arg0> <arg1> <arg2> <arg3> <arg4>

Parameter

int32_t arg0: Encoder axis index which is used for triggering;

int32_t arg1: Start position for encoder triggering;

int32_t arg2: Stop position for encoder triggering;

float arg3: Distance between adjacent triggering points in increments;

int32_t arg4: 1: triggering is active; 0: trigger is inactive during the return move from stop position to start pos

More Details If the encoder is selected to be the trigger source of the device, user can use this command to set the property of the encoder trigger. The supported axis index for CHRocodile 1 and CHRocodile C device is 0-2, other devices is 0-4. Note, the query command can only be executed for synchronous connection.

C.6 Device Trigger Mode Command

Description Set Chrocodile device trigger mode

Command ID CmdID_Device_Trigger_Mode: 0x4d525425;

Command Name "%TRM".

Command Format

Set command: %TRM <arg0>

Response: %TRM <arg0>

Parameter

int32_t arg0: triggering mode;

More Details CHRocodile device can be in four different trigger modes: "free-run", "trigger each", "wait for trigger" and "trigger window" mode. User can use this command to select the mode he needs. The exact meaning of each mode, please check the documentation of the device. The corresponding value of the modes is defined in A.14.

© Precitec Optronik GmbH

Part III

Programming Language Bindings

© Precitec Optronik GmbH

Introduction

Being part of CHRocodileLib distributions, wrappers (language bindings) help easily integrate the library with application programming languages and platforms. Currently supported programming languages are:

- C# (see chapter D)
- Python (see chapter E)

© Precitec Optronik GmbH

Appendix D

C#

D.1 Preface

This section describes the C# wrapper for the CHRocodileLib. As described in further detail below, it mainly reflects concepts implemented in the library itself (connections, commands, responses, data samples etc.) in a more C#-like manner.

While the translation of (shared) connections and commands to C# classes is straight-forward, the notion of Data objects containing all currently available data samples is worth further reading. Under the hood, the `Data` class makes use of CHRocodileLib's user buffers (see sections 7.16 and 7.17) and offers different ways of accessing and storing that data efficiently (see section D.5).

For the sake of brevity, all example code given in this chapter is basically taken from console demo projects but should be readily usable in GUI projects, too.

.NET / C# Compatibility

In order to provide broad compatibility while making use of some more recent C# features, the C# wrapper has been created based on C# version 7.3 which is supported by .NET framework (4.8), .NET core >= 2.2 and all .NET versions thereafter.

D.2 Architecture

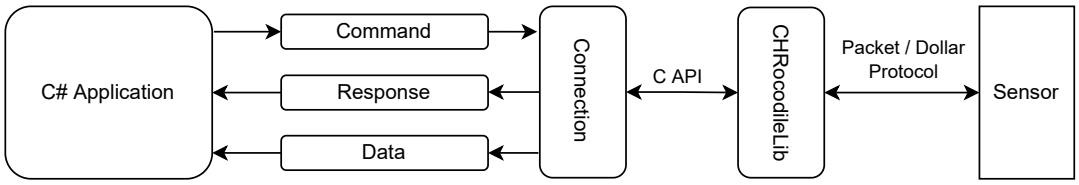


Figure D.1: The C# Wrapper Architecture

The C# wrapper basically exposes four different kinds of objects you need to know:

- Connection objects
- Command objects
- Response objects
- Data objects

© Precitec Optronik GmbH

D.2.1 Connection Objects

Any communication to a CHRocodile sensor begins with creating a connection object. Note that there are two different flavours of connections: Synchronous and asynchronous connections. With synchronous connections, executing a command will always block until the response from the sensor has arrived. With asynchronous connections, command execution functions will always return immediately while the corresponding response is later forwarded as callback function calls.

With synchronous connections, data is picked up via `GetNextSamples()` calls which never block and will only return data that is already available at the point of invocation.

With asynchronous connections, the user registers data callback functions which get called whenever a certain number of samples have arrived or, alternatively, a certain period of time has elapsed. Both are user-defined.

D.2.2 Command Objects

Commands such as “Set the scan rate to 5000Hz” are wrapped up in objects containing the command ID plus arguments. Since they are normally used in the moment they are created, commands have no C# representation other than a handle which is managed by CHRocodileLib:

```
Cmd_h command = Cmd.Command(CmdID.ScanRate, 1000);
```

Queries are created like this:

```
Cmd_h query = Cmd.Query(CmdID.ScanRate);
```

Sending the command is done with `Exec`. If any error occurs, exec will throw an exception of type `CHRocodileLib.Error`.

```
Response res = con.Exec(command);
```

Thanks to some `Exec` variants and overloads, creating and sending a command can be done in one go like:

```
con.Exec(CmdID.ScanRate, 1000);
```

In some cases, however, (specifically with asynchronous connections) creating and executing commands needs to be split up as will be seen later.

D.2.3 Response Objects

For every command the application will receive a response. Responses reflect their command counterpart and so consist of a command ID and a number of arguments, too. Response objects are returned from synchronous `Exec()` or `Query()` calls - or passed into command response callback routines. They offer properties and functions to query the ID, the state, and the arguments.

D.2.4 Updates

Updates are a special feature of asynchronous connections. They are just like responses - the difference is that they do not prompt commands but can occur more or less “spontaneously” - typically to notify about sensor state changes.

Some commands, however, cause both a response *and*, later on, consecutive updates which report the progress of an ongoing process triggered by the command.

© Precitec Optronik GmbH

D.2.5 Data Objects

Data objects are a little bit more complex and therefore deserve a separate section (see section D.5). For now the only thing to remember is that they contain zero, one or more samples of data which can be reached by either a get function (`get(sampleIndex, signalIndex)`) or a generator (`Samples()`). The result of `Samples()` can be used in a `foreach` loop. Each sample generated exposes in turn a get function to access the different signal values (`get(signalIndex)`). For multi-channel devices, an additional `channelIdx` argument denotes the channel addressed.

You cannot receive samples until you request them by a command. Example:

```
con.Exec(CmdID.OutputSignals, 83, 256);
```

The order of the arguments passed in (83, 256 here) defines the oder of the signal values obtained via `GetNextSamples()`: Inside a `foreach` loop, `get(0)` yields values of signal 83 (sample counter), `get(1)` returns signal 256 values (first distance or thickness).

With *multi-channel devices* there is one more thing to note:

Global signals must be first in the list of SODX signals, followed by peak signals. If they do not, then this will result in an error.

D.3 Basic Usage

D.3.1 Synchronous Mode

Taking a quick look at listing D.1, you can probably guess what that little program does: It opens a connection to a sensor (let's say single channel, confocal mode), queries the current scan rate, requests data for two signals 83 (sample counter), 256 (1st distance), and finally waits for 1000 samples which in turn get averaged.

Some interesting points are worth mentioning:

1. Connections implement the `IDisposable` interface and thus support `using` clauses. The connection is opened in the SynchronousConnection constructor and will be closed automatically when execution leaves the `using` block.
2. Alternatively (e. g. in GUI applications) you open a connection by creating a connection object. To close a connection, call `close()` on that object and de-reference it for clean-up.
3. Connection objects offer various alternative command execution functions:
   `Exec, ExecString, Query`.
   They all block until the command has been processed and then return a **response object** which in turn can be queried with regard to it's ID, flags, and arguments. In case the command has failed, a respective exception will be thrown which contains a human-readable error string.
4. Measurement data, i. e. specific signals of interest have to be requested by means of an `SODX` command (CmdID.OutputSignals).
5. The data itself is received by calls to `GetNextSamples`. Each of these calls returns a data object that contains sample data that has not yet been picked up by prior calls.
6. The number of samples obtained in one call can be limited by an optional argument.
7. `GetNextSamples()` does *not* block, i. e. wait for samples; it just picks up data that has already been received from the sensor.
8. The data object returned is of type `Data` and exposes data access functions such as `get(sample, signal, channel)`.

 © Precitec Optronik GmbH

Listing D.1: Synchronous mode example

```
...

static void Main(string[] args)
{
    const int total = 1000;
    try
    {
        using (var con = new SynchronousConnection("192.168.170.2", DeviceType.Chr2))
        {
            var response = con.Query(CmdID.ScanRate); // throws on error
            // response carries one parameter which is the queried value:
            Console.WriteLine($"The current scan rate is {response.Params[0]}Hz.");

            // request signals 83 (sample counter) and 256 (1st distance)
            // NOTE: In this example signal 83 is not being used.
            // access to signal values (see below): signal 83: Get(0), signal 256:
                Get(1)
            con.ExecString("SODX 83 256");
            double d = 0.0;
            long left = total;
            while (left > 0) {
                Data data = con.GetNextSamples(left); // retrieves at most "left"
                    samples
                foreach (var s in data.Samples())
                    d += s.Get(1); // sum up distance values
                left -= data.NumSamples;
                Thread.Sleep(100); // do something else
            }

            Console.WriteLine($"The average distance is {d / total}");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Failure: {ex.ToString()}");
    }
}
...
```

9. Such data objects also provide a `Samples()` function that serves as a **generator** for use in `foreach(...)` loops.

10. NOTE: Data objects as returned by `GetNextSamples()` are by default **only valid until the next call to** `GetNextSamples()` where it will be *overwritten* with new data. Thus, data returned by `GetNextSamples()` should immediately be processed by your application.

For basic usage scenarios there is not really much more that you need to know: Open a connection, send a few commands to configure the measurement, call `GetNextSamples()` repeatedly to read as many samples as you need.

D.3.2 Recording Data

Listing D.2 shows a variant of the simple program where the data is *not* immediately overritten. Instead, `con.StartRecording(1000);` creates a new, dedicated data object for exactly 1000 samples. `StopRecording()` leaves the *recording* state and returns the data object with the samples just obtained. Again, some points are worth mentioning:

1. In the recording state (`StartRecording(<sampleCount>) ... StopRecording()`), commands and queries are *forbidden* and will cause an exception.

2. You can call `GetNextSamples()` during recording any time - and the data will not be overwritten.
3. You can also call `StopRecording()` any time. If called early, the number of samples contained in the data object may be less than requested. If called late, recording may have stopped already because the `<sampleCount>` limit has been reached - which is probably what you want.
4. You can call `StartRecording()` right after `StopRecording()` (avoiding any `GetNextSamples()` inbetween) to make sure all data is contained in data objects without gaps.

Listing D.2: "Recording" data

```
con.ExecString("SODX 83 256");

// Recording session:
con.StartRecording(1000); // record 1000 samples
while (con.IsRecording)
    Thread.Sleep(100); // do something else
Data data = con.StopRecording(); // data now contains all recorded data

// Calculate average:
double d = 0.0;
foreach (var s in data.Samples())
    d += s.Get(1); // sum up distance values
Console.WriteLine($"The average distance is {d / data.NumSamples}");
```

D.4 Asynchronous Mode

D.4.1 Minimal Example

In asynchronous mode (see section 2.2), the API is a bit different. In listing D.3 the task of collecting 1000 samples is the same, this time implemented with an asynchronous connection.

The overall structure of the asynchronous variant is quite similar. A connection is opened, the current scan rate is queried, signals 83 and 256 are requested, and data is collected to finally determine the average distance. Again, a few points are noteworthy:

1. The connection type is different; now an `AsynchronousConnection` is instantiated.
2. At the beginning of a connection, you need to decide which way you want to have data and responses collected: Either by your application or by CHRocodileLib. If you want your application to take care of data and response processing then the application has to repeatedly call ProcessDeviceOutput(). If you decide for the latter, then activate the *automatic mode* (see StartAutomaticDeviceOutputProcessing()) which means: CHRocodileLib spawns a separate thread from which the user's callback functions are invoked. **NOTE:** Client code implementers then need to make any callback code *thread-safe* with respect to their own application's thread(s).
3. Data is now received in a callback function.
4. `Exec()` and `Query()` do not return a response anymore. Instead they immediately return.
5. The response can now be received in two ways:
   (a) By a callback routine passed into `Exec()` or `Query()`, see example D.3
   (b) By a general callback routine which has been registered (see RegisterGeneralResponseAndUpdateCallback())

6. C# wrapper functions are thread-safe (just as CHRocodileLib C API functions are), i. e. they can be called from any thread. In our example, `con.SetDataCallback()` is called from within the `con.ExecString()` response callback routine which is save to do.

D.4.2 A Super-Minimal Scanning Example

Listing D.4 contains a minimalistic code section to demonstrate the usage of data buffers. It involves the following simplified steps:

1. move the “axis” (just a mock axis, of course) to it’s starting point
2. set the sensor to “wait-for-trigger” state
3. prepare a data buffer (type `Data`) to receive all samples of one scan move
4. start the scan move
5. wait for the buffer to become full - and then *detach* it from the connection
6. finally process the data in that buffer

In this example, the “axis” is simply a mock object that exposes the member functions `MoveToSTart()` (which just prints a message) and `MoveToEnd()`. The latter will wait for some milliseconds, then send an `STR` command to the sensor which generates a trigger pulse to simulate a respective axis / encoder event. The demo code can be found in the CHRocodileLib package. Again, here are some important points to note:

1. Data is always stored in data buffers of type `XData`.
2. Asynchronous connections offer no way of blocking. The function `Execute()` returns immediately. Thus, the application cannot know whether a command has been completed or has succeeded at all until a respective response callback function has been invoked by CHRocodileLib.
3. On the other hand, in order to properly allocate buffer for the number of samples requested (see function `DetachRenewDataBuffer()`), the sample size must be known. Therefore, always make sure that `SODX` / `CmdID.OutputSignals` *is completed* before calling `DetachRenewDataBuffer()`. In the present example, this is ensured by putting the `DetachRenewDataBuffer()` call behind the `SynchronousCommandGroup`, see below.
4.

D.4.3 SynchronousCommandGroup

With asynchronous connections, it is sometimes necessary to wait for command responses before execution can proceed. The `SynchronousCommandGroup` is a utility class that serves this purpose. It can also help improving command throughput especially during initialization phases because all commands are sent asynchronously without waiting. The final waitAll() blocks and collects all responses, see listing D.5.

NOTE: Do *not* use SynchronousCommandGroups inside response or data callbacks. This would block the thread which is supposed to deliver the responses being waited for.

 © Precitec Optronik GmbH

D.4.4 Executing Commands that generate Updates

Some commands (most notably the Flying Spot Scanner (FSS) Plugin's `Run()` command) initiate processes that can take much longer than a simple sensor parameter setting. In the course of such a process, data may be produced that is directly related to the initial command but cannot be captured in one single response. For example, a complete FSS scan can include many separate scan figures - with data produced for each.

With asynchronous connections, applications call `CmdID_Flying_Spot_Exec` once on the FSS plugin and then receive one response plus one update for each scan shape. For this to work, the following points must be observed:

- The delegate holding the reference to the command callback must remain valid throughout the whole scan process. Therefore...
- Keep the delegate as a member variable
- Call the `ExecWithUserResponseDelegate` variant of `Exec`, **not** Exec. Example:

```
Plugin.ExecWithUserResponseDelegate(
    Cmd.Command((CmdID)CHRLibPlugin.CmdID_Flying_Spot_Exec,
        programHandle), ScanProgramCallback);
```

  This way, the C# wrapper assumes an externally referenced delegate.

- If, on the other hand, Exec() is used, then the C# wrapper keeps a GCHandle to the delegate and releases it after the response callback has been called. The advantage is that Exec() can be directly called with any method or lambda without worrying about delegate lifetime. The disadvantage is that updates cannot be handled after the delegate has been released.
- **So long story short:**
  - You normally use Exec() - with a delegate, a lambda, a method reference.
  - ONLY in case you expect directly command-related updates:
    - declare an `AsynchronousConnection.ResponseAndUpdateCallback` delegate field / property
    - assign your application's designated callback method to it
    - pass this delegate to `ExecWithUserResponseDelegate()`.

© Precitec Optronik GmbH

Listing D.3: Asynchronous mode example

```
static void Main(string[] args)
{
try
{
    bool mustInitShadowCounter = true;
    int counter = 0;
    double distance = 0.0;
    using (var con = new AsynchronousConnection("192.168.170.2",
        DeviceType.MultiChannel))
    {
        con.AutomaticMode = true; // samples and command responses will be pushed by
            CHRocodileLib
        // Command is ASYNC: A lambda routine is called on command completion:
        con.Query("SHZ", rsp => Console.WriteLine($"Response:
            SHZ={rsp.GetParam<float>(0)}"));

        const int total = 1000;
        int count = total;
        double d = 0.0;
        double dist = 0.0f;

        // request some signals, then register data receiver:
        con.ExecString("SODX 83 256", rsp => {
            Console.WriteLine("SODX done.");
        });

        Thread.Sleep(500);
        // this function is called on SODX response arrival:
        con.SetDataCallback((data) => {
            // this is the data receiver routine - running in a separate thread
            foreach (var s in data.Samples()) {
                if (--count < 0) {
                    Volatile.Write(ref dist, d / total);
                    break;
                }

                d += s.Get(1);
            }
        },
        100, 100);

        while (Volatile.Read(ref dist) == 0.0f)
            Thread.Sleep(100);

        Console.WriteLine($"The average distance is {Volatile.Read(ref dist)}");
    }
}
catch (Exception ex)
{
    Console.WriteLine($"Failure: {ex.ToString()}");
}
}
```


© Precitec Optronik GmbH

Listing D.4: Minimal scanning example

```
static void Main(string[] args)
{
    using (var con = new AsynchronousConnection("192.168.170.2", DeviceType.Chr2))
    using (var axis = new Axis() { Con = con }) {
        try {
            const int scanSampleCount = 1000;
            ConcurrentQueue<Data> myData = new ConcurrentQueue<Data>();
            con.SetDataCallback((status, data) => {
                Console.WriteLine("Acquiring data...");
                switch (data.Status) {
                    case DataStatus.Error: state = State.Error; break;
                    case DataStatus.Stopped: {
                        //if scan is running, buffer is full, means enough samples
                            have been collected
                        if (state == State.Running) {
                            con.Exec(CmdID.StopDataStream, null);
                            //keep the data object
                            myData.Enqueue(data.DetachRenew());
                            state = State.FinishedMove;
                        }
                        break;
                    }
                }
            }, 100, 100);

            con.AutomaticMode = true; // let CHRocodileLib push

            axis.MoveToStart(); // ``move'' axis - just a message
            var exec = new CHRocodileLib.SynchronousCommandGroup(con);
            exec.Add(Cmd.Command(CmdID.ScanRate, 1000));
            exec.Add(Cmd.Command(CmdID.DataAverage, 5));
            exec.Add(Cmd.Command(CmdID.OutputSignals, 256));
            // stop data, wait:
            exec.Add(Cmd.Command(CmdID.DeviceTriggerMode, TriggerMode.WaitTrigger));
            var responses = exec.WaitAll(); // wait for all commands to complete
            while (responses.TryDequeue(out var response))
                if (response.IsError()) // check all responses
                    throw new Error($"Command {response} has failed.");

            // remove old samples, begin new buffer for 1000 samples:
            con.DetachRenewDataBuffer(scanSampleCount);
            state = State.Running;
            axis.MoveToEnd();

            while (state == State.Running)
                Thread.Sleep(10); // wait for all data

            if (!myData.TryDequeue(out Data outData))
                throw new Exception("no data");
            //finish scanning
            double d = 0.0;
            outData.Rewind(); //read data from beginning
            foreach (var s in outData.Samples())
                d += s.Get(0);
            Console.WriteLine($"Average distance = {d / outData.TotalNumSamples} -
                press return.");
        }
        catch (Exception e) { Console.WriteLine($"Error occurred: {e.Message}"); }
    }
}
```

Listing D.5: Using a SynchronousCommandGroup

```
1
2     var exec = new CHRocodileLib.SynchronousCommandGroup(con);
3     exec.Add(Cmd.Command(CmdID.ScanRate, 1000));
4     exec.Add(Cmd.Command(CmdID.DataAverage, 5));
5     exec.Add(Cmd.Command(CmdID.OutputSignals, 256));
6     exec.Add(Cmd.Command(CmdID.DeviceTriggerMode, TriggerMode.WaitTrigger));
7     var responses = exec.WaitAll(); // wait for all commands to complete
8
9     while (responses.TryDequeue(out var response))
10    {
11        if (response.IsError())
12            // ... error handling ...
13    }
```


© Precitec Optronik GmbH

D.5 Data Objects

In CHRocodileLib's C# wrapper, data is always stored inside data buffers of type `Data`. These are wrappers around the `CHRocodileLib`'s AutoBuffer functionality: Behind the scenes, the C# wrapper allocates a (pinned) byte array and lets the library fill that memory block with measurement data asynchronously. This way, copy operations are limited, and the user has control over the buffer size.

These data buffers can be used in different ways. The standard way is emulating the library's `GetNextSamples()` function as outlined in figure D.2: Repeated calls to `GetNextSamples()` will give access to "new" samples, i. e. sample data that has not yet been delivered or read so far.

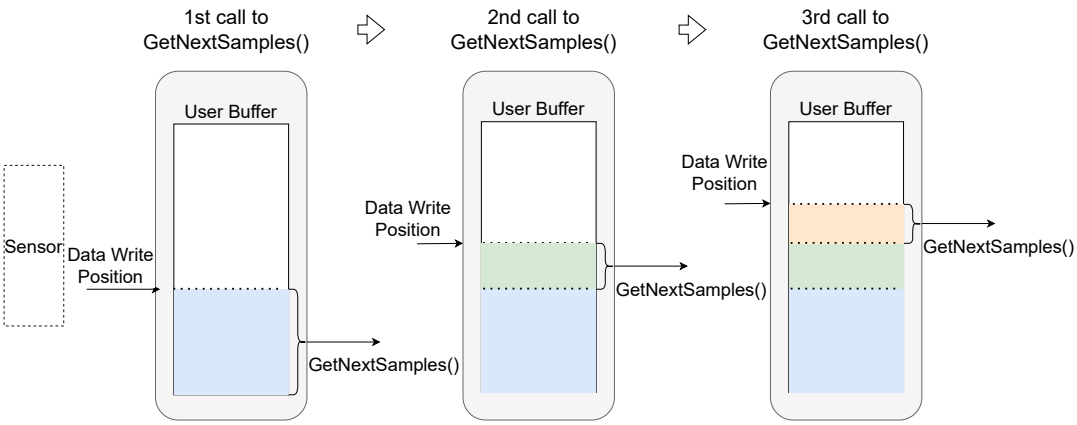


Figure D.2: The C# Data Buffers

Once the buffer is full and read out completely, it will be reset the next time `GetNextSamples()` is called. This means in particular that in general data returned from `GetNextSamples()` is valid only until the

NOTE: This means that if a `foreach` loop on the result of the last `GetNextSamples()` call is still running, the data it runs on will be *corrupted*.

Thus, if the application runs GetNextSamples() and foreach() in different threads, the state of the data buffer must be observed. If filling has stopped (because the buffer is full or the data format has changed) it is important to stop any asynchronous access to it's data before the next call to GetNextSamples().

D.5.1 Detaching Data Buffers

If you just go with `GetNextSamples()` or your data callback routine you do not need to know anything about data buffers. However, you can let C# wrapper fill the data buffers and then *take ownership* of them while the library creates a new data buffer to be subsequently filled. This is done within the data callback routine by `calling data.DetachRenew(sampleCount)` or (with synchronous connections) by calling `connection.DetachRenewDataBuffer(sample)`.

Detached data buffers can be safely used in other threads. The API for accessing data remains the same, of course. The process of detaching data buffers is illustrated in figures D.3 and D.4.

The important points to note are:

- The CHRocodileLib C# wrapper always internally employs data buffers to receive sample data - even if the user just uses `GetNextSamples()` or the asynchronous data callback functions.

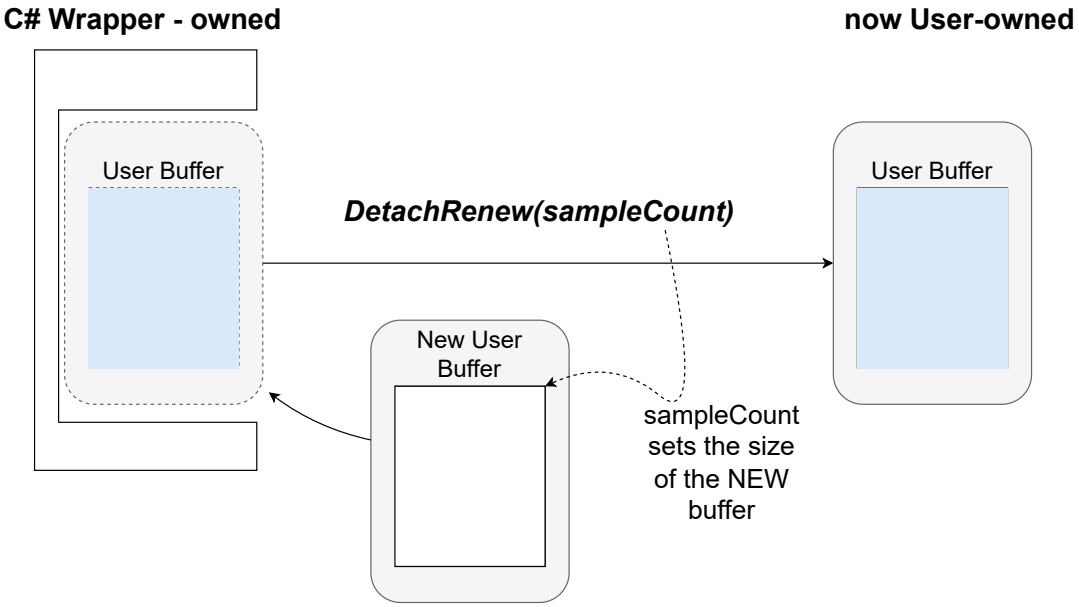


Figure D.3: The C# Data Buffers

- – in synchronous mode, the data obtained from a `GetNextSamples()` call is valid until the *next* call to GetNextSamples()
  - – in asynchronous mode, the data passed into the data callback function is valid only during execution of that function.
- However, your application can take ownership of the current user buffer, minimizing the number of data copy operations:
  - – in synchronous mode, use `Start / StopRecording()`
  - – in asynchornous mode, call `data.DetachRenew()`
- Once detached, data buffers can be safely used in any thread.

© Precitec Optronik GmbH

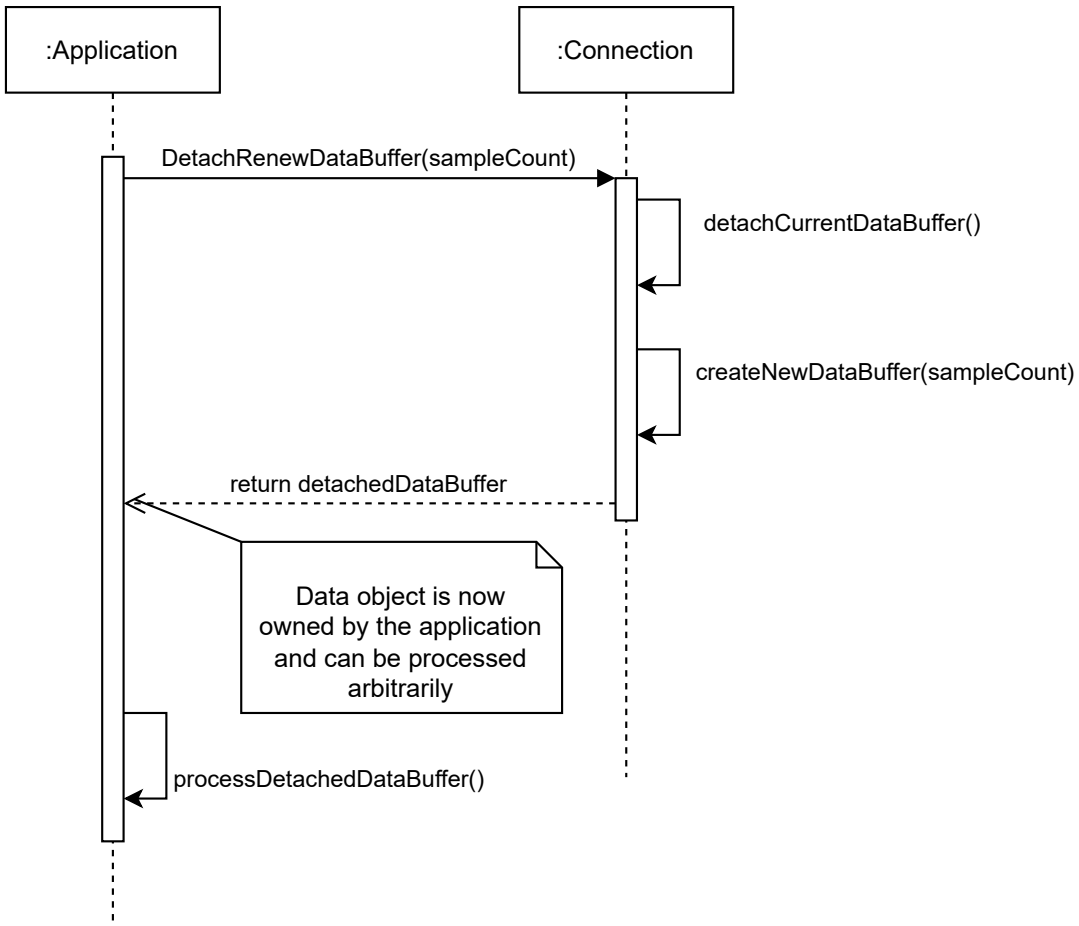


Figure D.4: The C# Data Buffers

© Precitec Optronik GmbH

Appendix E

Python

E.1 Introduction

This documentation refers to the Python wrapper/bindings for the Chrocodile C++ client DLL version 5.x. The Python wrapper is written using the `ctypes` Python module. For details regarding the C++ DLL functionality, the reader is referred to the C++ DLL documentation.

The motivation for this package is to provide the users a way to deploy the C++ client DLL in a Python environment for rapid development, prototyping, analysis and testing. An effort has been made to provide good performance by minimizing copying of data and the use of numpy. However, for high performance applications, it may be better to use the C++ API directly. If in doubt, one can quickly find out if the usage of the Python wrapper meets the performance expectations or not.

The supplied examples are a good source to get familiar with the basic usage of the the Python bindings. A brief description of the usage is given in a later section.

E.2 Requirements

- Python 3.7 and greater.
- numpy (if the wheel package is installed, numpy will be installed automatically)

E.3 Basic usage

The user creates a connection object and sends commands and receives commands responses and data. There are factory functions available for creating connection objects. The Python bindings support two operation modes - synchronous and asynchronous. In synchronous mode, the command execution calls are blocking and the data is explicit fetched. On the contrary in asynchronous mode, callbacks are specified for command responses and data. These callbacks are then called by the library when a command response or data is available.

A synchronous connection one can be created as follows (not all arguments are shown):

```
#
# Connections support the python context manager protocol
# and used in a "with" clause, the connection is opened
# and closed automatically.

with connection_from_params(addr='192.168.170.3',
    device_type=DeviceType.CHR_MULTI_CHANNEL,) as conn:
#
```

A connection in asynchronous operation mode can be created as follows (not all arguments are shown):

© Precitec Optronik GmbH

```
#
# Connections support the python context manager protocol
# and used in a "with" clause, the connection is opened
# and closed automatically.
# The responses and data are received
# in the supplied callback functions.
# The callbacks can be free functions or class methods.

with connection_from_params(addr='192.168.170.3',
    conn_mode=OperationMode.ASYNC,
    device_type=DeviceType.CHR_2,
    resp_callback=gen_callback,
    data_callback=data_callback,
    dll_path=path_to_lib) as conn:
#
```

After a connection to the sensor device has been opened, one can send commands and queries like:

```
#
response = conn.exec('SODX', 83, 16640, 16641)

response = conn.query("FLTC")

response = conn.exec_from_string("SODX 83 16640 16641")
#
```

In case of synchronous connections, the returned **Response** object is completely filled. But in case of asynchronous connections a partial response is delivered with a ticket ID and the full response is received in the specified callback.

For synchronous connections there are methods available to fetch data. For example,

```
#
sample_data = conn.get_next_samples(10, False)
#
```

The returned **Data** object contains metadata and sample data.

For asynchronous connections the responses and data are received in the specified response and data callback functions. For example,

```
# Response callback
def resp_callback(resp: Response):
    print(resp)

# Data callback
def data_callback(data: Data):
    print(data)
#
```

Please note that in the case of asynchronous operation mode, it is also possible to specify a callback for a particular command execution. For example,

```
#
def my_sodx_callback(cb: Response):
    print("Special SODX callback\n", cb)

# Specify my_sodx_callback function as the response callback for the following
↪  command execution.
conn.exec('SODX', 83, 16640, 16641, resp_cb=my_sodx_callback)
#
```

Example Python scripts are supplied demonstrating the implementation of many common use cases.

© Precitec Optronik GmbH

E.4 Command Response and Data objects

Command responses are delivered as objects of type **Response**. In addition to metadata it has a member **args** which is a list of response arguments.

The data is delivered as objects of type **Data**. In addition to metadata it has a member **samples** which is a numpy ndarray containing the data for one or more samples. The **Data** class provides utility functions for accessing signals values.

E.5 Data Format

The received sample data contains global signals followed by channel signals. For example,

Global Signal 1
Global Signal 2
Channel 1
  PeakSignal 1
  PeakSignal 2
Channel 2
  PeakSignal 1
  PeakSignal 2
...
...
Channel N
  PeakSignal 1
  PeakSignal 2

By default this data is delivered as float64 values even if the underlying type of the received signal requirer fewer bytes such as an uint16. If one were to visualize the ***samples*** member of type ndarray of the **Data** object with a print statement one would see something like (in this case one global signal 83 and tw peak signals 16640 and 16641 were requested):

```
sample_cnt=2
gen_signal_info=channel_cnt=1200,global_sig_cnt=1,info_index=2,peak_sig_cnt=2
signal_info=[(255, 83), (255, 16640), (255, 16641)]
error_code=0
samples=[[15600.  5027.   402. ...   313.  6852.   311.]
 [15601.  5035.   405. ...   312.  6859.   310.]]
```

However, the Python wrapper can be configured to work with raw sensor data. For example,

```
#
conn.set_output_data_format_mode(OutputDataMode.RAW)
#
```

This maybe desirable when one wants to reduce the memory usage. In this case the raw data buffer is used in conjunction with a structured array to present data as global signals followed by tuples of channel signals. The type of the elements is same as the underlying type delivered by the sensor. So one can have signals with different data types next to each other. If one were to visualize the ***samples*** member of type ndarray of the **Data** object with a print statement one would see something like (here two global signals 83 and 75, and, two peak signals 16640 and 16641 were requested)::

```
sample_cnt=1
gen_signal_info=channel_cnt=1200,global_sig_cnt=2,info_index=3,peak_sig_cnt=2
signal_info=[(2, 83), (2, 76), (3, 16640), (3, 16641)]
error_code=0
samples=[(48000, 1264, [(5032, 409), (5034, 403), (5030, 420), (5032, 427), ..., (8935,
```

Please note that the **Data** class provides utility functions for obtaining signals values for a particular signal ID. These normally provide a view of the underlying data without making copies.

```
#
signal_data = data.get_signal_values(sig_id, sample_no)

signal_data = data.get_signal_values_all(sig_id)
#
```

E.6 Error handling

Usually the Python wrapper connection methods throw an exception if a C++ function call results in an error code being returned. The exception contains the error code and the string representation of the error code. However, when operating in asynchronous mode, the user is expected to evaluate the error codes and react accordingly in the callbacks. ***Please handle all the exceptions in the asynchronous callbacks, as currently an unhandled exception will result in an abort signal.***

E.7 Auto buffer mode

Normally the memory for the data buffers that are forwarded to the client is managed internally by the C++ client DLL. However, the C++ client DLL supports auto buffer mode in which a client can provide a user allocated buffer to the C++ client DLL for receiving data. The Python wrapper supports this functionality by providing a numpy buffer to the C++ client DLL for receiving data. The user just has to specify how many maximum number of data samples should be saved in the auto buffer. The rest is handled by the Python wrapper. This functionality is available in both synchronous and asynchronous operations modes.

In asynchronous mode the auto buffer functionality is used by default but can be turned off. From a user point of view there is no difference in the data callback. Please note that if the application is slow in processing the data, there will be data loss. The DLL generates special logs for this and the error code in the Data object can be checked for warning and error. For example,

```
#
def data_callback(cb: Data):
    print(cb)

    if chr_warning(cb.error_code):
        print("Lib auto flushed buffer")
        # Just for debugging
        signal.raise_signal(signal.SIGABRT)
    elif chr_error(cb.error_code):
        print("Lib error in processing device output")
        # Just for debugging
        signal.raise_signal(signal.SIGABRT)

    if cb.error_code == ReadData.BUFFER_FULL:
        print("In case the config async_auto_activate_buffer flag is set,"
              " after this callback the auto buffer will be internally reset,"
              " copy data if needed later."
              " If not, please make sure that the after the data has been"
              " worked on the auto buffer is reactivated with the connection"
              " reset_async_auto_buffer method to continue receiving data.")
#
```

In synchronous mode one has to explicitly enable the auto buffer functionality. Also, the user can wait for the buffer to be filled before processing the data, or alternatively fetch the newly added samples to the buffer and process these while the DLL is receiving the remaining samples. The latter approach is more efficient as a client DLL thread is filling the buffer and the

Python interpreter thread is processing the already received data. The following information is specific to synchronous mode.

If one want to collect all the samples, before processing these, one could use the provided utility method as show in the code example that follows.

```
#
# In this case 10.0 is the timeout for collecting data. Please look at
# the documentation of this method for other arguments. With flush buffer
# one can control if one wants to discard older data or not.
# Default is False, to keep reading from where one left off.
max_samples_to_request = 75000
data = conn.wait_get_auto_buffer_samples(max_samples_to_request, 10.0,
↪  flush_buffer=False)
#
```

In case one would rather process the samples as they are coming in, one could implement it as shown in the sample code that follows.

```
#
_, err = conn.activate_auto_buffer_mode(max_samples_to_request, flush_buffer=True)
last_sample_cnt = 0
max_samples_to_request = 75000
while last_sample_cnt < max_samples_to_request:
    data = conn.get_auto_buffer_new_samples(last_sample_cnt)
    if data is None:
        time.sleep(0.1)
        continue
    last_sample_cnt += data.sample_cnt
#
```

E.8 Plugins

The plugin functionality available in the C++ client DLL can be used with the Python wrapper. A plugin can be added as shown in the the sample code that follows.

```
#
# Add a plugin to a connection with plugin ID.
plugin = conn.add_plugin_with_id(Test_Plugin_ID)
# Add a plugin to a connection with plugin name.
plugin = conn.add_plugin(FlyingSpot_Plugin_Name)
#
```

These call written a Plugin object and the operation mode for the command execution depends is the same as the that for the connection to which the plugin was added. The commands for a plugin can be carried out as is possible for a connection. For example in asynchronous operation mode,

```
#
# resp_cb specifies the callback for this particular execution of the command.
plugin.exec('EXEC', exec_arg, resp_cb=response_callback)
#
```

E.9 Shared connections

These are supported as well. A shared connection can be opened from an existing connection. For example,

```
#
# Open a shared connection in asynchronous operation mode from
# an existing connection. # The operation mode of a shared
# connection can be different from the connection with
# which it is shared.
shared_conn = conn.open_shared_connection(conn_mode=OperationMode.ASYNC,
                                          resp_callback=shared_resp_callback,
                                          data_callback=shared_data_callback)
#
```


© Precitec Optronik GmbH

E.10 Performance Tips

It is recommended to avoid making unnecessary copies of the data and as far as possible operate on numpy arrays instead of converting these to, for example, Python lists. It is also advisable, wherever possible, to operate on large chunks of data or multiple samples simultaneously.

E.11 Good To Know

By default, the chrpy package loads the CHRocodile .dll/.so library from the installation directory of the package and the ini file (CHRocodile.ini) from the current working directory.

One can override this behavior by specifying these paths while creating the connection object. Please note that the name of the file is part of the path. For example,

```
#
# The names of the DLL and ini file are part of the specification.
path_to_lib = "C:/lib/Chrocodile.dll"
path_to_ini = "C:/config/CHRocodile.ini"

with connection_from_params(addr=addr, device_type=DeviceType.CHR_2,
↪  dll_path=path_to_lib, ini_file=path_to_ini) as conn:
#
```

It is also possible to specify these paths explicitly. **NORMALLY IT IS NOT REQUIRED TO MAKE AN EXPLICIT CALL TO *"load_client_dll"*. HOWEVER, WHEN THIS IS DONE, PLEASE PASS THE LIBRARY HANDLE OBTAINED BY THE *"load_client_dll"* CALL TO THE CONNECTION CONSTRUCTOR OR CREATOR.**

For example,

```
#
# The names of the DLL and ini file are part of the specification.
path_to_lib = "C:/lib/Chrocodile.dll"
path_to_ini = "C:/config/CHRocodile.ini"

path_to_lib, dll_handle = load_client_dll(path_to_lib)

error_code = set_ini_file(dll_handle, path_to_ini)

with connection_from_params(addr=addr, device_type=DeviceType.CHR_2,
↪  dll_path=path_to_lib, dll_h=dll_handle) as conn:
#
```

Please note that for performance reasons the Python wrapper avoids making copies of received data as far as possible and delivers numpy arrays which use the supplied memory without copying, or in case of slicing to obtain signal values, views of the underlying data. Therefore, if one wants to keep data for later processing or the data processing step after a synchronous data fetch or in an asynchronous data callback is time consuming relative to the data rates, one will have to make a copy of the data. It is recommended that before making a copy, one may want to check if the obtained samples or signals ndarray is already a copy or not. There are other strategies one can follow for processing data depending on the use case. For example, if the sensor measurements are made with triggering and the start of the next triggering scan is flexible, one could use a larger library buffer so that a complete scan fits in the library buffer. Then the next scan can be triggered after the current scan buffered data has been processed.

E.12 Troubleshooting

- Please read the section **Good To Know**.

- If when running the supplied examples after unzipping the archive, a module not found error occurs, one can add the chrpy and examples folders to the PYTHONPATH.
- If the client application has problems adding plugins, one may have to specify the correct working directory and edit the PluginDir entry in the Chrocodile.ini.

E.13 Support

Please contact service@precitec-optronik.de.

© Precitec Optronik GmbH

Part IV

Plug-ins

© Precitec Optronik GmbH

Appendix F

CLS2 Calibration Plugin

CLS2 calibration plugin supports following commands:

F.1 Calibration Correction Active Command

Description Activate calibration correction

Command ID CmdID_Calib_Active: 0x414C4143

Command Name “CALA”.

Command Format

Set command: CALA <arg0>
Query command: CALA?
Response: CALA(?) <arg0>

Parameter

int32_t arg0: whether correction is active: 0: inactive; else: active;

More Details User can use this command to switch on/off the correction. By default, the correction is activated.

F.1.1 Calibration Correction Mode Command

Description Set/get calibration correction mode

Command ID CmdID_Calib_Mode: 0x444D4143

Command Name “CAMD”.

Command Format

Set command: CAMD <arg0>
Query command: CAMD?
Response: CAMD(?) <arg0>

Parameter

int32_t arg0: calibration mode:
0: use external file to calibrate;
1: use the calibration file inside the CLS2 device;

More Details User can use this command to set the correction mode. By default, the external file is used. When the device internal calibration file is selected, upon connecting to the device and the changing of optical sensor, calibration file will be automatically downloaded and resampled. The use of calibration file will be then disabled in the device by sending command “SEN -1”.

© Precitec Optronik GmbH

F.1.2 Calibration File Name Command

Description Set/get the used calibration file name

Command ID CmdID_CalibFile_Name: 0x464C4143

Command Name “CALF”.

Command Format

Set command: CALF <arg0>
Query command: CALF?
Response: CALF(?) <arg0>

Parameter

ascii string arg0: Calibration file name;

More Details This command is to set or query used external calibration file name. By setting the file, an ascii string parameter of the file name (including path) is required. The file path can be relative path. Only one file can be set at one time. Without setting the calibration file for current optical sensor, no correction is performed at peak distance data.

F.1.3 Calibration File List Command

Description Set/get the used calibration file list

Command ID CmdID_CalibFile_List: 0x4C464143

Command Name “CAFL”.

Command Format

Set command: CAFL <arg0> <arg1> <arg2> <arg3>...
Query command: CAFL?
Response: CAFL(?) <arg0> <arg1> <arg2> <arg3>...

Parameter

int32_t arg0: the first optical sensor index;
ascii string arg1: Calibration file name for the first optical sensor;
int32_t arg2: the second optical sensor index;
ascii string arg3: Calibration file name for the second optical sensor;

More Details Through this command, the list of used external calibration files can be set or queried. Inside parameters, the optical sensor index always comes in front of corresponding external calibration file name. When used optical sensor is changed, the external calibration file in the list will be automatically read in and applied.

F.1.4 Resample Count Command

Description Set/get resample count of device internal calibration table

Command ID CmdID_Calib_Resample_Count: 0x43524143

Command Name “CARC”.

© Precitec Optronik GmbH

Command Format

Set command: CARC <arg0>

Query command: CARC?

Response: CARC(?) <arg0>

Parameter

int32_t arg0: resample count

More Details After internal calibration table is downloaded from the CLS2 device, the calibration data will be first resampled. Resample count is simply the count of resampled data.

© Precitec Optronik GmbH

Appendix G

Flying Spot Scanner Plugin

G.1 Overview

G.1.1 Flying Spot Scanner Operation

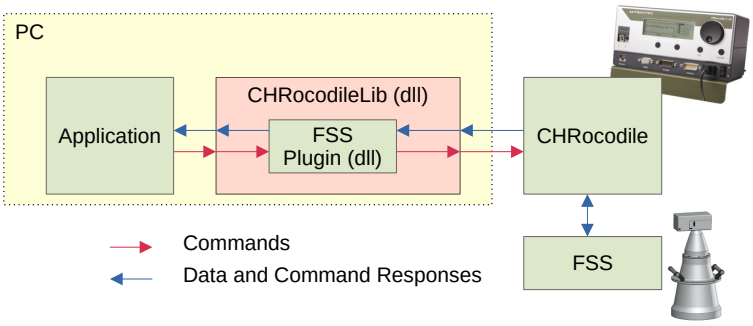


Figure G.1: Block diagram of the FSS operation

The FSS scanner and the sensor operate in close coordination. The sensor, while processing interferometric measurement data, is in charge of feeding new set positions into the scanner driver - based upon scanner byte code which has been created by means of a script compiler and uploaded upon start-up of the measurement cycle.

The block diagram shown in figure G.1 includes:

- The user application which typically features machine integration, visualization, further measurement data treatment etc...
- The CHRocodileLib, a shared library / DLL which takes care of managing sensor connections including translating and transmitting sensor commands and receiving / buffering sensor data
- The Flying Spot Scanner plugin, a shared library loaded into the CHRocodileLib and taking part in command processing and data post-treatment.
- The sensor (controller) which covers the whole measurement process
- The flying spot scanner controller / driver which is controlled by the sensor content

From a user's perspective, running an FSS application typically involves the following steps as presented in figure G.2.

© Precitec Optronik GmbH

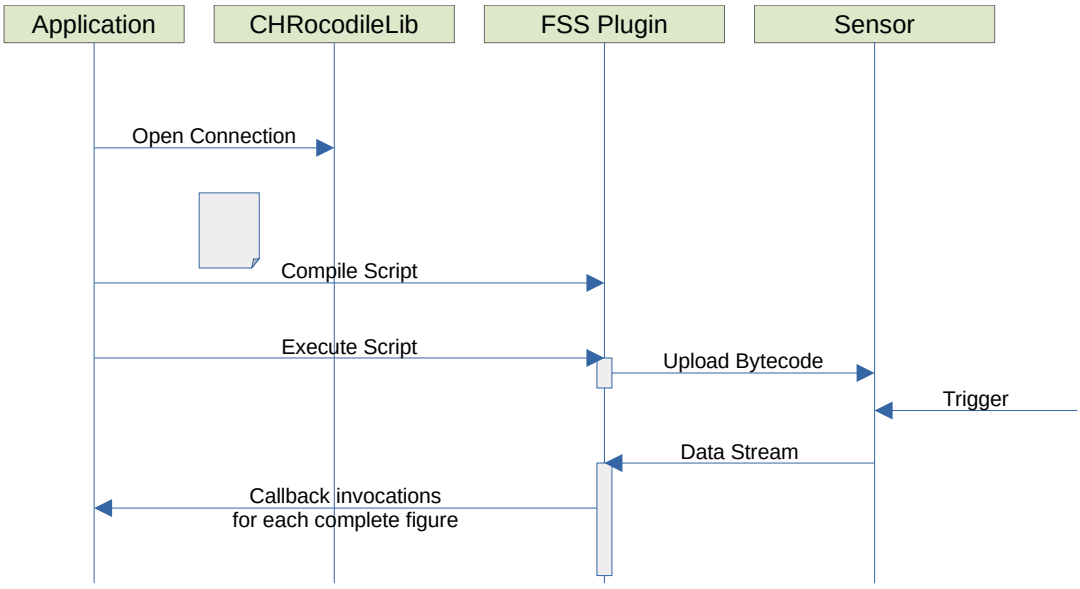


Figure G.2: Sequence diagram of the FSS operation

- Create a scanner script
- Compile this script
- Execute it (upload it to the sensor)
- Let the CHRocodileLib (and the FSS plugin) deliver data for each of the scan figures defined in the script.

For each data sample, the current scanner position is latched and attached to the thickness or distance values just measured.

G.2 The FSS Scripting Language

Before diving into details, a simple script shall be given as an example:

```
init
{
    $SHZ 10000; // set _sensor_ frequency
    $SODX 65 66 67 68 69 256 83; // set sensor output signals: encoder, peak 1
    ↪  position, sample counter
}

fn myFunc()
{
    shape(label = "horz_line") {
        // the origin within this function (0, 0) is equal to
        // the arguments ofsX, ofsY passed in (see main function):
        moveTo(0, 0)
        waitUsec(10000) // give the scanner some time to settle
        startMeasure() // now activate sensor
        lineTo(80, 0, 1000) // move the scanner to pos (0, 10), acquiring 1000
↪   points
    }
}

fn main(scanFreq=10000) // set _scanner_ frequency to 10kHz
{
    for i in range(-40, 40, 4) {
        myFunc(ofsX=-40, ofsY=i)
    }
}
```


© Precitec Optronik GmbH

The script above first sends some initial commands to the sensor (in “dollar protocol” format: `$<command> <arg1> <arg2> ... ;`), setting the sensor frequency (NOT the scanner frequency) to 10,000 samples / sec and the signals to obtain from the scan. It then moves on to the the `main` function which in turn repeatedly calls a user-defined function `myFunc` that scans a horizontal line. For each user function call, the starting point (the function’s local origin) is moved down in y direction, resulting in a rectangular grid.

In real code, you will not have to write user functions for line scans since there is a number of built-in shape functions available (see Section G.4).

G.2.1 Language design

The FSS scripting language has a number of similarities to the Rust programming language. Rust’s syntax is largerly similar to C++ (i.e., curly braces, comments style, etc.). Yet, some language constructs are simplified. For instance, function definitions do not have a return type, and recipe script functions cannot return a value. Besides, there is a special loop construct which might be conveniently adopted to define infinite loops in recipe scripts. One extension beyond the Rust’s syntax is a limited support for positional and keyword arguments for built-in function calls. Finally, any text editor supporing the syntax highlighting for Rust can be conveniently used to edit recipe scripts, thus facilitating development.

G.2.2 Case sensitivity

The recipe scripts syntax is case-sensitive (i.e., the identifiers `functionFoo` and `FunctionFoo` are different) with the exception being that *keyword* function arguments are case-*insensitive*. The latter will be exemplified in sections below.

Recipe scripts support single-line as well as multi-line comments in a C-style syntax:

```
// this is a single-line comment
/* this is
        a multi-line
                    comment */
```

G.2.3 The script’s structure

An FSS script typically contains an `init` section, a few user-defined functions followed by the main function.

A minimal script just contains the `main` function.

```
init // optional
{
    // sensor commands such as $SHZ 1000;
}

fn userDefinedFunc() // optional
{
    // function body
    ...
}

fn main(scanFreq="scanning frequency in Hz") // mandatory
{
    // main function body
    ...
}
```

Similarly to build-in functions described in section G.4, `main` accepts positional and keyword arguments styling.

Whitespace and new-line characters are allowed everywhere, for example, the following are valid constructs:

```
fn
SomeFunc(

)
{    }

loop

{   SomeFunc()
}
```

G.2.4 Mathematical expressions

The arguments of built-in functions might be arbitrary mathematical expressions expr formally defined with the recursive syntax given below:

```
expr :  Expressions  ::=  num
                     |    var
                     |    math_const
                     |    expr + expr
                     |    expr - expr
                     |    expr * expr
                     |    expr / expr
                     |    expr < expr
                     |    expr <= expr
                     |    expr > expr
                     |    expr >= expr
                     |    expr == expr
                     |    expr != expr
                     |    -expr
                     |    ( expr )
                     |    unary_func ( expr )
                     |    binary_func ( expr, expr )
                     |    expr ?  expr : expr ;; math_const
```

The operator precedence and associativity of arithmetic, comparison and ternary operator can be taken from C99 operator precedence. The mathematical constants definition is compilant with the GNU standard. The mathematical functions are named according to cmath C++ library header. All expressions are evaluated in double-precision arithmetic, and then, if necessary, converted to integers (if a specific function argument has an integer type).

G.3 Built-in language constructs

As noted above, built-in commands support positional as well as keyword argument syntax. In the definitions below, i32 denotes a 32-bit signed integer type, and f64 is a 64-bit floating-point type. Optional arguments are indicated by (optional) mark at the end.

G.3.1 The optional `init` section

A script program may contain zero or more `init` sections where the user places CHRocodile native commands to be executed before the script execution starts. Each command string shall start with a dollar sign and end with a semicolon, the syntax is the same as used by the CHRocodile explorer to submit commands. Note that, the actual interpretation of CHRocodile commands happens during the script's **execution** and **not** during its compilation. Therefore, if there is a syntax error in a command string, the script execution will fail. Below is an example of the `init` section which sets device output signals, scanning frequency and some other useful parameters:

```
init
{
    $SODX 256 257 260 82 68 69 65 66 67;
    $SHZ 70000;$LAI 99;$AVD 1;$AVS 1;
    $SRI 1;$ABE 0;$SRT 0;$NOP 1;$POD 0;
    $OFN 0 0;
}
```

The following commands are **forbidden** in the `init` section: ASC, BDR, BIN, IDE, IPCN, MMD, RST, SFD, SSU, SSQ, STO, STA, ULFW, DNLD, TABL. These commands are **ignored** during the script's execution generating a corresponding warning message in a log file.

G.3.2 The function `main()`

```
fn main(
        scanFreq: i32 = 10000 [1000; 100000] /* initial scanning frequency in Hz */
↪   (optional)
        markerDelay: i32 = 500 [100; 50000] /* delay in usec need to set end-figure
↪   marker signal (after a figure has been scanned) */ (optional)
        waitForTrigger: i32 = 1 [0; 1] /* whether waitForTrigger command shall be
↪   added at the beginning of main() */ (optional)
)
```

This defines the program's entry point: each script must contain the one and only function `main`.

G.3.3 User Functions

A user-defined function name identifier must comply with the following regex:

`[_a-zA-Z][_a-zA-Z0-9]*`

and must not match any reserved keywords or built-in function names given in the next section.

User functions are declared without parameters:

```
    fn functionName() {
        // user function code: shape, moveTo, lineTo, startMeasure ....
    }
```

However, a call to such a user-defined function looks like this:

```
userFunc(
        ofsX: f64 = 0: [-32767; 32767] /* X-offset applied to all scanning commands
↪   of this function in mm */ (optional)
        ofsY: f64 = 0: [-32767; 32767] /* Y-offset applied to all scanning commands
↪   of this function in mm */ (optional)
)
```

While function-internal scan commands refer to an origin (0, 0), the two parameters `ofsX`, `ofsY` define where that origin is located in the coordinate system of the caller site. This way, shapes defined inside functions can be placed anywhere by outside code.

G.3.4 Statements

All script statements must be placed within the main or a user-defined function (with the exception of single-line comments). Supported statements / syntactic constructs are listed below.

calls to user-defined functions accepting no arguments

`userDefinedFunc()`

Nested function calls are supported, though the maximal nesting depth is limited to 10.

built-in functions calls

These include commands for scanning geometric primitives and commands to control the system's behavior. Following Python syntax, built-in script commands shall accept positional as well as keyword arguments. Keyword arguments are case-insensitive. E.g., suppose we have a built-in function:

```
foo(x, y, width, height)
```

This function can be called in the following ways which are all equivalent:

```
foo(1.0, 3, 123.44, 777)                    // only positional arguments given
foo(Y=3, Height=777, width=123.44, x=1.0)   // only keyword arguments
foo(1.0, 3, width=123.44, height=777)       // the mixture of positional and keyword
↪  arguments
foo(1.0, 3, Width=123.44, HEIGHT=777)       // the mixture of positional and keyword
↪  arguments in different case
```

Yet, these function calls will produce a compile error:

```
foo(1.0, 3, width=123.44, x=777)            // the argument "x" is specified twice,
↪  the argument "height" is missing
foo(x=1.0, 3, width=123.44, height=777)     // the positional argument is specified
↪  after the keyword argument "x"
foo(1.0, 3, height=777)                     // the agrument "width" is missing
foo(1.0, 3, 123.44, height=777, blabla=331) // unknown keyword argument "blabla"
```

loop statement with a finite number of iterations executed in DSP

```
while(numIters = N)
{
    // loop body limited to N iterations
    ...
}
```

where (N $\leq$ 4096) is an integer specifying the number of loop iterations. The `while`-loops *cannot* be nested. However it is allowed to put a `while`-loop insider an infinite loop presented below.

loop statement with an infinite number of iterations executed in DSP

```
loop
{
    // loop body executing forever
    ...
}
```

`for`-loop statements unrolled during the compilation

```
for i in range(start, end, step)
{
    // loop body executing '(end - start) / step' iterations
    // here i changes as [start, start + step, start + step*2, ... start + step*j ]
    // where j = floor((end - start) / step)
}
```

or with a unit step size:

```
for i in range(start, end)
{
    // loop body executing 'end - start' iterations:
    // here i changes as [start, start+1, start+2, ... end-1]
}
```

or starting with 0 and with a unit step size:

```
for i in range(end)
{
    // loop body executing 'end' iterations:
    // here i changes as [0, 1, 2, ... end-1]
}
```

Note that, `start`, `end` and `step` variables might be arbitrary arithmetic expressions which are then **rounded down** to 32-bit signed integers before loop unrolling takes place.

G.3.5 Z-Correction for thickness signals

Controls the direction (sign) of z-correction for thickness signals

```
correction(
        thicknessID: i32 = 0 [0; 376] /* Thickness signal ID for which correction is
↪   to be applied (256, 264, ..., 376). If thicknessID is set to zero => correction
↪   is applied to *all* 16 thickness signals */ (optional)
        zdir: i32 = 0 [-1; 1] /* z-correction direction for a selected thickness
↪   signal (or to *all* thickness signals if 'thicknessID' is zero) */ (optional)
        keepLinear: i32 = 1 [0; 1] /* whether to use linear terms in X and Y for a
↪   selected thickness signal */ (optional)
)
```

Example:

```
// all scanning functions following this statement will be handled in the same way
↪   until
// new Z-correction statement appears
correction(thicknessID=256, zdir=1, keepLinear=1)

rect(-2,-2,2,2,width=200,height=200, label="myrect")
...
// scanning functions, from now on, will be handled differently
correction(thicknessID=256, zdir=-1, keepLinear=0)
rect(-2,-2,2,2,width=300,height=300, label="myrect2")
ellipse(3.3, 5.6, 20, 100,angle=0, nTurns=2, numPts=10000, label="myellipse"
...
```

G.3.6 The `shape` environment

Shapes are special code blocks that define self-contained scan recipe elements with a user-defined label. All geometric primitives produced within one shape will be treated together as a one single shape where all resulting data samples have the same label ID. This enables the FSS plugin to assign chunks of measurement data to specific shapes (by means of the label ID being part of the data).

A shape can have two attributes attached to it: A shape-specific scan frequency and a label:

```
shape(
        scanFreq: i32 = 10000 [1000; 100000] /* scanning frequency for this block in
↪   Hz */ (optional)
        label: String /* user string label assigned to a geometric figure produced
↪   by this block */ (optional)
)
```

Code example:

```
fn drawPoly() { // scans a hexagon
    let nPts = 100, N = 6, x0 = 0, y0 = 0, R = 2
    shape(label = "poly") {
        moveTo(x0 + R, y0)
        // move to the start position and wait until the scanner stabilizes
        waitUsec(10000)
        startMeasure()
        for i in range(1, N+1) {
            let ang = M_PI*2*i / N
            let X = x0 + R*cos(ang), Y = y0 + R*sin(ang)
```

```
            lineTo(X, Y, nPts)
        }
    }
}
```

Note that the user should manually call `startMeasure` within the shape environment in order to start the scanning process, otherwise no data samples will be collected.

Shapes cannot contain other shapes, therefore only the following statements are allowed within a shape environment (descriptions can be found below):

- `moveTo`
- `lineTo`
- `for-loop`
- `while-loop`
- `startMeasure`
- `stopMeasure`
- `waitTrigger`
- `waitExtAxisPos`
- `waitUsec`

The full list of built-in script commands are given in G.4.

G.3.7 The `interp2D` environment

This block can be used to interpolate arbitrary raw data within a specific rectangular area. The code sample below scans 36 spirals arranged in a grid and interpolates the results within [-15;-15] x [15;15] rectangular area with 500x500 resolution:

```
fn main(scanFreq=20000)
{
    let w = 30, h = 30, xs = -w/2, ys = -h/2, N = 6
    let stepX = w/N, stepY = h/N
    interp2D(xs, ys, xs + w, ys + h, nCols=500, nRows=500, innerRad=5, outerRad=5,
↪   label="spirals") {
        for yi in range(N) {
            for xi in range(N) {
                spiral(xs + (xi + 0.5)*stepX, ys + (yi + 0.5)*stepY, a=0, b=0.1,
↪   nTurns=5, numPts=5000)
            }
        }
    }
}
```


© Precitec Optronik GmbH

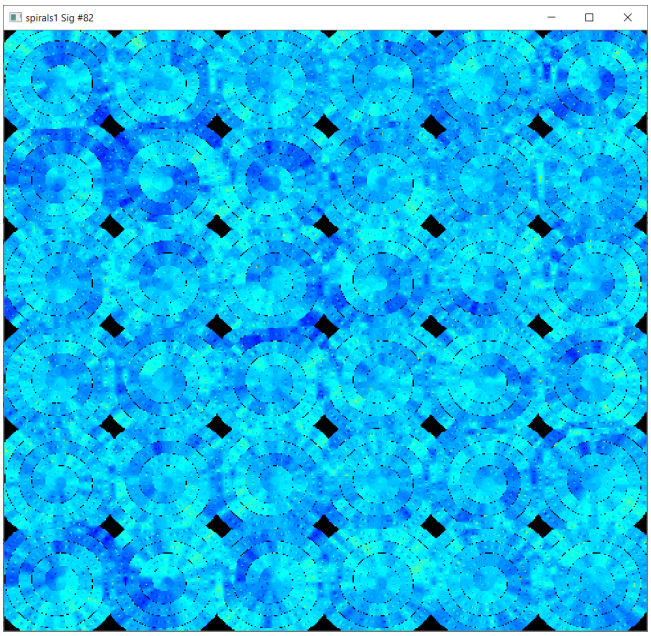


Figure G.3: 2D interpolated set of spirals

G.3.8 DSP embedded control commands

There is a set of special built-in script commands which do not trigger scanning process but but serve to control internal CHRocodile's behavior. These commands are:

- `exposure`
- `adaptiveExposure`
- `doubleExposure`
- `avd`
- `avs`
- `qth`
- `enc`
- `add`
- `clear`

For detailed description, see the reference section G.4. The byte-code generated by these commands is directly interpreted by DSP and triggers the desired changes immediately.

Though, **add** and **clear** commands worth particular attention: these commands are only allowed within a special **dwd** environment block, and are used to add or clear CHRocodile detection windows (please consult CHRocodile command reference for further details). All **add** and **clear** commands are aggregated into a single DSP command packet. The detection window changes take effect immediately by leaving the **dwd** block and stay active until the next **dwd** block comes during program execution.

The **dwd** block may also contain for-loop statements and variable definitions. All other commands/statements are *prohibited* within this block. For example, the following code snippet activates 16 equidistant detection windows, and then scans an ellipse shape:

 © Precitec Optronik GmbH

```
fn dwdTest()
{
    dwd {
        let num = 16, L = 0, R = 5500, wofs = 100
        let wsz = (R - L - (num + 1)*wofs) / num
        for i in range(num) {
            let left1 = i * (wsz + wofs) + wofs
            add(left=left1, right=left1 + wsz, wndID=i)
        }
    }
    ellipse(x0=0.0000, y0=0.0000, radX=15.0000, radY=5.0000, angle=0.0000,
                nTurns=1.0000, numPts=7500,waitAtBegin=10000, label="ellipse1")
}
```

G.3.9 Rectangular scan interpolation algorithm details

For interpolation we use a modification of the *Inverse distance weighting* (IDW) algorithm. The reason for this is that well-known methods for scattered data interpolation like thin-plate splines or those based on `Delaunay` triangulation are quite computationally demanding in that finally the whole set of points being interpolated contribute to each interpolated value. Furthermore, these approaches would close all gaps between the scattered data points no matter how far away these points are located. Yet, it might be the case that there is indeed a hole in an object being scanned with the `FlyingSpot`: Such an area certainly does not need to be filled up with interpolated values. In contrast, the IDW algorithm can be modified in such a way that only a local subset of data points contribute to each interpolated value. Using some heuristics described below, we can also control how the data gaps shall be closed. For that matter, we have introduced two configurable parameters, inner circle radius and outer circle radius as shown in the figure below.

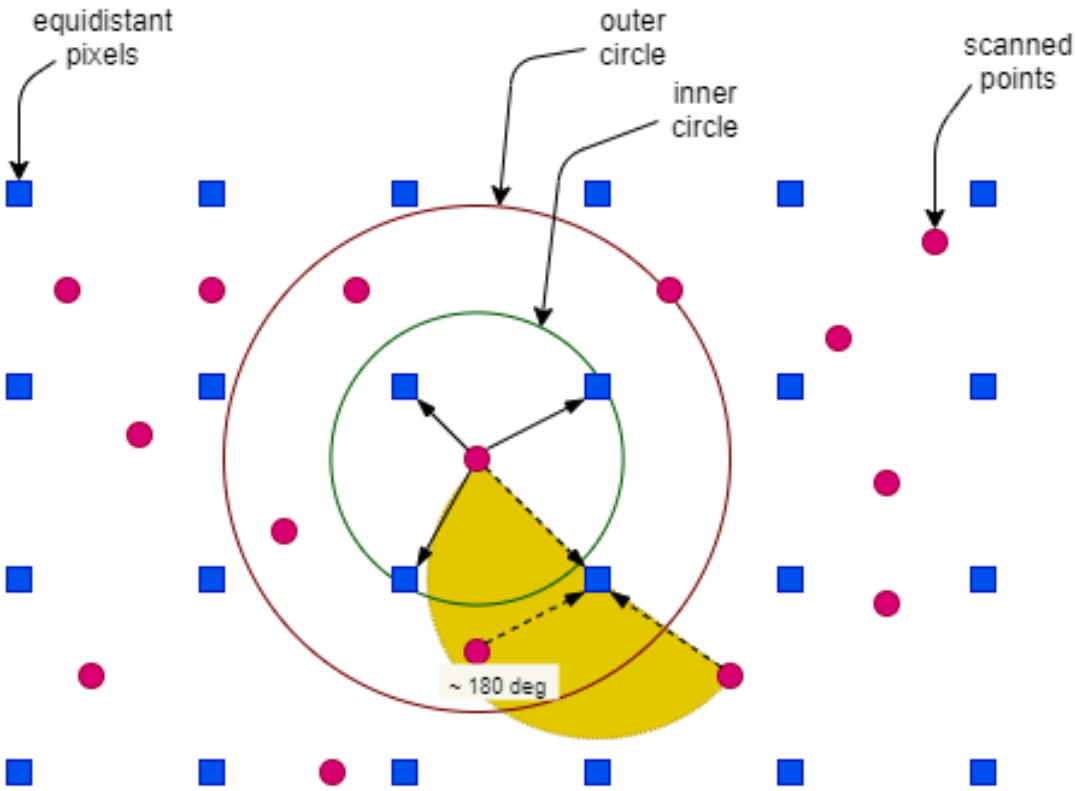


Figure G.4: Interpolation algorithm details

The algorithm works as follows:

© Precitec Optronik GmbH

- for each scanned data point S (depicted by small red circles), we "draw" (consider) two concentric circles centered at this point
- if there are any pixels (shown as blue squares in the figure) lying within the inner circle, then the selected data point immediately contribute to the interpolated value of all these pixels (its contribution is weighed by the inverse distance to each pixel)
- otherwise, we check if there are any pixels within the outer circle. If there are, for each such pixel we compute the minimal angle spanned by the two most distant (opposite) scanned data points (see the yellow "pie" in the figure above). If this angle is not less than 180 degrees, the pixel being considered gets the contribution from S
- if there are no pixels inside the outer circle of S, or the angle spanned by two most distant scanned data points is less than than 180 degrees (meaning that the scanned points approach this pixel from one side only), then this pixel is assigned a NaN (invalid) value (indicating that there is a possible hole in an object being scanned).

Here we provide some considerations regarding the inner and outer radius settings:

- the inner radius must be smaller than or equal the outer radius
- increasing the inner radius increases the running time of the scanner algorithm (sometimes considerably)
- the outer radius setting controls on how good "object holes" will be closed

Below there are some interpolation examples with different settings (both radii are given in pixels):

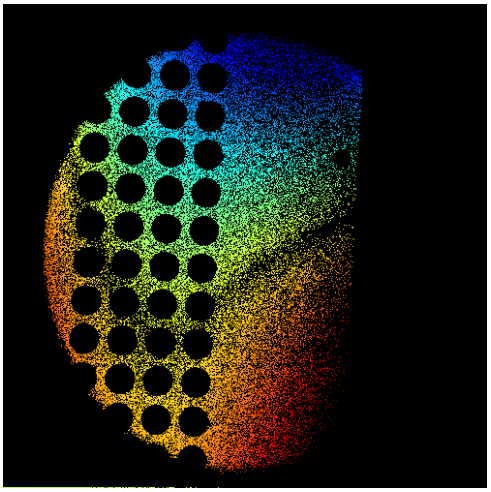


Figure G.5: innerRad=0.5 outerRad=0.5 (interpolation is disabled)

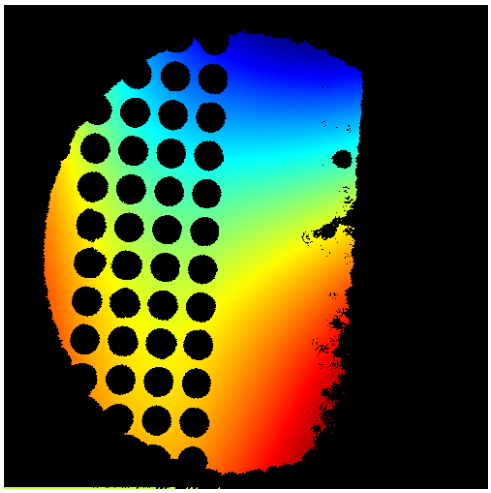


Figure G.6: innerRad=1 outerRad=5

© Precitec Optronik GmbH

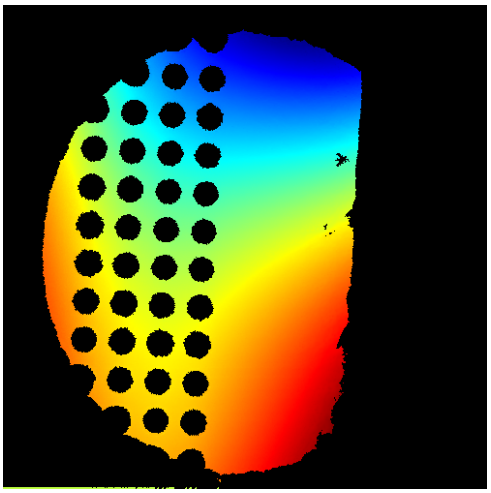


Figure G.7: innerRad=1 outerRad=10

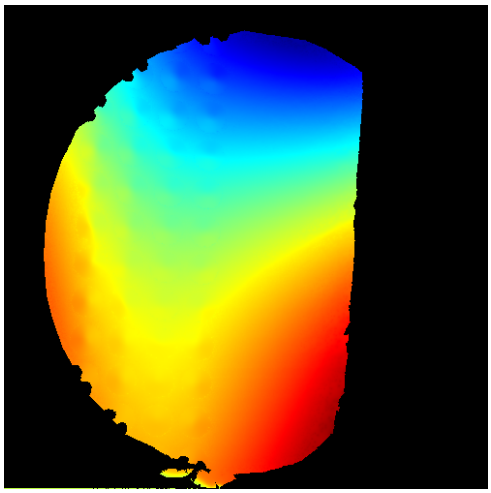


Figure G.8: innerRad=1 outerRad=20

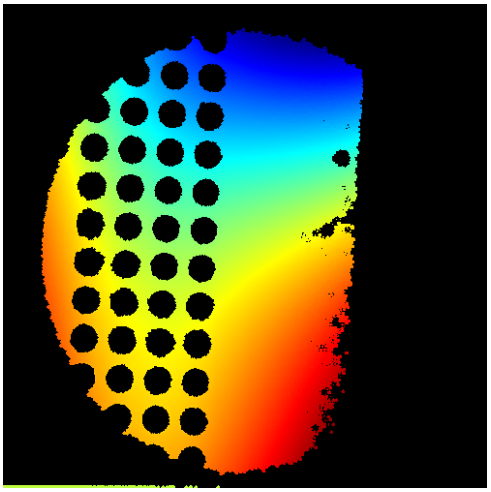


Figure G.9: innerRad=2 outerRad=5

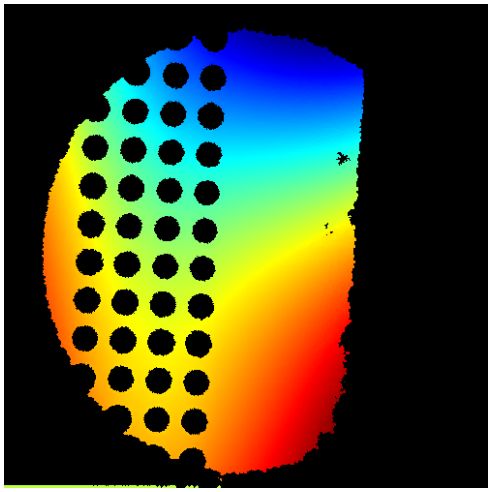


Figure G.10: innerRad=2 outerRad=10

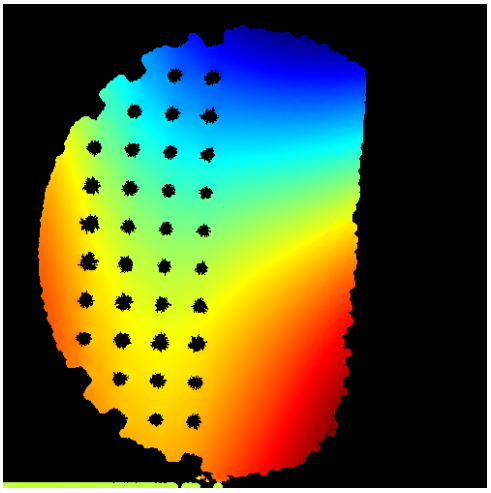


Figure G.11: innerRad=5 outerRad=15

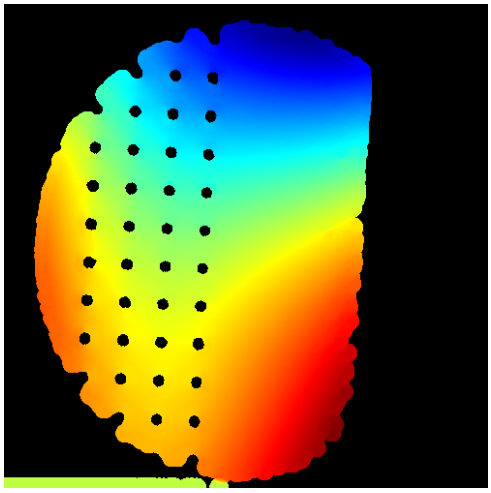


Figure G.12: innerRad=10 outerRad=10

© Precitec Optronik GmbH

G.3.10 FSS Plugin usage

G.3.10.1 Executing plugin script

In order to compile and start an FSS script executing on the device, one has to run the following commands (see FSS command reference in Section **??**):

- execute `CmdID_FlyingSpot_Cfg` command **once** upon plugin initialization. Though, one can run the CFG command mulitple times (when the XML configuration file has been changed), but most of the times this is not necessary.
- compile one or more scripts with `CmdID_FlyingSpot_Compile`. FSS Plugin keeps a small database of 16 most recently compiled programs, so that one can execute precompile the scripts in advance for faster processing.
- start executing a previously compiled script with `CmdID_FlyingSpot_Exec` command. Once the next geometric shape is completely scanned, the user will be notified by a callback function with the `FSS_PluginShapeData` structure containing the shape data. Upon script termination, the user receives a callback with the figure of type `FSS_PluginRecipeTerminate` in the corresponding field of `FSS_PluginShapeData` structure.
- it is allowed to prematurely stop execution of one script and start another one, simply by sending a new `CmdID_FlyingSpot_Exec` command (while previous script is being running)
- in order to forcefully stop a current script execution, one can use `CmdID_FlyingSpot_Stop` command which switches the FSS Plugin back to the idle state

G.3.10.2 Configuration file

The FSS config file (called **ScannerGlobalConfig.cfg** by default) contains the following relevant parameters:

UseXYCoeffsFromDevice : whether XY Field correcton coefficients shall be downloaded from device's non-volatile memory or taken from the config file (must be 0)

UseZCoeffsFromDevice : whether Z-correction coefficients shall be downloaded from device's non-volatile memory or taken from the config file (must be 1)

ZCorrectionCoefficientsIn32k : Z-correction coefficients to be used when UseZCoeffsFromDevice = **0**

X/YScannerFieldCorrectionCoefficientsRWTo32k : XY-correction coefficients (Real-World to 32K conversion) to be used when UseXYCoeffsFromDevice = **0**

X/YScannerFieldCorrectionCoefficients32kToRW : XY-correction coefficients (32K to Real-World conversion) to be used when UseXYCoeffsFromDevice = **0**

ScalingFactorRWto32k : scaling factor (Real-World to 32K Encoder conversion) defining how many 16-bit encoder ticks corresponds to 1 mm (this depends on the device)

As a rule of thumb, there are currently 3 different values of the scaling factor depending on the Flying Spot device type:

FSS40 : ScalingFactorRWto32k = **732**

FSS80 : ScalingFactorRWto32k = **366**

FSS310 : ScalingFactorRWto32k = **199**

 © Precitec Optronik GmbH

G.3.10.3 How the plugin recognizes different figures and setMarker command

The FSS Plugin distinguishes individual geometric shapes (figures) by looking at Encoder V signal (called **marker**) during the runtime. In the compilation phase, each figure found in a script is assigned a unique marker value (starting from 1). Then, during the script's execution, the plugin maps a marker value to a corresponding geometric shape (thereby deciding which data samples belong to which figure).

There are, however, a few ***special cases*** where this automatic figure recognition fails. Namely, it happens when there is just a ***single*** shape placed in a **while** loop, an infinite loop or in a funtion call. Indeed, if we have a loop with a single shape inside, the plugin will get the **same** marker value over and over again during the loop iterations. Hence, all shapes produced within the loop will be basically "glued together" into a single one. And this is where the **setMarker** command comes into play: this command induces a short delay (controlled by the 'markerDelay' parameter of the function **main**) and sets the marker to some different value which serves as shapes delimiter. The user is allowed to set negative or zero markers (positive marker values are ***reserved*** for the compiler). Note that, if there is **waitExtAxisPos** or **waitTrigger** command inside the loop, then the explicit call to **setMarker** is no longer necessary (since these wait commands reset the marker value automatically). These cases are exemplified below:

```
fn main()
{
    while(10) {
        // WRONG: ellipses will be merged together since there is no marker
        ↪  delimiter in between
        ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
    }

    loop {
        // WRONG: lines will be merged together since there is no marker delimiter
        ↪  in between
        line(-10, -10, 10, 10, numPts=1000, label="lineX")
    }
}

fn main(markerDelay = 1000) {   // setting markerDelay explicitly if needed
    while(10) {
        // CORRECT: setMarker() delimits the individual shapes
        setMarker(0)
        ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
    }

    loop {
        // CORRECT: setMarker() delimits the individual shapes
        setMarker(0)
        line(-10, -10, 10, 10, numPts=1000, label="lineX")
    }
}

fn main()
{
    while(10) {
        // CORRECT: waitTrigger resets the marker => no setMarker() is needed
        ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
        waitTrigger()
    }

    loop {
        // CORRECT: waitExtAxisPos resets the marker => no setMarker() is needed
        line(-10, -10, 10, 10, numPts=1000, label="lineX")
        waitExtAxisPos()
    }
}

fn main()
    while(10) {
        // CORRECT: setMarker() is not needed since we have at least 2 alternating
        ↪  shapes in a loop
        ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
```

```
        line(-10, -10, 10, 10, numPts=1000, label="lineX")
    }

    loop {
        // CORRECT: setMarker() is not needed since we have at least 2 alternating
        ↪  shapes in a loop
        ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
        line(-10, -10, 10, 10, numPts=1000, label="lineX")
    }
}
```

The same applies for user functions when such a function produces a ***single*** geometric shape. However, the user should decide here himself, if such a function might be called from a **while**, an infinite loop, (or just called several times one after the other). For instance:

```
fn drawMyEllipse()
{
    ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
}

fn main()
{
    while(10) {
        // WRONG: ellipses will be merged together since there is no marker
        ↪  delimiter in between
        drawMyEllipse(x0=5, y0=5); // draw ellipse with some offset
    }

    loop {
        // WRONG: ellipses will be merged together since there is no marker
        ↪  delimiter in between
        drawMyEllipse(x0=5, y0=5); // draw ellipse with some offset
    }
}
```

Correct usage:

```
fn drawMyEllipse() {
    ellipse(0, 0, 10, 20, angle = 0, nTurns=2, label="EllX")
}

fn drawMyLine() {
    line(-10, -10, 10, 10, numPts=1000, label="lineX")
}

fn main()
{
    // CORRECT: these functions are not called from within the loop
    drawMyLine(11, 11)
    drawMyEllipse(x0=10, y0=15);

    while(10) {
        // CORRECT: setMarker() delimits the individual shapes
        setMarker(0)
        drawMyEllipse(x0=5, y0=5); // draw ellipse with some offset
    }

    loop {
        // CORRECT: setMarker() delimits the individual shapes
        setMarker(0)
        drawMyEllipse(x0=5, y0=5); // draw ellipse with some offset
    }
}
```

G.4 Script Command Reference

G.4.1 adaptiveExposure

Sets the adaptive exposure (lamp intensity closed-loop controlled)

```
adaptiveExposure(
        shz: f64: [32; 3.40282e+38] /* sample rate (sensor measurements, NOT scanner
↪  frequency) */
        aal: f64: [0; 100] /* Adaptive Level (0..100%) */
)
```

G.4.2 add

Sets the ith detection window: up to 16 windows (0..15) can be set. This command is only allowed within **dwd**{ ... } environment block. If left > right, the boundaries for the ith window will be swapped. If left == right (within a small tolerance), the ith window will be **disabled**. if wndID == -1, sets the same boundaries for **all** 16 windows; this can be used to quickly reset all detection windows.

```
add(
        left: f64: [0; 30000] /* ith detection window, left limit in micrometer */
        right: f64: [0; 30000] /* ith detection window, right limit in micrometer */
        wndID: i32 = 0 [0; 15] /* The number of detection window to set */
↪  (optional)
)
```

G.4.3 avd

Sets the current data average factor

```
avd(
        factor: i32 [0; 999] /* Data averaging factor */
)
```

G.4.4 avs

Sets the current spectra average factor

```
avs(
        factor: i32 [1; 999] /* Spectra averaging factor */
)
```

G.4.5 clear

Disables (clears) the ith detection window. This command is only allowed within **dwd**{ ... } environment block. If wndID == -1, all 16 windows will be disabled, and the full spectrum will be used for peak detection.

```
clear(
        wndID: i32 = -1 [-1; 15] /* The number of detection window to set */
↪  (optional)
)
```

G.4.6 correction

Controls a direction (sign) of z-correction for thickness/median signals. This command sets a special marker value (EncV) for a given amount of time (controlled by 'markerDelay' parameter of the function **main**)

© Precitec Optronik GmbH

```
correction(
        signalID: i32 = 0 [0; 380] /* Thickness/median signal ID for which
↪  correction is to be applied (256, 260, 264, ..., 380). If signalID is set to
↪  zero => correction is applied to *all* 32 thickness and median signals */
↪  (optional)
        zdir: i32 = 0 [-1; 1] /* z-correction direction for a selected
↪  thickness/median signal (or to *all* thickness and median signals if 'signalID'
↪  is zero) */ (optional)
        keepLinear: i32 = 1 [0; 1] /* whether to use linear terms in X and Y during
↪  Z-polynomial evaluation for a selected signal */ (optional)
        keepOffset: i32 = 0 [0; 1] /* whether to use zero-offset coeff during
↪  Z-polynomial evaluation for a selected signal */ (optional)
)
```

G.4.7 doubleExposure

Sets the double exposure (extended dynamic range)

```
doubleExposure(
        shz: f64: [32; 3.40282e+38] /* sample rate (sensor measurements, NOT scanner
↪  frequency) */
        lai: f64: [0; 100] /* Lamp intensity (0..100%) */
        dcy: f64: [0; 100] /* Duty cycle (0..49%) */
)
```

G.4.8 ellipse

Scans a 1D path given by a (rotated) ellipse equation

```
ellipse(
        x0: f64: [-32767; 32767] /* center X coordinate of an ellipse in mm */
        y0: f64: [-32767; 32767] /* center Y coordinate of an ellipse in mm */
        radX: f64: [0.001; 32767] /* radius in X-direction of an ellipse in mm */
        radY: f64: [0.001; 32767] /* radius in Y-direction of an ellipse in mm */
        angle: f64: [-360; 360] /* angle (CCW) between the coordinate X-axis and the
↪  X-radius of an ellipse in degrees */
        nTurns: f64: [0.001; 1000] /* number of 360 degrees turns around an ellipse
↪  center: could be fractional */
        numPts: i32 [2; 100000] /* total number of points to scan */
        waitAtBegin: i32 = 10000 [0; 1000000] /* waiting time before scanning the
↪  first point of the figure (in usec) */ (optional)
        label: String /* user string label assigned to this geometric figure */
↪  (optional)
)
```

G.4.9 enc

Configures encoder of specified axis

```
enc(
        axis: i32 [0; 4] /* target axis */
        function: i32 [0; 4] /* encoder function index */
        value: i32 [-2147483648; 2147483647] /* value */
)
```

G.4.10 exposure

Sets the fixed exposure (fixed lamp intensity)

```
exposure(
        shz: f64: [32; 3.40282e+38] /* sample rate (sensor measurements, NOT scanner
↪  frequency) */
        lai: f64: [0; 100] /* Lamp intensity (0..100%) */
)
```


© Precitec Optronik GmbH

G.4.11 fn main

Definition of the function main() where the program execution begins

```
fn main(
        scanFreq: i32 = 10000 [1000; 100000] /* initial scanning frequency in Hz */
↪  (optional)
        markerDelay: i32 = 500 [100; 50000] /* delay in usec need to set end-figure
↪  marker signal (after a figure has been scanned) */ (optional)
        waitForTrigger: i32 = 1 [0; 1] /* whether waitForTrigger command shall be
↪  added at the beginning of main() */ (optional)
)
```

G.4.12 help

Prints out all built-in commands help to the console in markdown format

```
help(
)
```

G.4.13 initSectionItem

Specifies a single CHRocodile command to be executed on program startup. The following commands are **not** allowed within the 'init' section and will **not** be executed: **ASC**, **BDR**, **BIN**, **IDE**, **IPCN**, **MMD**, **RST**, **SFD**, **SSU**, **SSQ**, **STO**, **STA**, **ULFW**, **DNLD**, **TABL**

```
initSectionItem(
        CHR command: String /* CHR command starting with $ sign */ (optional)
)
```

G.4.14 interp2D

Aggregates all scanned data (within this block) that fall into a given rectangle (x0; y0) - (x1; y1) and performs 2D interpolation with a resolution nCols x nRows. The global signals in the range [64.. 78] are **not** interpolated: i.e., not included from the set of output signals for this shape.

```
interp2D(
        x0: f64: [-32767; 32767] /* top-left corner X of a scanned rectangle in mm
↪  (x0 < x1) */
        y0: f64: [-32767; 32767] /* top-left corner Y of a scanned rectangle in mm
↪  (y0 < y1) */
        x1: f64: [-32767; 32767] /* bottom-right corner X of a scanned rectangle in
↪  mm */
        y1: f64: [-32767; 32767] /* bottom-right corner Y of a scanned rectangle in
↪  mm */
        nCols: i32 [8; 15000] /* horizontal resolution of the resulting bitmap in
↪  pixels */
        nRows: i32 [8; 15000] /* vertical resolution of the resulting bitmap in
↪  pixels */
        innerRad: f64 = 3: [0.125; 30] /* interpolation inner radius in pixel
↪  fractions (innerRad <= outerRad) */ (optional)
        outerRad: f64 = 10: [0.125; 30] /* interpolation outer radius in pixel
↪  fractions */ (optional)
        label: String /* user string label assigned to this geometric figure */
↪  (optional)
)
```


© Precitec Optronik GmbH

G.4.15 line

Scans a 1D path given by a straight line segment

```
line(
        x1: f64: [-32767; 32767] /* beginning X-coordinate of a line segment in mm
↪   */
        y1: f64: [-32767; 32767] /* beginning Y-coordinate of a line segment in mm
↪   */
        x2: f64: [-32767; 32767] /* last X-coordinate of a line segment in mm */
        y2: f64: [-32767; 32767] /* last Y-coordinate of a line segment in mm */
        numPts: i32 [2; 100000] /* total number of points to scan */
        waitAtBegin: i32 = 10000 [0; 1000000] /* waiting time before scanning the
↪   first point of the figure (in usec) */ (optional)
        label: String /* user string label assigned to this geometric figure */
↪   (optional)
)
```

G.4.16 lineTo

Scans a 1D path given be a line segment starting at the last moveTo/lineTo position within **shape** block. This command is *only* allowed within **shape**{ . . . } environment block

```
lineTo(
        x: f64: [-32767; 32767] /* X-coordinate of a line segment's last point to be
↪   scanned in mm */
        y: f64: [-32767; 32767] /* Y-coordinate of a line segment's last point to be
↪   scanned in mm */
        numPts: i32 [5; 100000] /* total number of points to scan */
)
```

G.4.17 moveTo

Sets the scanning position to a point with coordinates (x,y). This command is *only* allowed within **shape**{ . . . } environment block

```
moveTo(
        x: f64: [-32767; 32767] /* X-coordinate of a point to be scanned in mm */
        y: f64: [-32767; 32767] /* Y-coordinate of a point to be scanned in mm */
)
```

G.4.18 qth

Sets the current detection threshold

```
qth(
        threshold: f64: [0; 1000] /* detection threshold (0..1000) */
)
```

G.4.19 rect

Performs a 2D rectangular area scan (with optional interpolation). The global signals in the range [64.. 78] are **not** interpolated: i.e., not included from the set of output signals for this shape.

```
rect(
        x0: f64: [-32767; 32767] /* top-left corner X of a scanned rectangle in mm
↪   (x0 < x1) */
        y0: f64: [-32767; 32767] /* top-left corner Y of a scanned rectangle in mm
↪   (y0 < y1) */
        x1: f64: [-32767; 32767] /* bottom-right corner X of a scanned rectangle in
↪   mm */
```

```
        y1: f64: [-32767; 32767] /* bottom-right corner Y of a scanned rectangle in
↪  mm */
        nCols: i32 [8; 15000] /* horizontal resolution of the resulting bitmap in
↪  pixels */
        nRows: i32 [8; 15000] /* vertical resolution of the resulting bitmap in
↪  pixels */
        innerRad: f64 = 3: [0.125; 30] /* interpolation inner radius in pixel
↪  fractions (innerRad <= outerRad) */ (optional)
        outerRad: f64 = 10: [0.125; 30] /* interpolation outer radius in pixel
↪  fractions */ (optional)
        interp: i32 = 1 [0; 1] /* whether the resulting data shall be interpolated
↪  with equidistant pixels */ (optional)
        scanHoriz: i32 = 1 [0; 1] /* whether to perform scan in row-major or
↪  column-major order */ (optional)
        waitAtBegin: i32 = 10000 [0; 1000000] /* waiting time at the start of each
↪  scan line (in usec) */ (optional)
        waitAtEnd: i32 = 500 [0; 1000000] /* waiting time at the end of each scan
↪  line  (in usec) */ (optional)
        label: String /* user string label assigned to this geometric figure */
↪  (optional)
)
```

G.4.20 setFrequency

Sets the current scanning frequency

```
setFrequency(
        scanFreq: i32 [1000; 100000] /* scanning frequency in Hz */
)
```

G.4.21 setMarker

Sets the special marker value (EncV) for a given amount of time (controlled by 'markerDelay' parameter of the function **main**). Positive marker values are forbidden since they are reserved by the compiler for regular geometric shapes. This command is required when, for example, a ***single*** figure is scanned withina **while**() { . . . } loop, an infinite loop, or within a function call.

```
setMarker(
        marker: i32 [-10000000; 0] /* marker value */
)
```

G.4.22 shape

Defines a *scanner box*, i.e., a special code block which can contain for/while loops and low-level scanning primitives like **moveTo** or **lineTo**. No *high-level* commands like **rect** or **spiral** are allowed inside this block

```
shape(
        scanFreq: i32 = 10000 [1000; 100000] /* scanning frequency for this block in
↪  Hz */ (optional)
        label: String /* user string label assigned to a geometric figure produced
↪  by this block */ (optional)
)
```

G.4.23 spiral

Scans a 1D path produced by an Archimedean spiral: R(phi) = a + b*phi

```
spiral(
        x0: f64: [-32767; 32767] /* center X coordinate of a spiral in mm */
        y0: f64: [-32767; 32767] /* center Y coordinate of a spiral in mm */
        a: f64: [-32767; 32767] /* offset of a spiral in mm */
        b: f64: [0.001; 32767] /* slope of a spiral in mm: R(phi) = a + b*phi */
```

```
        nTurns: f64: [0.001; 1000] /* number of 360 degree turns of a spiral: could
↪  be fractional */
        numPts: i32 [2; 100000] /* total number of points to scan */
        waitAtBegin: i32 = 10000 [0; 1000000] /* waiting time before scanning the
↪  first point of the figure (in usec) */ (optional)
        label: String /* user string label assigned to this geometric figure */
↪  (optional)
)
```

G.4.24 startMeasure

Starts collecting data samples from the next scanning command

```
startMeasure(
)
```

G.4.25 stopMeasure

Stops collecting data samples from the next scanning command

```
stopMeasure(
)
```

G.4.26 userFunc

Performs a user function call where 'userFunc' is an identifier of some function defined as: `fn userFunc() { ... }`

```
userFunc(
        ofsX: f64 = 0: [-32767; 32767] /* X-offset applied to all scanning commands
↪  of this function in mm */ (optional)
        ofsY: f64 = 0: [-32767; 32767] /* Y-offset applied to all scanning commands
↪  of this function in mm */ (optional)
)
```

G.4.27 waitExtAxisPos

Stops recipe execution until the external trigger position is reached

```
waitExtAxisPos(
        pos: i32 [0; 1000] /* external position axis number to wait for */
)
```

G.4.28 waitTrigger

Stops recipe execution until the trigger (STR) command comes

```
waitTrigger(
)
```

G.4.29 waitUsec

Stops recipe execution for a given time in microseconds

```
waitUsec(
        usec: i32 [0; 14316557] /* time in microseconds to wait for */
)
```


© Precitec Optronik GmbH

G.4.30 while

Defines a loop with a given number of iterations: in contrast to **for** loops, **while** loop statements are **not** unrolled but executed directly in DSP. Note that, **while** loops *cannot* be nested.

```
while(
        numIters: i32 [1; 4096] /* the number of loop iterations */
)
```


© Precitec Optronik GmbH